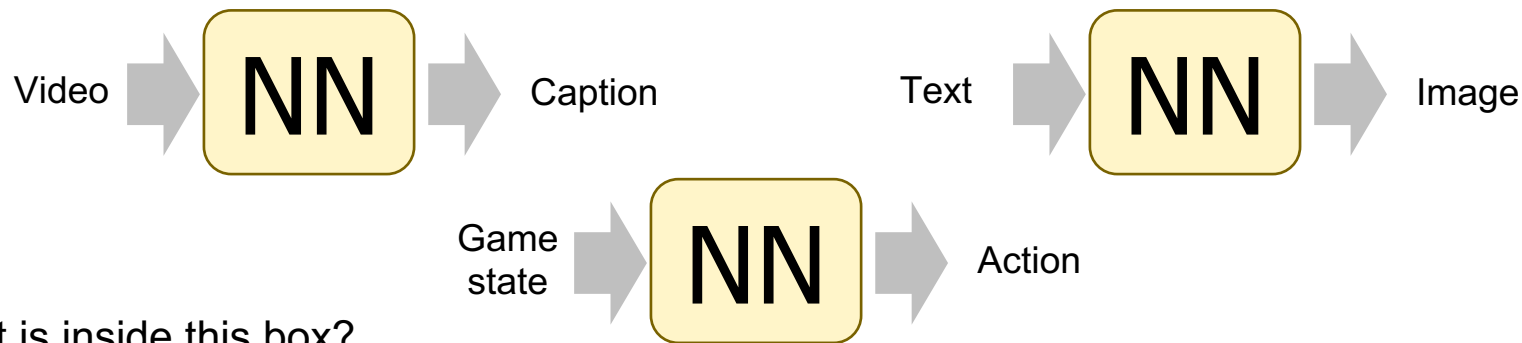


Neural Networks

CSCI 601-471/671 (NLP: Self-Supervised Models)

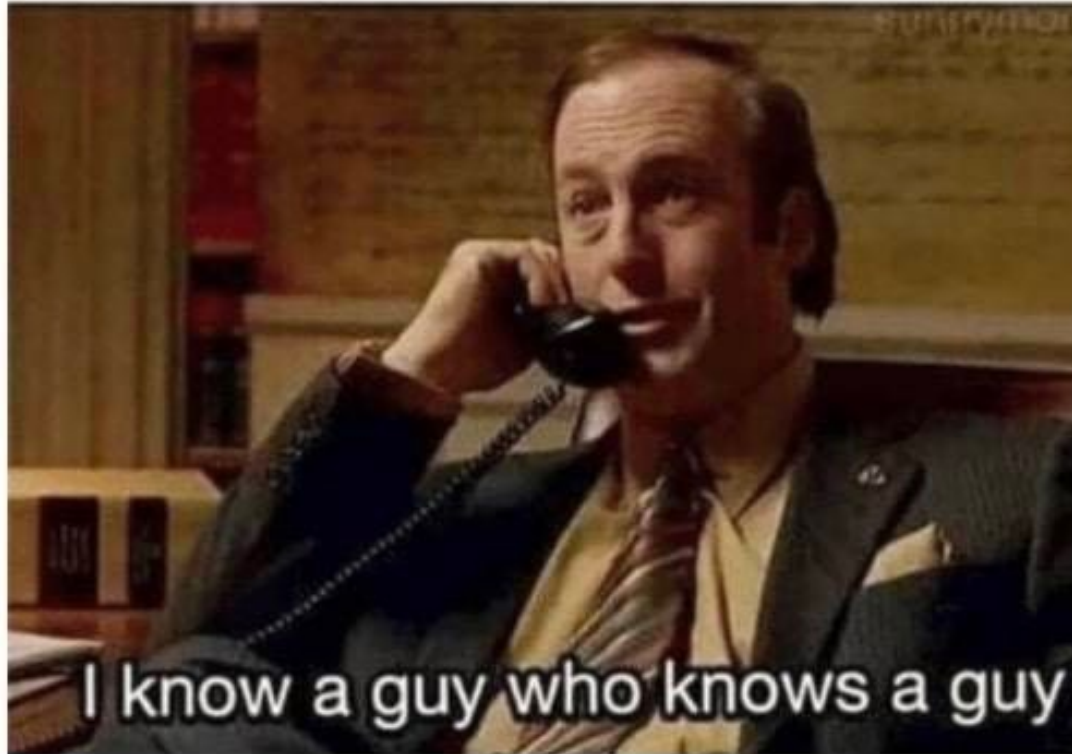
Neural Networks

- What are neural networks?
 - Functions that take an input and produce an output.



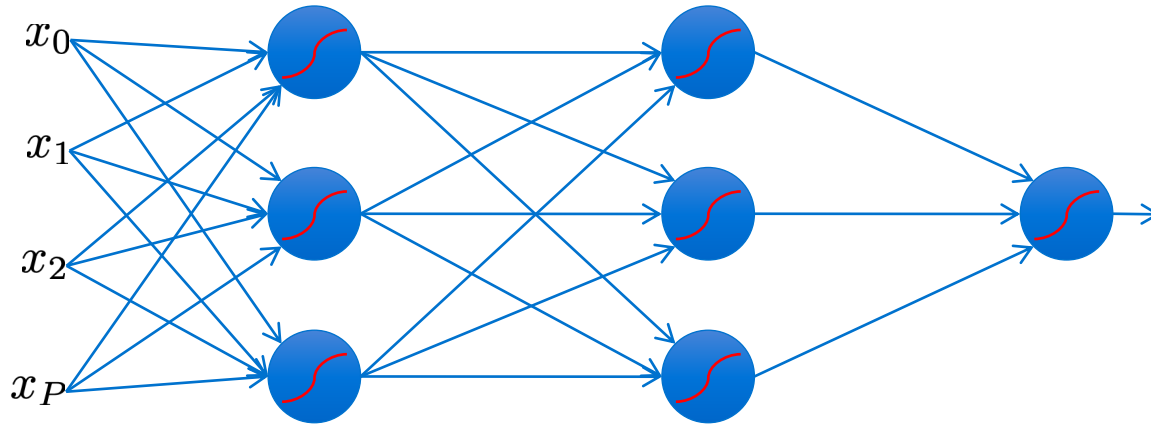
- What is inside this box?

What's Inside Neural Nets?



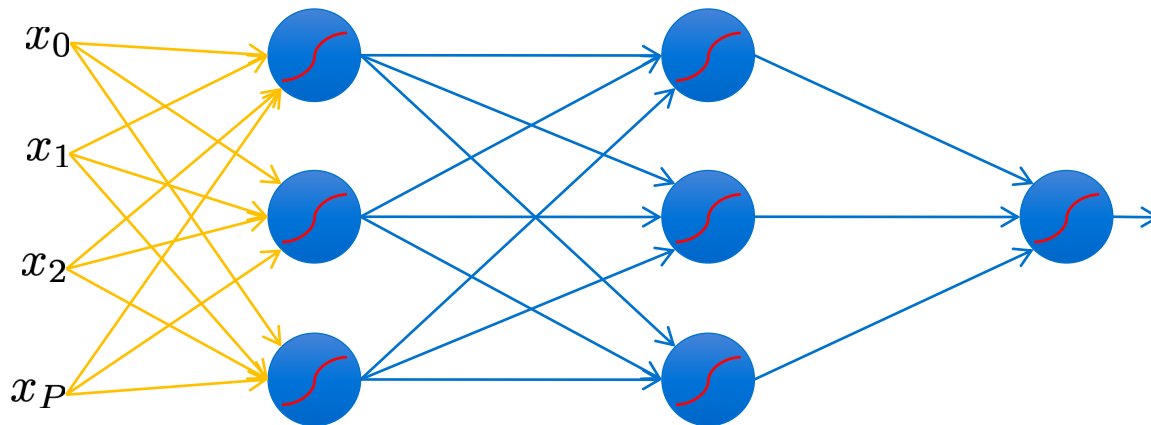
Feedforward networks

- This is a particular class called “feedforward” networks.
 - Information flows through a directed acyclic graph, *with no feedback cycles*.



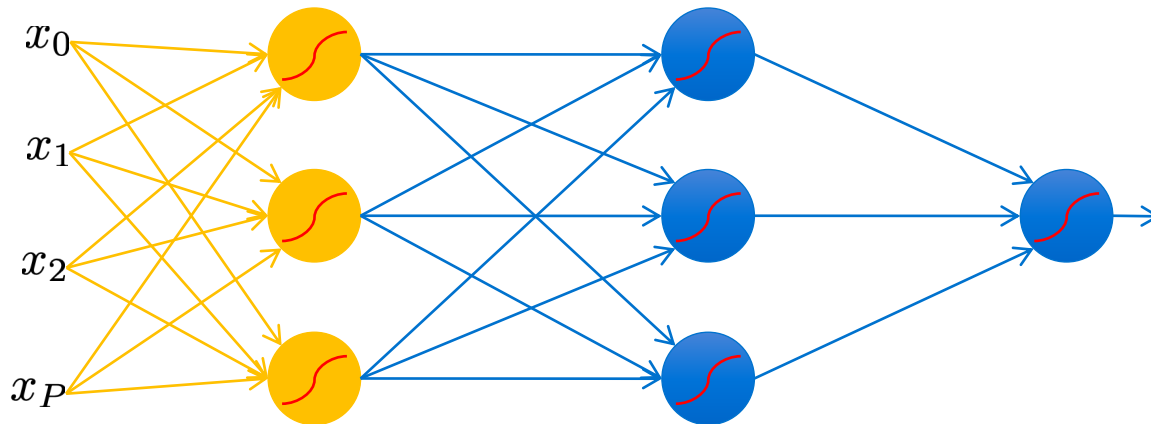
Feedforward networks

- Inputs multiplied by initial set of weights.



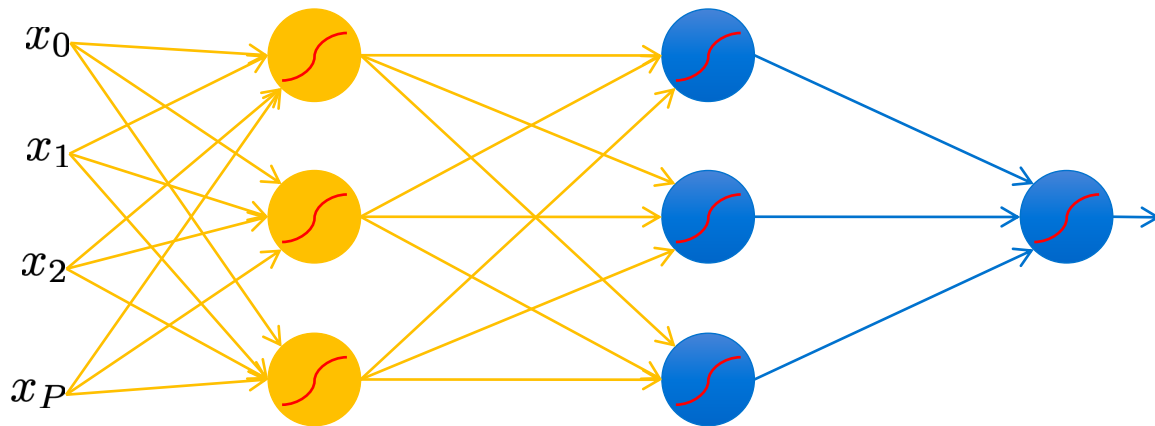
Feedforward networks

- Intermediate “activations” computed at first hidden layer.



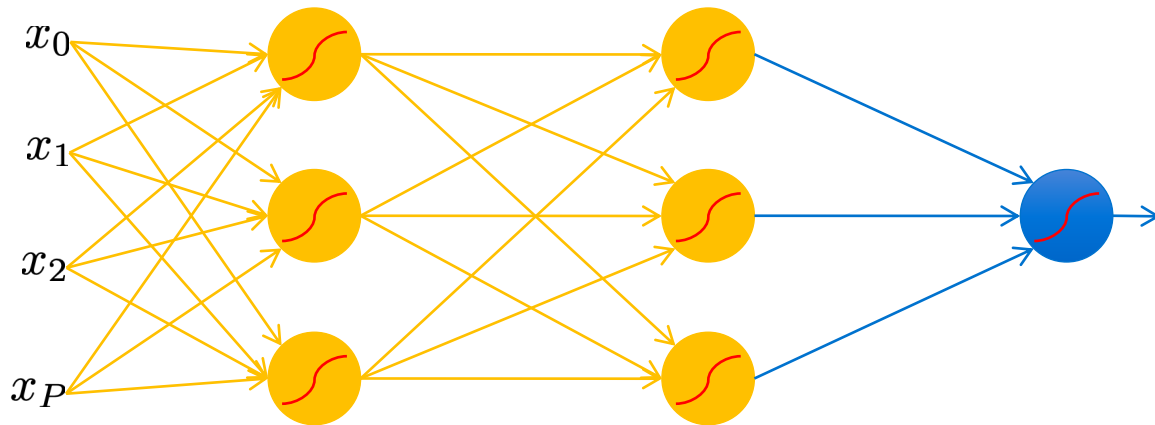
Feedforward networks

- Intermediate activations multiplied by second layer of weights.
- Activations are passed to the next layer.



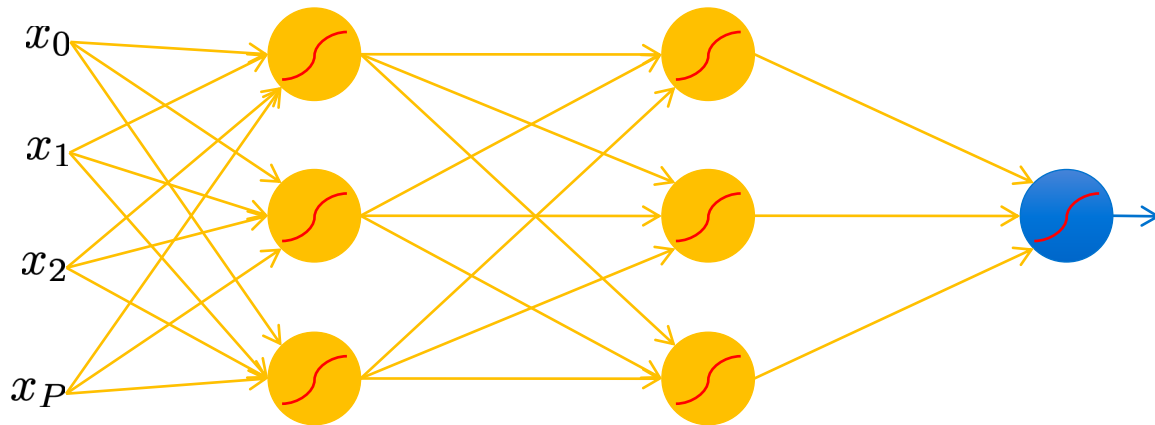
Feedforward networks

- Compute second set of intermediate activations



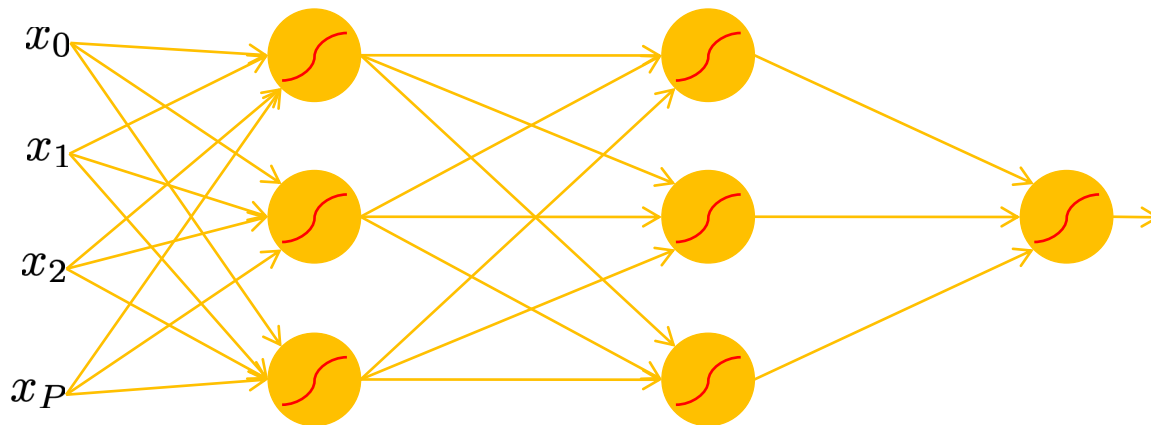
Feedforward networks

- Multiply by final set of weights



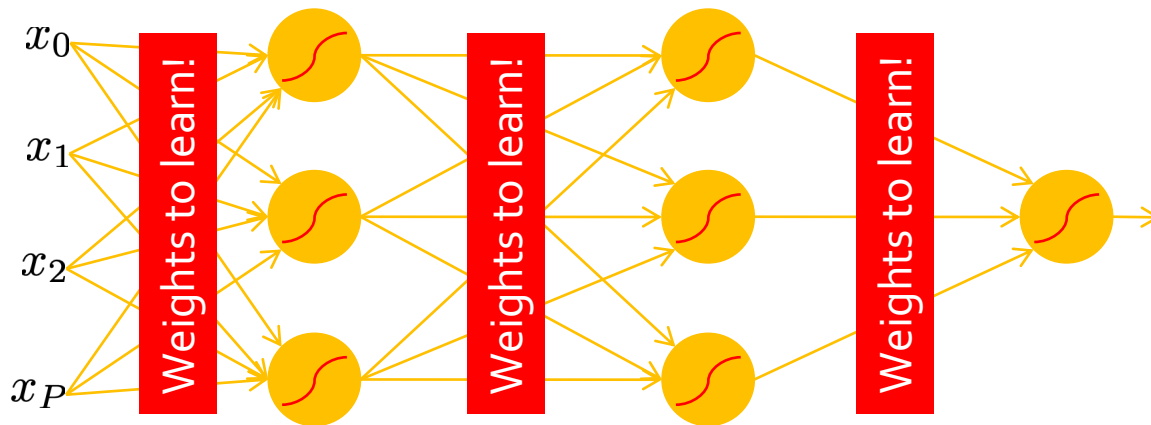
Feedforward networks

- Aggregate all the computations in the output
 - e.g., probability of a particular class



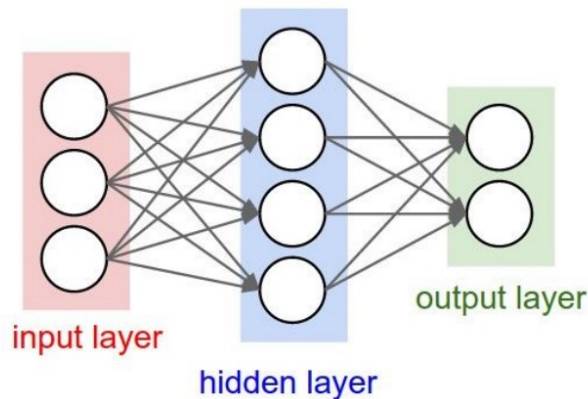
Feedforward networks

- All the intermediate parameters are ought to be learned.



Feedforward Neural Network

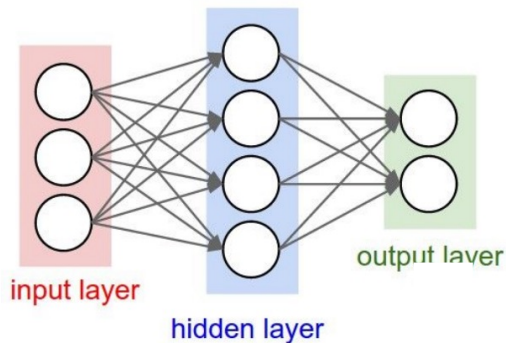
- Neural Networks are functions!
 - **Function class** for approximating **real-valued**, **discrete-valued** and **vector valued** target functions.
 - NN: $\mathbf{X} \rightarrow \mathbf{Y}$ where $\mathbf{X} = [0,1]^n$, or $\mathbb{R}^n$ and $\mathbf{Y} = [0,1]^d, \{0,1\}^d$
- Example: A **2-layer** neural network
 - The input, hidden and output **variables** are represented by **nodes**
 - The links are the **weight parameters**
 - Arrows denote **direction of information flow** through the network



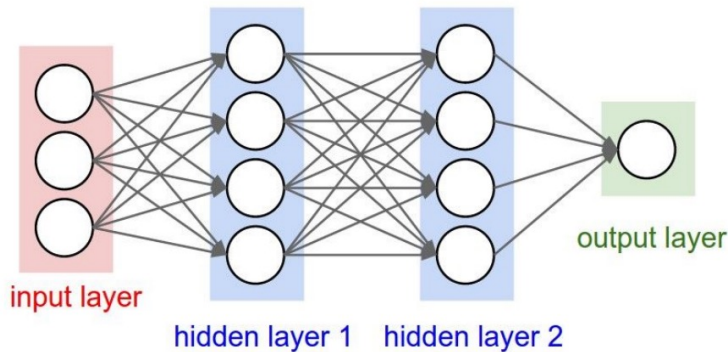
Neural Network: Making it bigger

Add more layers, or wider layers!

A **2-layer** neural network



A **3-layer** neural network

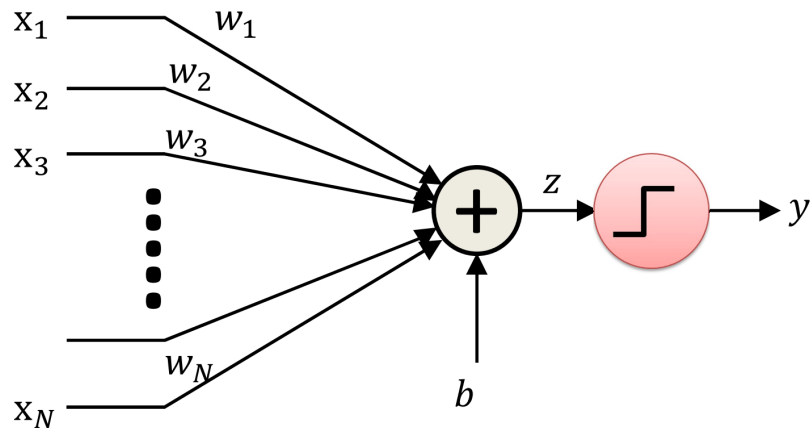
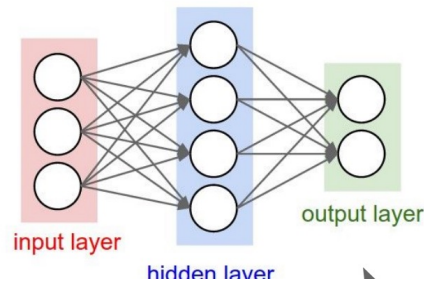


Feedforward Neural Network: The Neurons

- A specific formulation of neuron is “**perceptron**”.
- It consists of a non-linear function that “fires” if the affine (linear) function of *inputs* is above a *threshold*.

$$y = f\left(b + \sum_{i=1}^N w_i x_i\right)$$

$$f(z) = \mathbf{1}[z \geq 0] \text{ (threshold function)}$$

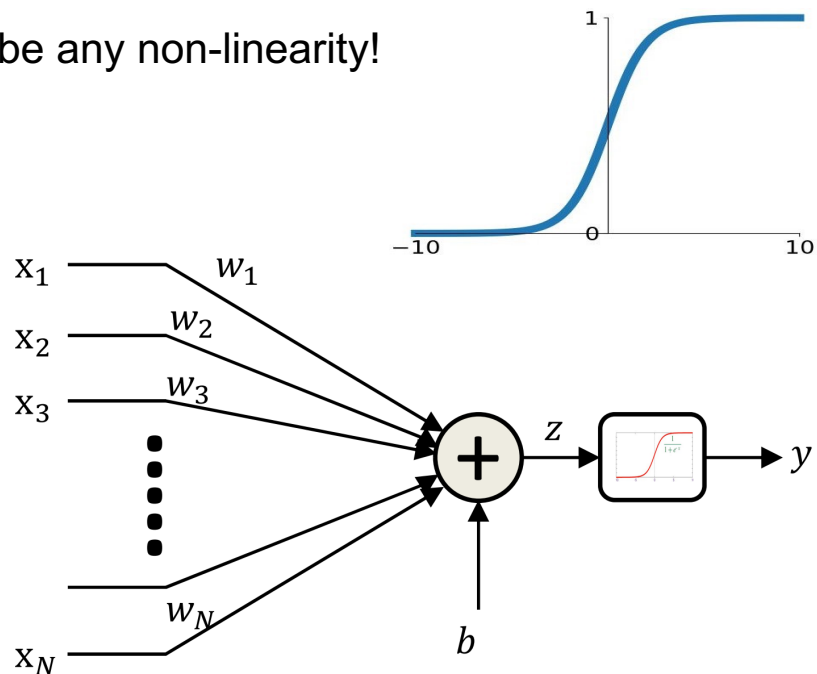


Feedforward Neural Network: The Neurons

- But more broadly, the activation functions can be any non-linearity!
- Sigmoid is a “squashing” function.
 - It maps small inputs to zero.
 - It maps large inputs to one.

$$y = \sigma \left(b + \sum_{i=1}^N w_i x_i \right)$$

$$\sigma(z) = \frac{1}{1+e^{-x}} \text{ (sigmoid function)}$$

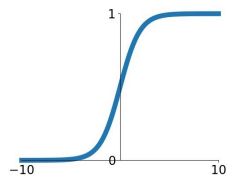


Other Activation Functions

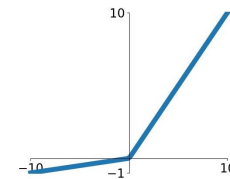
Does not always have to be a squashing function

Sigmoid

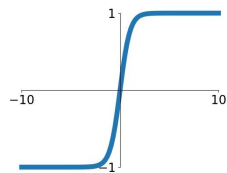
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



Leaky ReLU
 $\max(0.1x, x)$



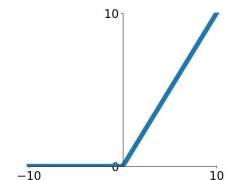
tanh
 $\tanh(x)$



Maxout

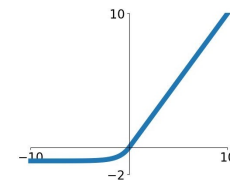
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ReLU
 $\max(0, x)$



ELU

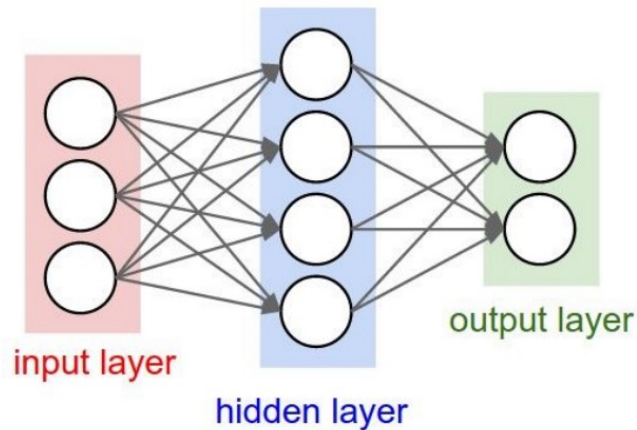
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



We will talk about their pro/cons later!

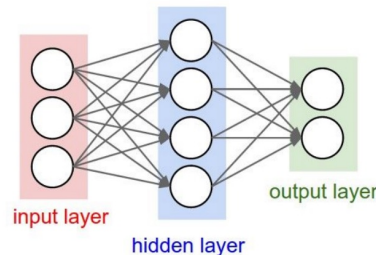
Terminology: Multi-Layer Perceptron (MLP)

- Multi-layer Perceptron (MLP):
 - A feedforward network of fully connected between layers.
- A feedforward network does **not** have to be an MLP.
 - But people sometimes use the names interchangeably! 🙄



Formally Defining a 2 Layer Network

- Example: A **2-layer** MLP network
 - The input, hidden and output **variables** are represented by **node**
 - The links are the **weight parameters**
 - Arrows denote **direction of information flow** through the network



$$f(\mathbf{x}) = W_2 g(W_1 \mathbf{x} + b_1) + b_2 \quad \mathbf{x} \in \mathbb{R}^n, \mathbf{y} \in \mathbb{R}^d$$

$$g(\mathbf{z}) = [\sigma(z_1), \dots, \sigma(z_h)] \text{ (nonlinearity)} \quad \sigma(z_i) = \frac{1}{1+e^{-x}} \text{ (sigmoid function)}$$

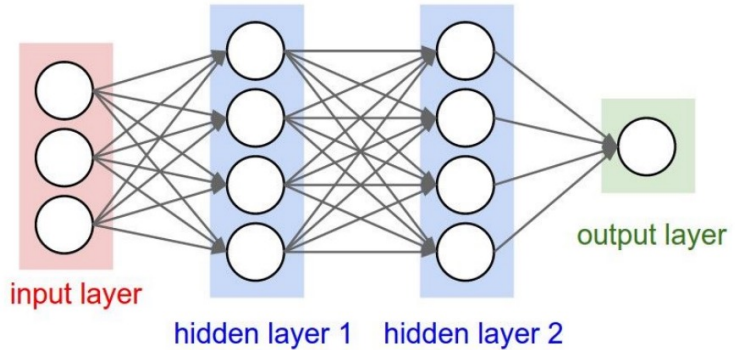
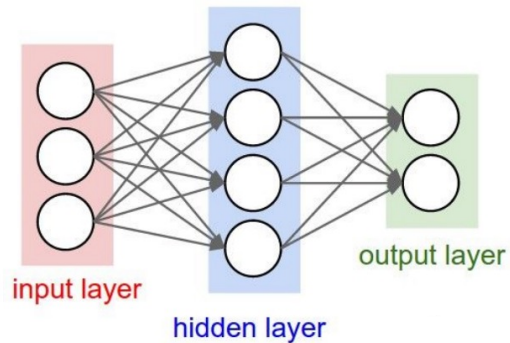
- $W_1 \in \mathbb{R}^{h \times n}$, $W_2 \in \mathbb{R}^{d \times h}$, $b_1 \in \mathbb{R}^h$, $b_2 \in \mathbb{R}^d$ are the **params** that need to be learned.
- The bias terms are often dropped for simplicity.

Quiz Time (1)

- What is needed to fully specify a neural network?
 1. Architecture (which input goes through what function etc.)
 2. Parameters of the function (the weights)
 3. Both

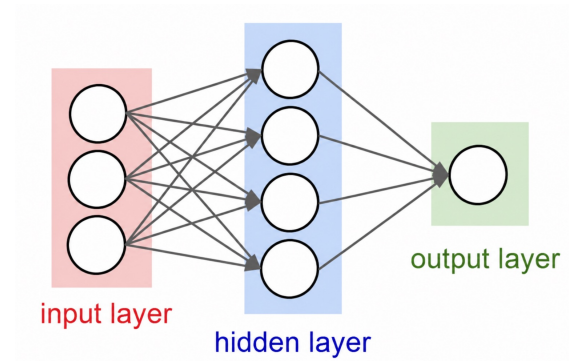
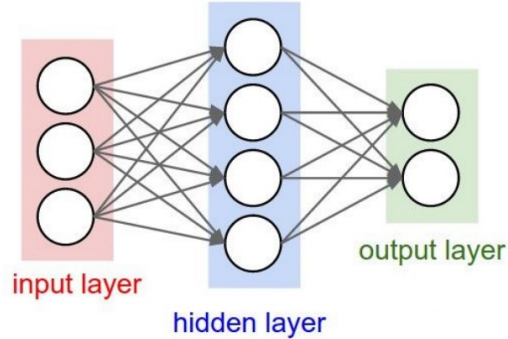
Quiz Time (2)

- Which of the followings has more parameters?



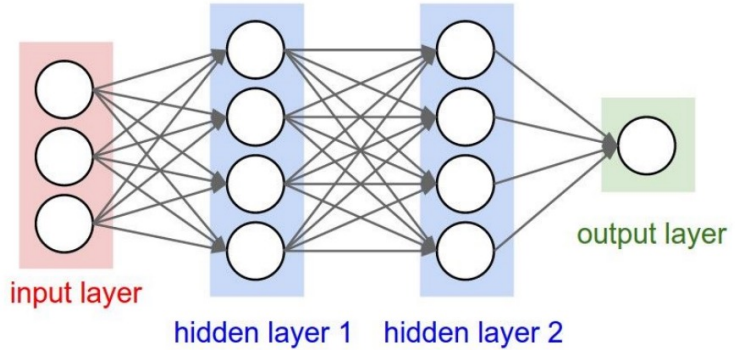
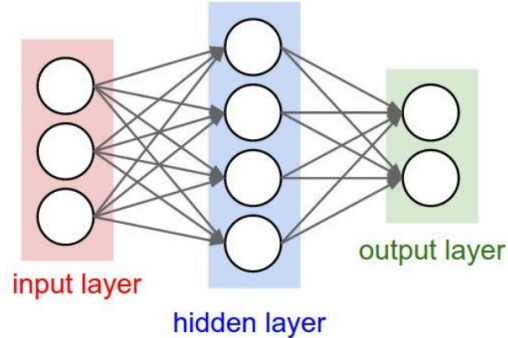
Quiz Time (3)

- Given an input to these models, which of them take longer to compute an output?

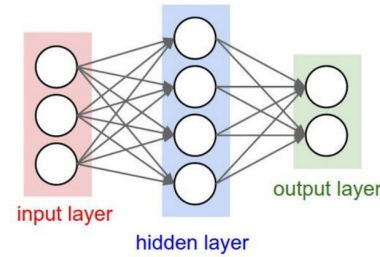


Quiz Time (4)

- Given an input to these models, which of them take longer to compute an output?



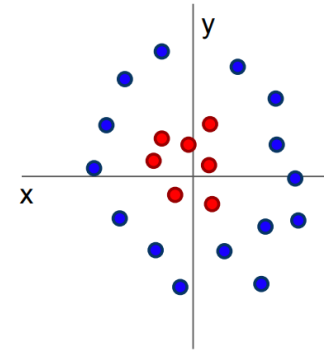
Why Add Non-linearity?



- Without non-linearity, the overall model amounts to a linear model.

$$f(\mathbf{x}) = W_2 g(W_1 \mathbf{x}) \quad \xrightarrow{\text{drop } g} \quad \tilde{f}(\mathbf{x}) = W_2 W_1 \mathbf{x} = W_3 \mathbf{x} \quad (\text{a linear function})$$

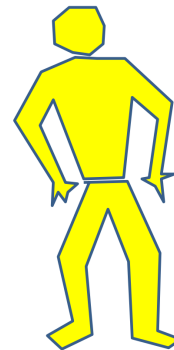
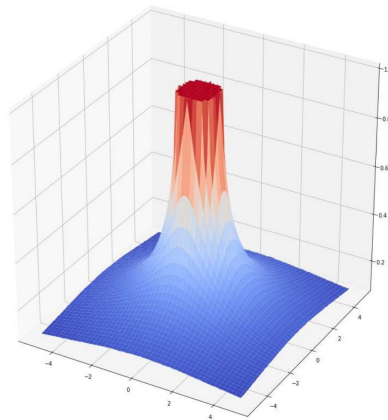
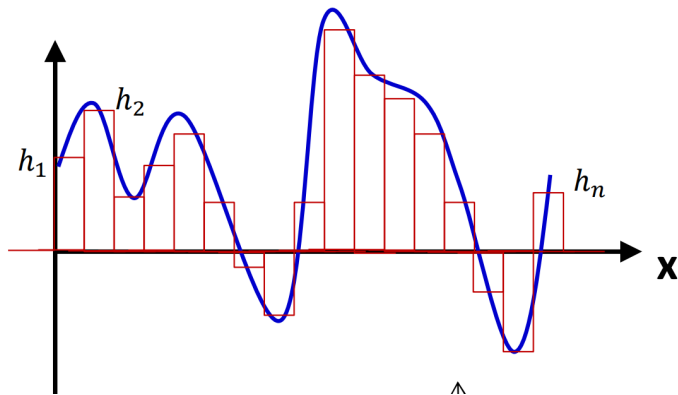
- A linear function cannot approximate complex tasks.
- Non-linearity **adds capacity** to the model to approximate **any** continuous function to **arbitrary** accuracy given sufficiently many hidden units.
 - See [“universal approximation theorem”](#)



Cannot separate red and blue points with linear classifier

Universal Approximation

- An MLP **can** represent **any function** (with a compact domain), with **enough** expressivity.
 - **Enough** expressivity := appropriate activation, sufficient width and depth.
 - (though, expressivity does not guarantee efficient optimization)



Maierov & Pinkus. Lower bounds for approximation by MLP neural networks, 1999.
Cybenko. Approximation by Superpositions of a Sigmoidal Function, 1989.

Quiz Time

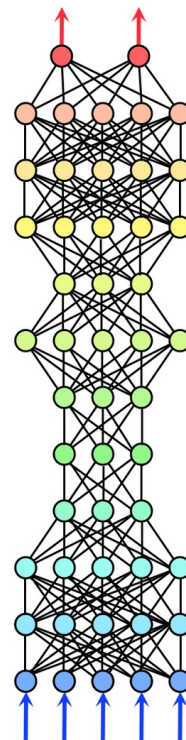
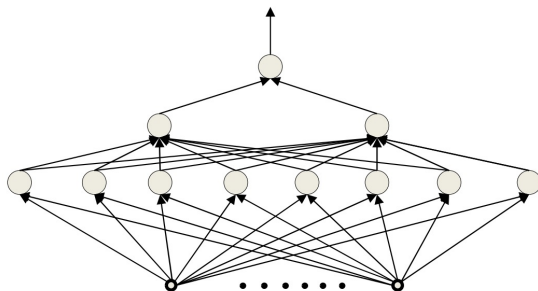
- What makes neural networks expressive functions?
 1. Activations (non-linearities)
 2. Depth (number of hidden layers)
 3. Width (number of variables in each hidden layer)
 4. All the above

Demo time!

- Link: <https://playground.tensorflow.org/>

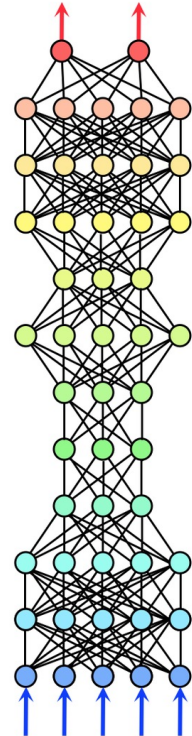
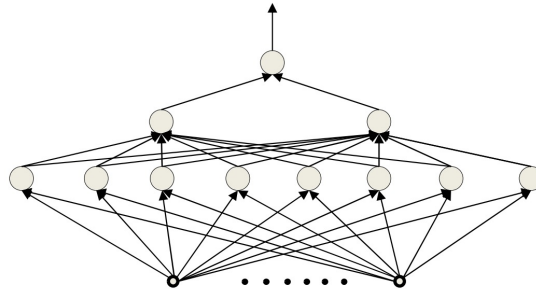
What is a good architecture? Depth vs. Width

- Architectural parameters of a neural network affect its capacity to learn.
 - Deep vs. wide



Depth vs Width tradeoff

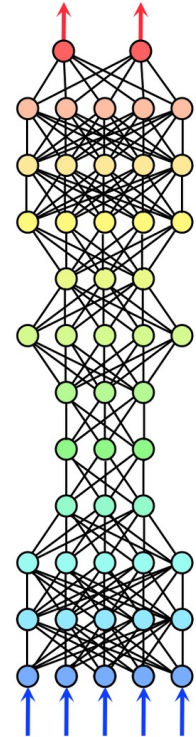
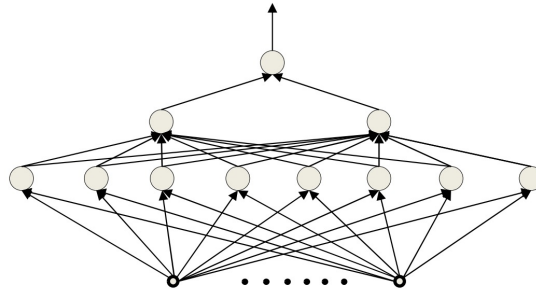
- An MLP is a universal **Boolean** function.
- A **shallow** (single hidden layer) is a universal Boolean machine
 - But it may require an **exponentially large** number of units.
- **Deeper** networks may require far **fewer** neurons than shallower networks to express the same function



Depth vs Width tradeoff

- **Theorem:** There are certain class of functions with n inputs that can be represented with a particular class of **deep** neural networks with $O(n)$ units, whereas it would require at least $2^{\sqrt{n}-1}$ units for a **shallow** network.

("a particular class" refers to sum-product networks—out of scope for us)



Hastad, Almost optimal lower bounds for small depth circuits, 1986.
Delalleau & Bengio. Shallow vs. deep sum-product networks, 2011.

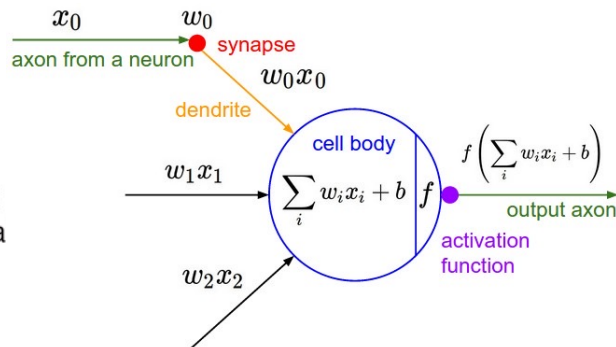
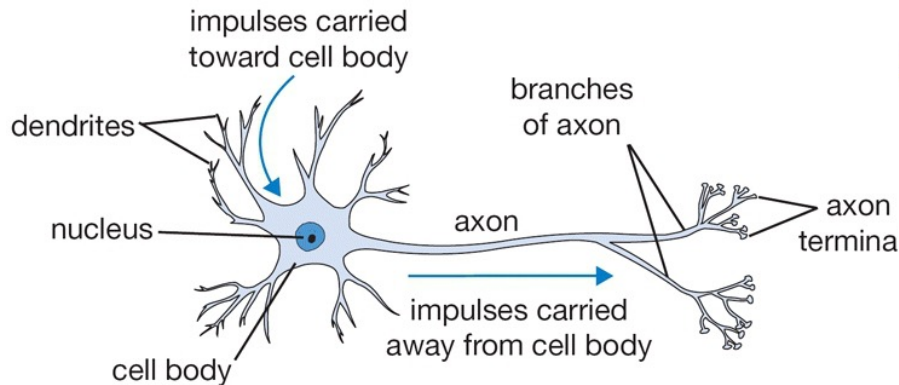
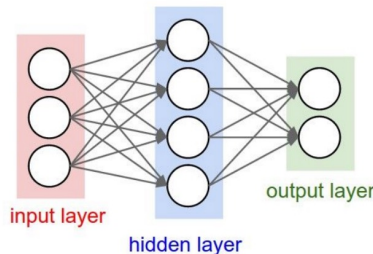
Summary

- An MLP is a universal function
- But can represent a given function with
 - sufficient width
 - sufficient depth
- Depth can be traded off for (exponentially larger) width
- Optimal width and depth depend on the complexity of the problem.
- **Next:** A bit of history.

Neural Nets: Origin and History

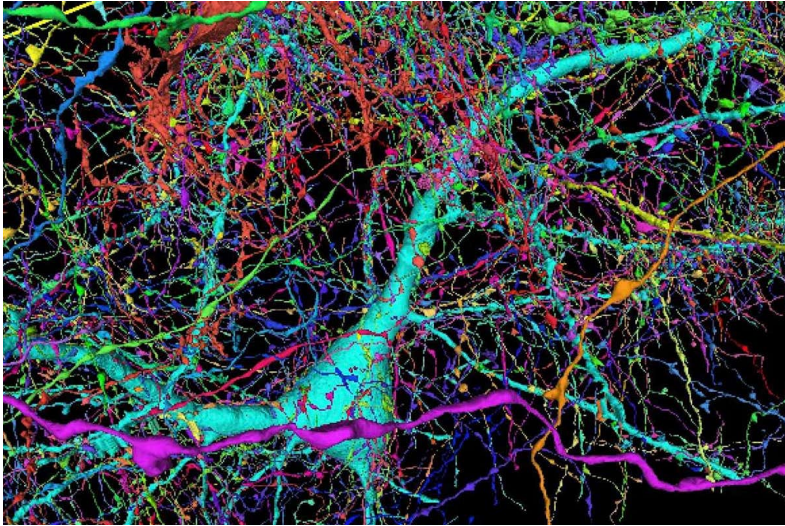
Artificial Neurons: An Inspiration from Nature

- A single node in your neural network
 - Accept information from multiple inputs
 - Transmit information to other neurons
- A neuron's function is inspired by its biological counterpart:
 - Apply some function on inputs signals
 - If output of function over threshold, neuron “fires”



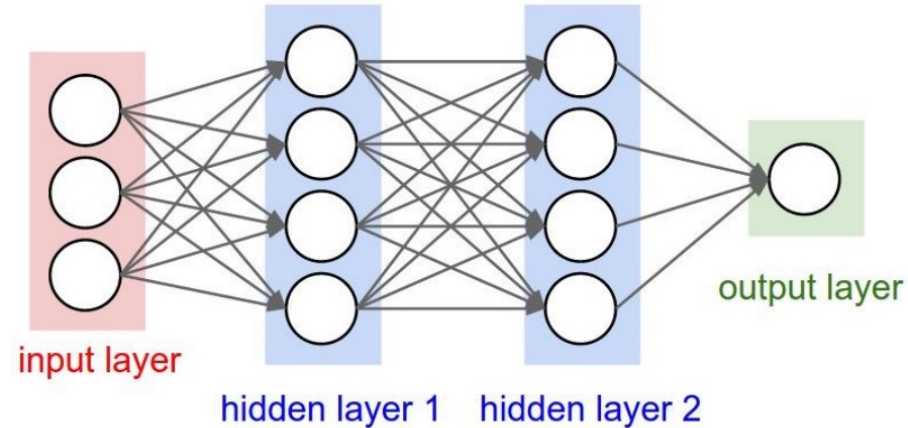
Artificial Neurons: Not Quite Analogous to Nature

Biological neurons:
complex connectivity



Source: Google Brain Map

Neurons in an artificial neural network:
organized based on a highly **regular**
structure for computational efficiency



Very Brief History of Neural Networks

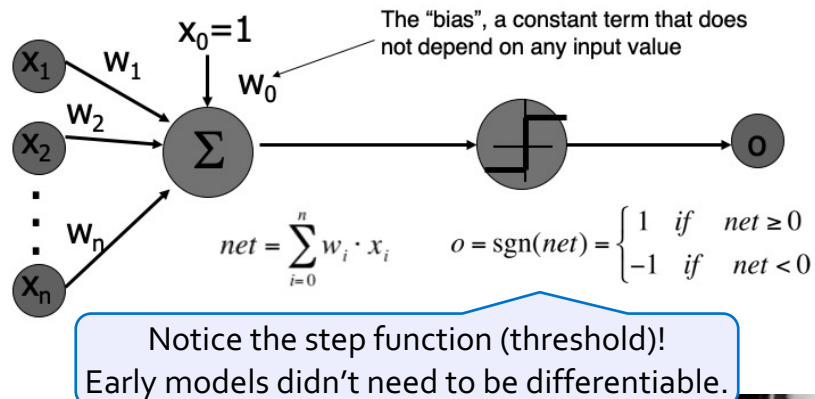
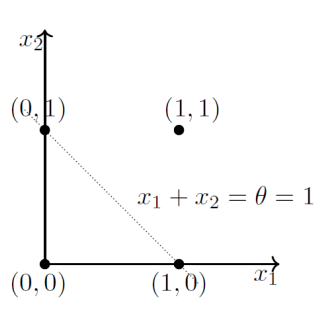
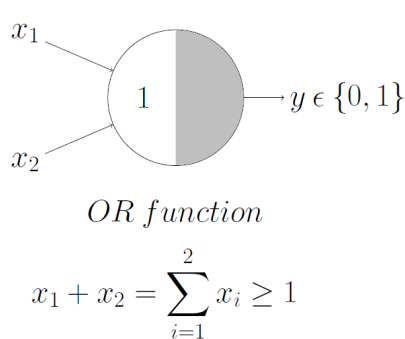
1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

Very Brief History of Neural Networks

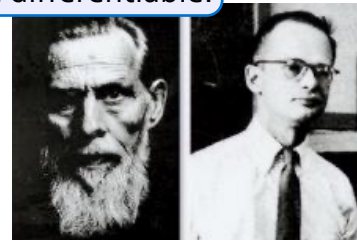
1. **Single-layer neural networks (1943-1969)**
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

A Neuron as a Mathematical Model of Computation

- **McCulloch and Pitts (1943)** showed how **linear threshold units** can be used to compute logical functions

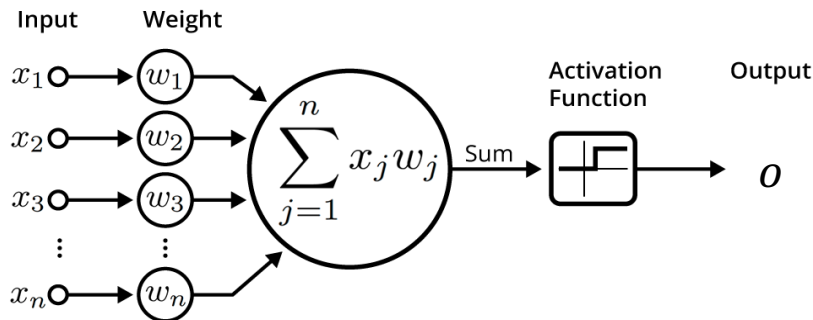
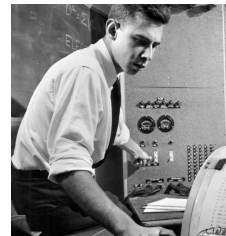


- An alternative model of computation (comparable to "Turing Machine")



Perceptron Learning Rule — Imitating Nature's Learning Process

- Rosenblatt (1959) developed the **Perceptron** algorithm —
 - An iterative algorithm for learning the weights of a **linear threshold unit**.



- A single neuron with a fixed input, it can incrementally change weights and learn to produce a fixed output using the Perceptron learning rule.
- Update each weights by:

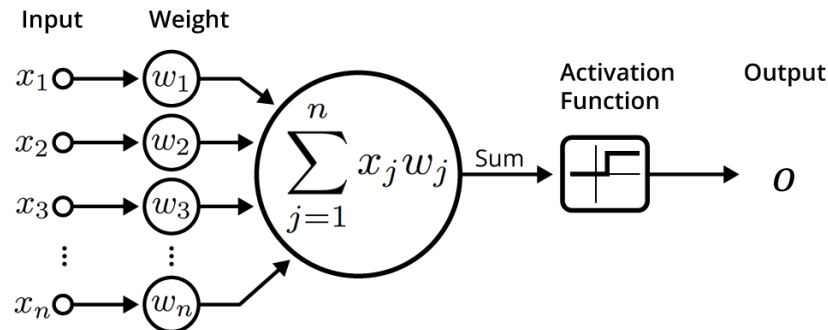
$$w_i = w_i + \eta(t - o)x_i$$

t : the target value

Quiz (1): Understanding Perceptron Update Rule

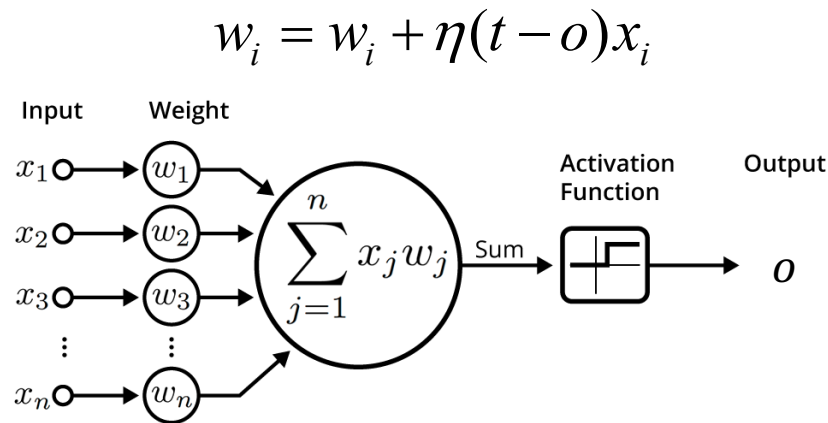
- Suppose the inputs $x_i \in \{0, 1\}$ and $\eta = 1$. If LTU's output o **exactly matches** the target value t , how would the update rule change the weights?
 - Would increase them
 - Would decrease them
 - Would not change them

$$w_i = w_i + \eta(t - o)x_i$$



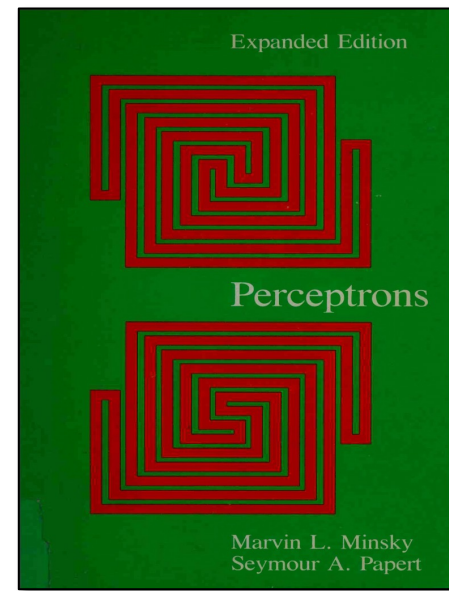
Quiz (2): Understanding Perceptron Update Rule

- Suppose the inputs $x_i \in \{0, 1\}$ and $\eta = 1$. If LTU's output o is **smaller** than the target value t , how would the update rule change the weights?
 - Would increase them
 - Would decrease them
 - Would not change them
- After this update, the new output o would be:
 - Larger
 - Smaller
 - Unchanged
 - May increase or remain unchanged



Perceptron: Demise

- “Perceptrons” (1969) by Minsky and Papert illuminated few **limitations** of the perceptron.
- It showed that:
 - A single threshold-unit **cannot represent** many classification functions (e.g. XOR).
 - Only the **linearly separable** functions are learnable.
- Also, there was an understanding that deeper networks were infeasible to train.
- Result: research on NNs dissipated during the 70’s and early 80’s!



Very Brief History of Neural Networks

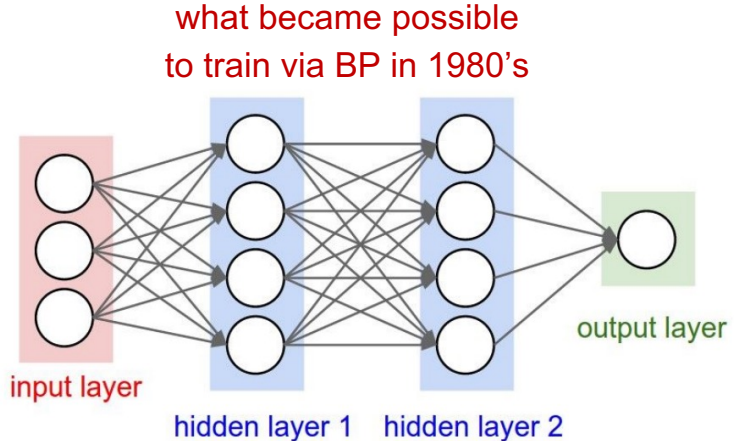
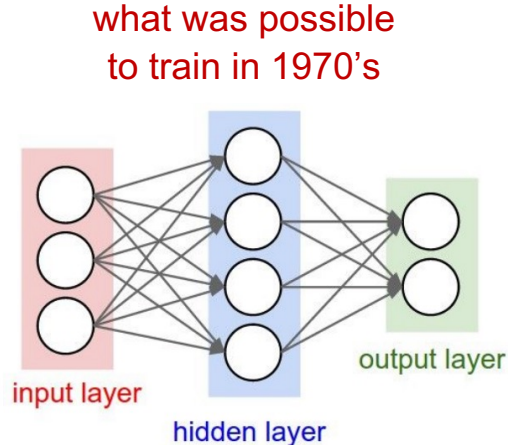
1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

Very Brief History of Neural Networks

1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. **Multi-layer NNs and symbolic learning (1985-1995)**
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

Neural Networks Resurgence (1986)

- Interest in NNs revived in the mid 1980's due to the rise of “connectionism.”
- **Backpropagation algorithm** was [re-]introduced for training three-layer NN's.
 - Backpropagation efficiently computes gradients through multiple layers, enabling gradient-based training.
 - Had no convergence guarantees; was still hard to make it work for deeper models.



[[Learning representations by back-propagating errors, Rumelhart, Hinton & Williams 1986](#);
for a broader context, see: <http://people.idsia.ch/~juergen/who-invented-backpropagation.html>]

Second NN Demise (1995-2010)

- Training deep networks remained difficult because of:
 - not good architectures,
 - limited compute,
 - limited data.
- Little theoretical justification for underlying methods.
- Machine learning research moved to graphical/probabilistic models and kernel methods.

Very Brief History of Neural Networks

1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

Very Brief History of Neural Networks

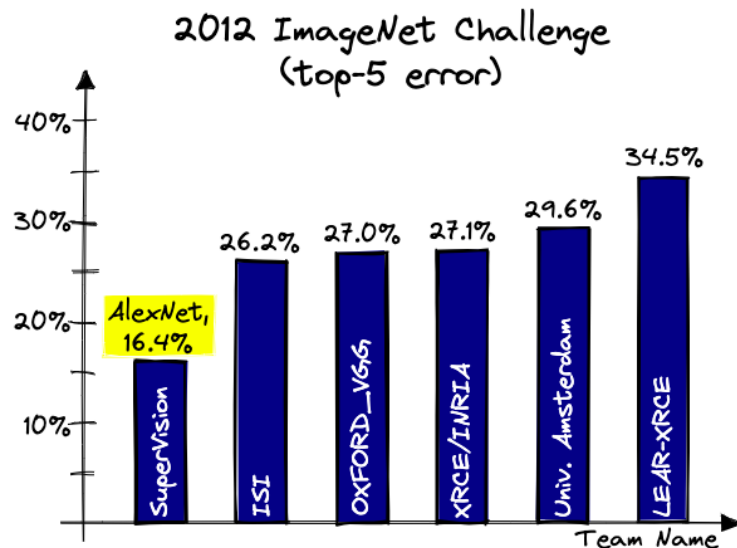
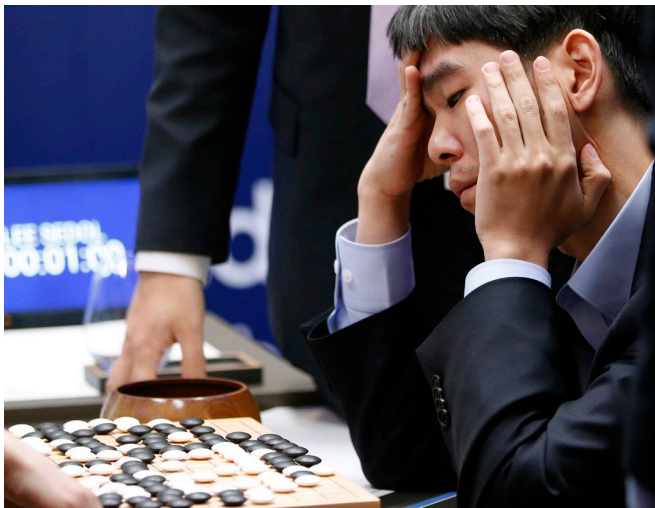
1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. Deep networks and self-supervised learning (2010-?)

Very Brief History of Neural Networks

1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. **Deep networks and self-supervised learning (2010-?)**

Deep Learning Revolution (2010...)

- Various successes with training deep neural works.
 - Convolutional neural nets (CNNs) for vision — 2012 AlexNet showed 16% error reduction on ImageNet benchmark.
 - Rise of deep reinforcement learning for games—AlphaGo beat human players.



Deep Learning Revolution (2010...)

- Various successes with training deep neural works.
 - Convolutional neural nets (CNNs) for vision — 2012 AlexNet showed 16% error reduction on ImageNet benchmark.
 - Rise of deep reinforcement learning for games—AlphaGo beat human players.

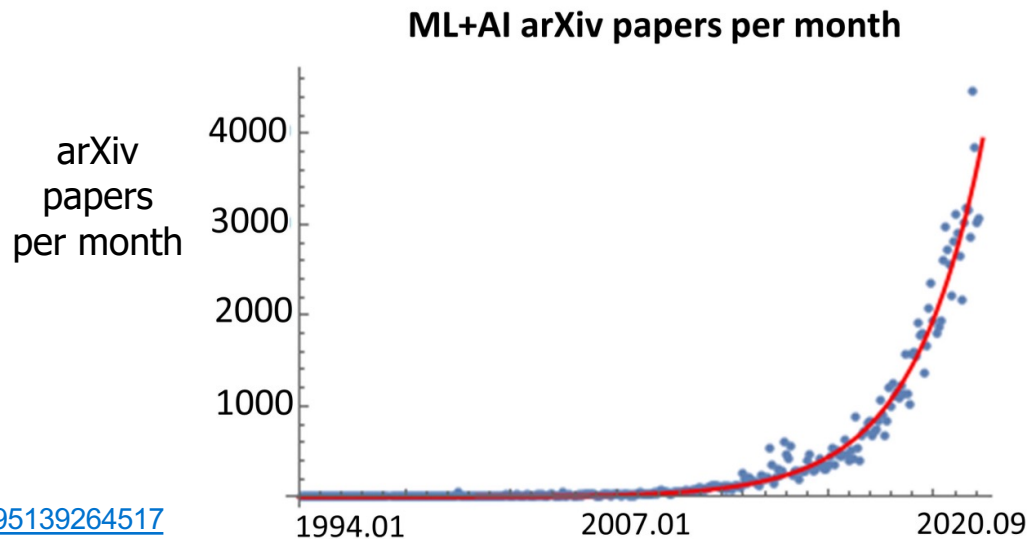
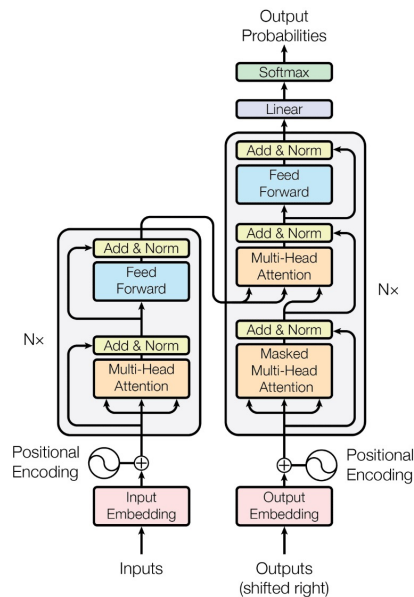
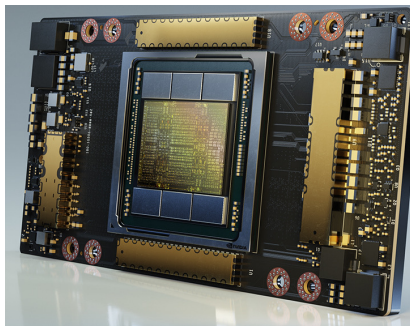
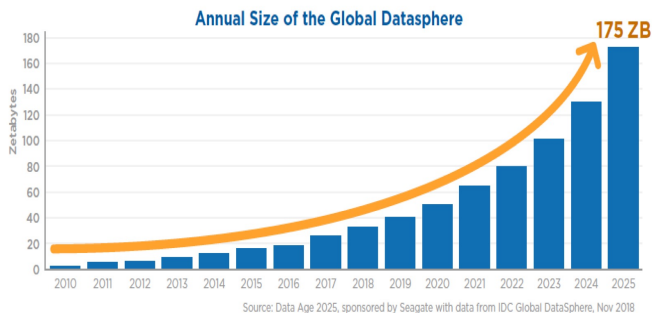


Figure credit:
<https://twitter.com/MarioKrenn6240/status/1314622995139264517>

Deep Learning Revolution (2010...)

- The success continued enabled by 3 forces:
 - Availability of massive [unlabeled] data — the data on Internet.
 - Faster computing technologies — specialized hardware (e.g., GPUs)
 - Algorithmic innovations — architectures, optimization, etc.

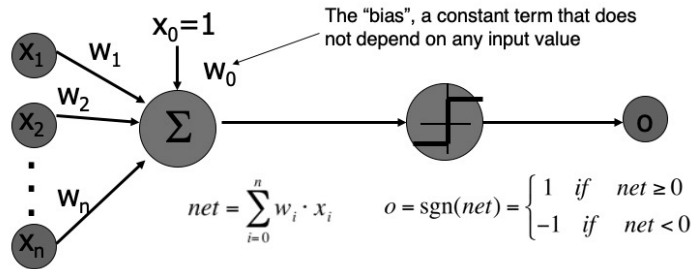


Very Brief History of Neural Networks

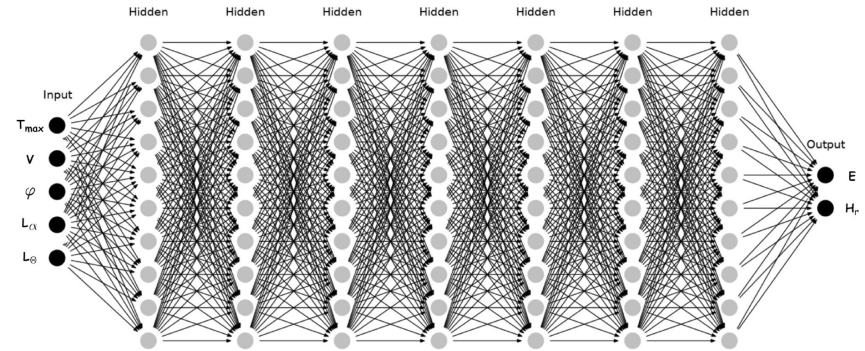
1. Single-layer neural networks (1943-1969)
2. Symbolic AI & knowledge engineering (1970-1985)
3. Multi-layer NNs and symbolic learning (1985-1995)
4. Shallow statistical learning/probabilistic models (1995-2010)
5. **Deep networks and self-supervised learning (2010-?)**



How it started



How it's going



Summary

- Neural networks have been long in the making since 1950s.
- It's a remarkable journey of science with many ups and downs.
- **Next:** How do you train NNs? We will start with some algebra refreshers.

Background for Training NNs

The Refreshers

Machine Learning Problems

- **Training data:** Given a set of inputs and output labels:
 - Inputs: $X = (x_1, \dots, x_n)$
 - Outputs: $Y = (y_1, \dots, y_n)$
- **Goal:** Find a function $f(x; \theta)$ with parameters θ that maps inputs in X to output to Y
- **Empirical risk:** measure the quality of the predictions with a loss function:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i; \theta), y_i)$$

Quiz: Loss functions

- Remember the objective function of our learning problem:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i; \theta), y_i)$$

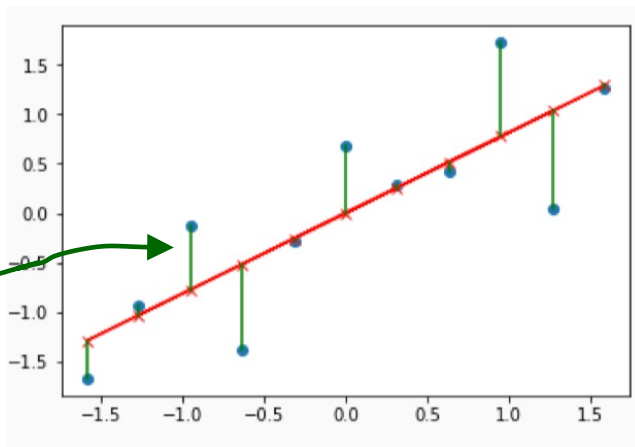
- Which of the followings is a more reasonable loss function $\ell(z, w)$?
 - If z and w are **far apart**, the loss value should be **higher**
 - If z and w are **far apart**, the loss value should be **lower**
 - Neither

A Special Case: Linear Regression

- **Training data:** Given a set of inputs and output labels:
 - Inputs: $X = (x_1, \dots, x_n)$
 - Outputs: $Y = (y_1, \dots, y_n)$
- **Goal:** Find a linear function $f(x; \theta) = \theta \cdot x$ that is best predictive of observations
- **Empirical risk:** measure the quality of the predictions with a loss function:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(\theta \cdot x_i, y_i)$$

What are good choices
for loss function?

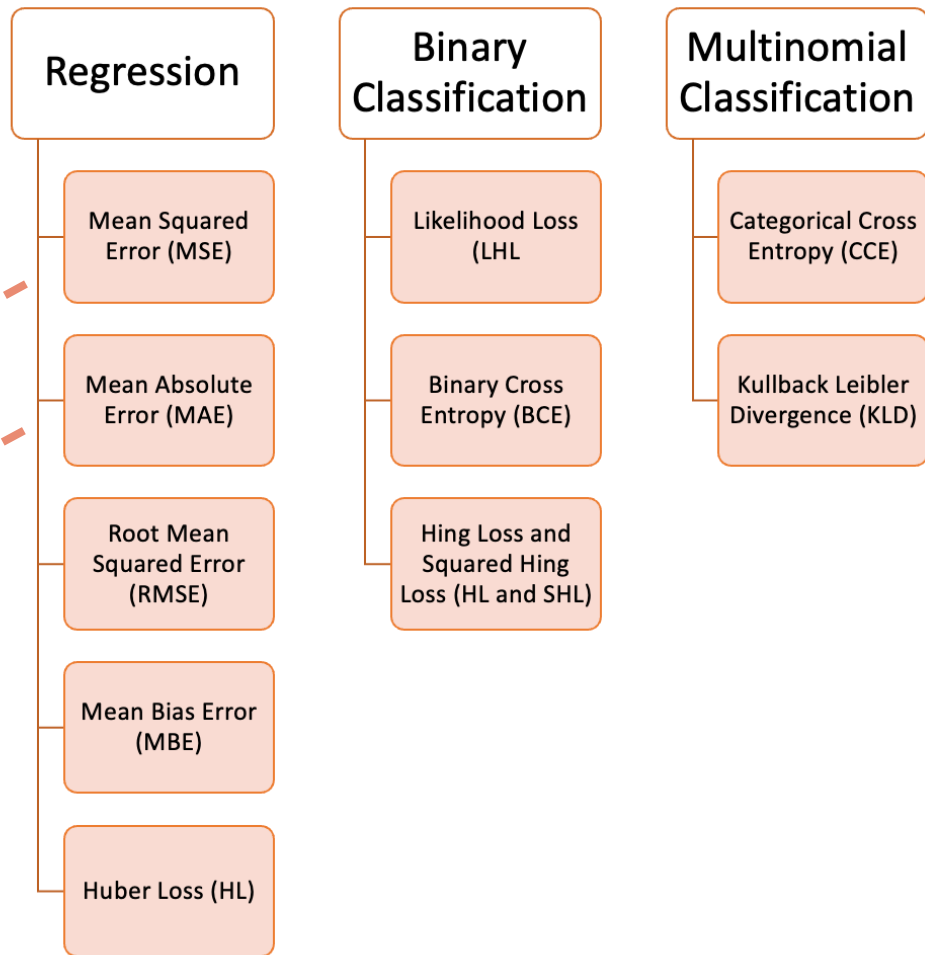
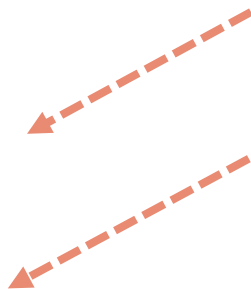


Loss Functions

- The choice of loss function depends on the problem

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

$$\ell(y, \hat{y}) = |y - \hat{y}|$$



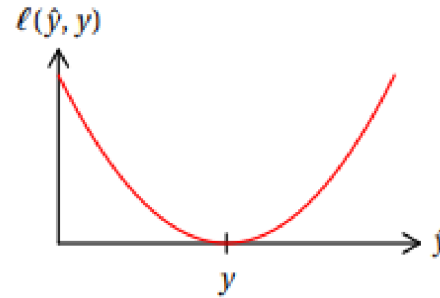
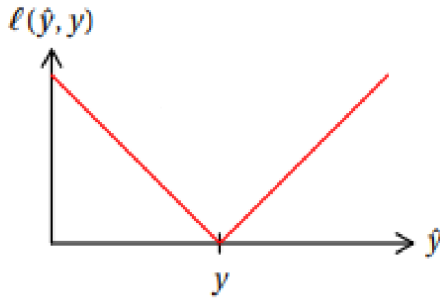
Quiz: MSE vs. MAE loss

- Remember MSE and MAE loss:

$$\text{MSE: } \ell(y, \hat{y}) = (y - \hat{y})^2$$

$$\text{MAE: } \ell(y, \hat{y}) = |y - \hat{y}|$$

- Which visualization corresponds to which loss?



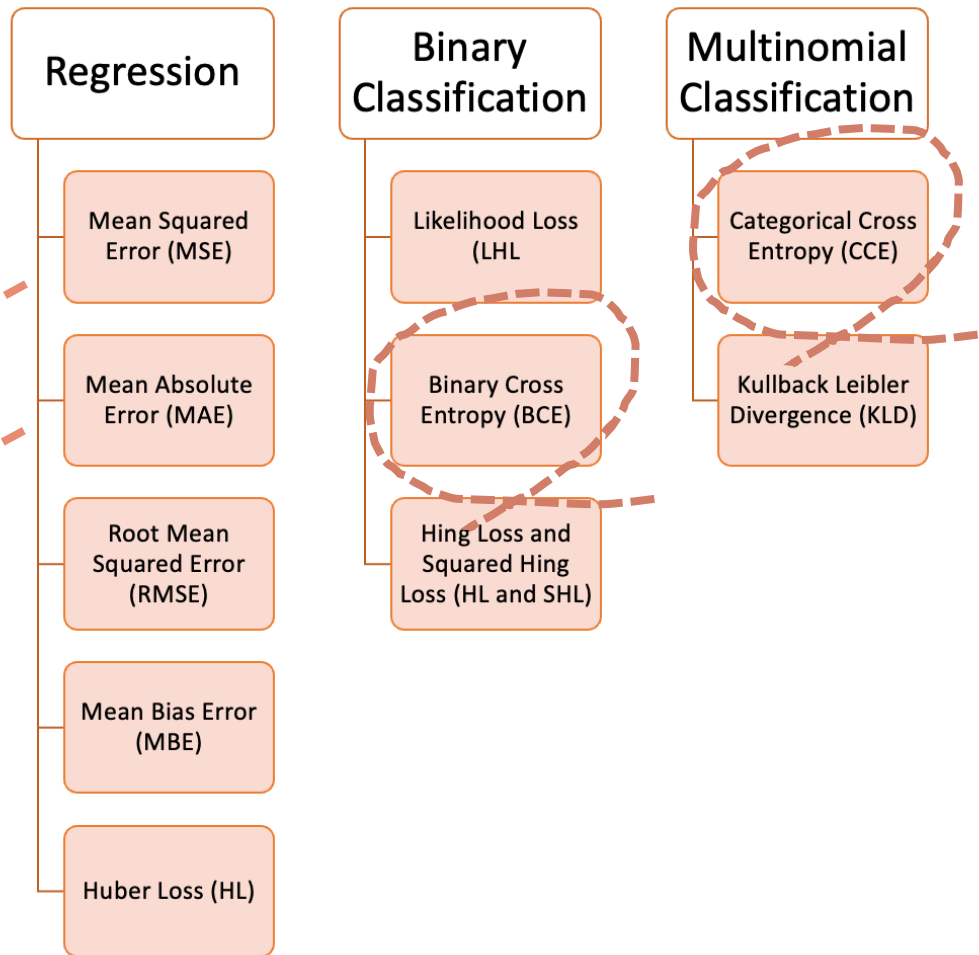
- Which loss is more sensitive to outlier data (noisy outputs)?
- Which loss is nondifferentiable at zero error?

Loss Functions

- The choice of loss function depends on the problem

$$\ell(y, \hat{y}) = (y - \hat{y})^2$$

$$\ell(y, \hat{y}) = |y - \hat{y}|$$



Loss Functions: Cross-Entropy

- A binary classification example: Without loss of generality:

- $\mathcal{C} = 2$
- Gold labels: $y = [1, 0]$ (i.e., first class is correct)
- Predictions: $\hat{y} = [p, 1 - p]$

- CE loss: $\ell(y, \hat{y}) = -1 \times \log p - 0 \times \log(1 - p) = -\log p$

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^{\mathcal{C}} y_j \log(\hat{y}_j)$$

Summation over the
dimensions of $\mathbf{y}$

Quiz: CE for Binary Classification

- A binary classification example: Without loss of generality:

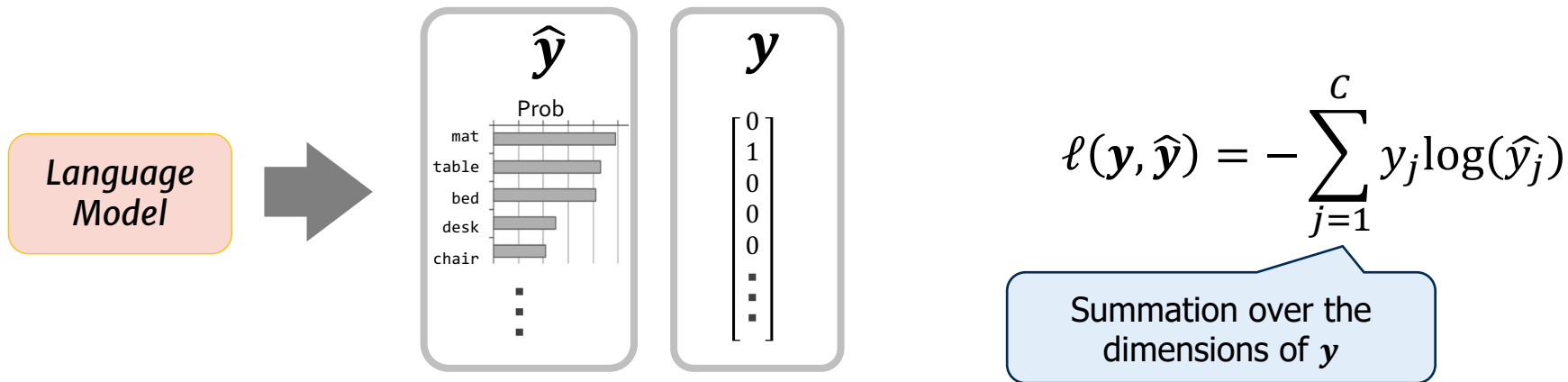
- Gold labels: $y = [1, 0]$ (i.e., first class is correct)
- Predictions: $\hat{y} = [p, 1 - p]$

$$\ell(\mathbf{y}, \hat{\mathbf{y}}) = - \sum_{j=1}^c y_j \log(\hat{y}_j)$$

Summation over the
dimensions of $\mathbf{y}$

- CE loss: $\ell(\mathbf{y}, \hat{\mathbf{y}}) = -1 \times \log p - 0 \times \log(1 - p) = -\log p$
- **Question 1:** Why the negative sign?
- **Question 2:** If the model prediction is miraculously accurate ($p = 1$), what is CE loss?
- **Question 3:** If the model prediction is embarrassingly off ($p = 0$), what is the CE loss?
- **Question 4:** If the model prediction is random ($p = 0.5$)—which is often the case once you initialize your models—what is the CE loss?

Cross-Entropy Over Next-Word Distribution



- **Q1:** Suppose for a fixed j , $y_j = 1$ (i.e., the j -th dimension is the correct answer). CE is?
 - It is the negative log-probability that our language model picks the j th term.
- **Q2:** Once we initialize our language model, what would we expect the loss to be? ...
 - $\log C$

Machine Learning Problems

- **Training data:** Given a set of inputs and output labels:
 - Inputs: $X = (x_1, \dots, x_n)$
 - Outputs: $Y = (y_1, \dots, y_n)$
- **Goal:** Find a function $f(x; \theta)$ with parameters θ that maps inputs in X to output to Y
- **Empirical risk:** measure the quality of the predictions with a loss function:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i; \theta), y_i)$$

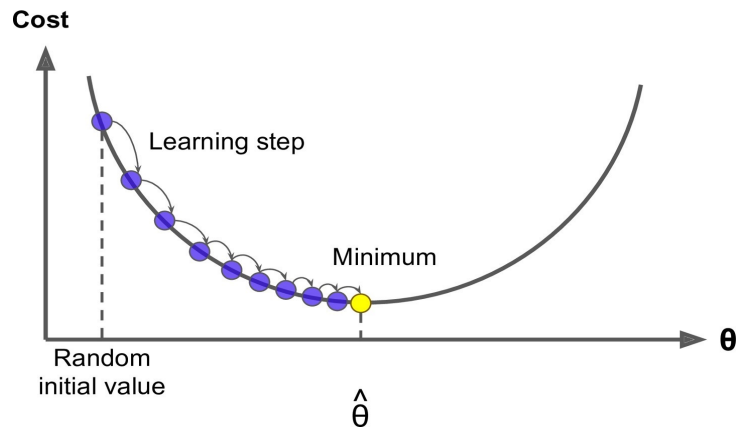
- Machine learning as optimization:

$$\operatorname{argmin}_{\theta} J(\theta)$$

How do you solve this optimization?

Gradient Descent

- We have a cost function $J(\theta)$ we want to minimize
 - We can use **Gradient Descent** algorithm!
- **Idea:** for current value of θ , take gradual steps to minimize $J(\theta)$.
- Note: Our objectives may not be convex like this. But life turns out to be okay!



Gradient Descent (1): Intuition

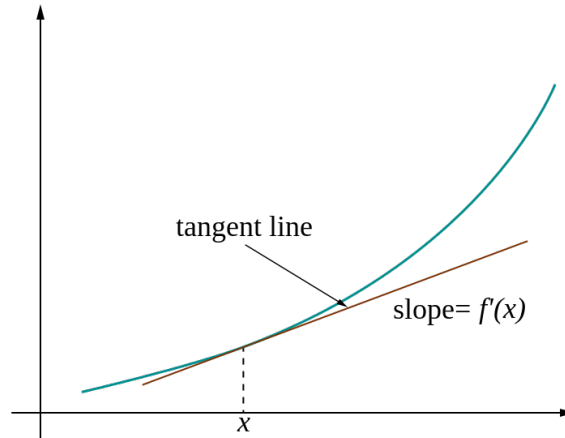
- Imagine you're blindfolded
- Need to walk down a hill
- You can use your hands to find the directions that may be downhill



Gradient Descent (2): Intuition

- In 1-dimension, the **derivative** of a function:

$$\frac{\partial J}{\partial \theta_j} = \lim_{h \rightarrow 0} \frac{J(\theta_j + h) - J(\theta_j)}{h}$$



- Gradient** is the vector of derivatives: $\nabla_{\theta} J = \left[\frac{\partial J}{\partial \theta_1}, \dots, \frac{\partial J}{\partial \theta_n} \right]$

Gradient Descent (3)

- Gradient descent involves iterative steps *in direction of **negative gradient***.
- Why step in direction of negative gradient?
 - Gradient quantifies how rapidly the function $L(\theta)$ varies when we change the argument θ_j by a tiny amount.
 - For a nonzero gradient and sufficiently small positive step size, this decreases the loss.
- First-order Taylor approximation:

$$J(\theta - \alpha g) \approx J(\theta) - \alpha \|g\|^2 \quad \text{where } g = \nabla J(\theta)$$

Gradient Descent (3)

- Iteratively subtract the gradient with respect to the model parameters (θ)

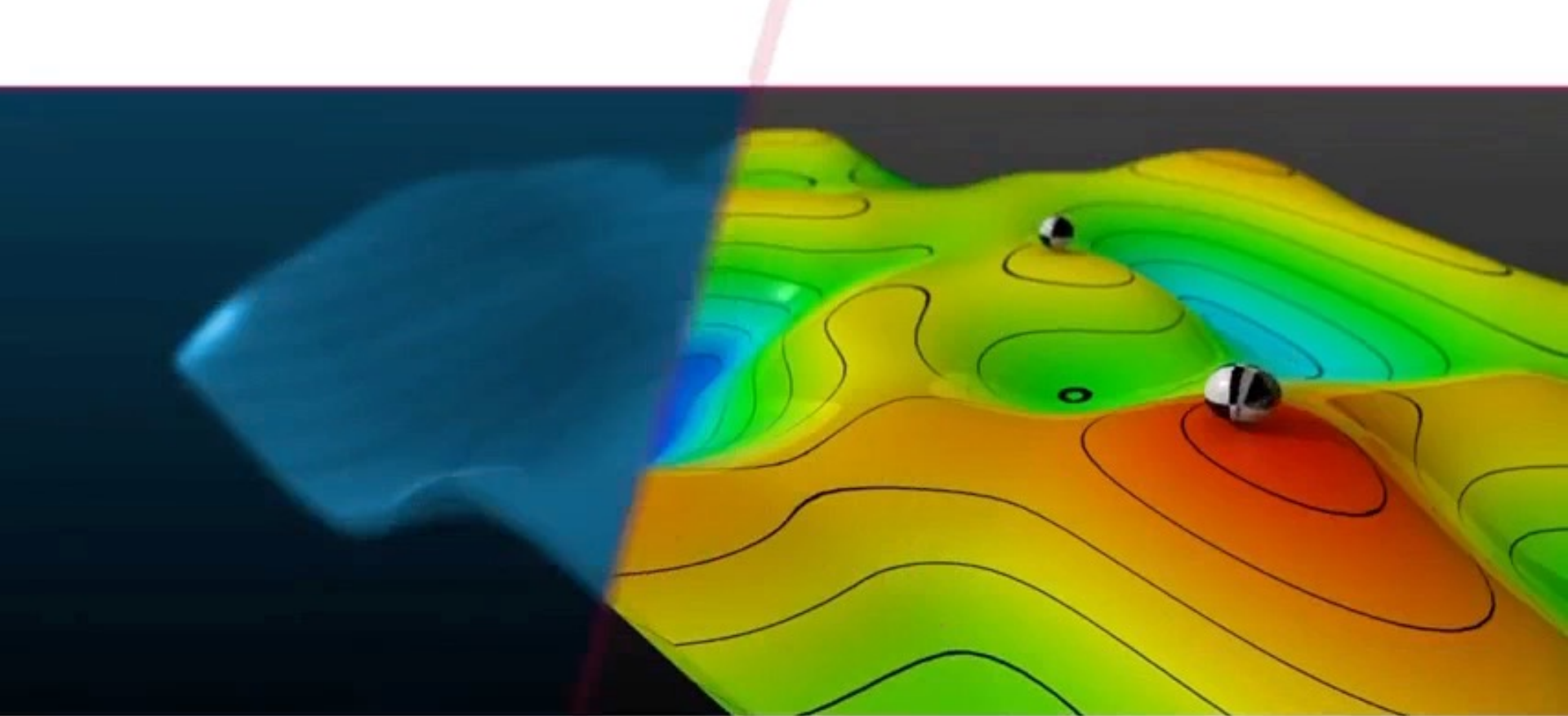
$$\theta^{new} \leftarrow \theta^{old} - \alpha \nabla_{\theta} J(\theta^{old})$$

α = Step size or learning rate

- We're moving in a direction opposite to the gradient of the loss $J(\theta)$
- I.e., we're moving towards smaller loss $J(\theta)$

- In PyTorch:

```
while True:
    theta_grad = evaluate_gradient(J, corpus, theta)
    theta = theta - alpha * theta_grad
```

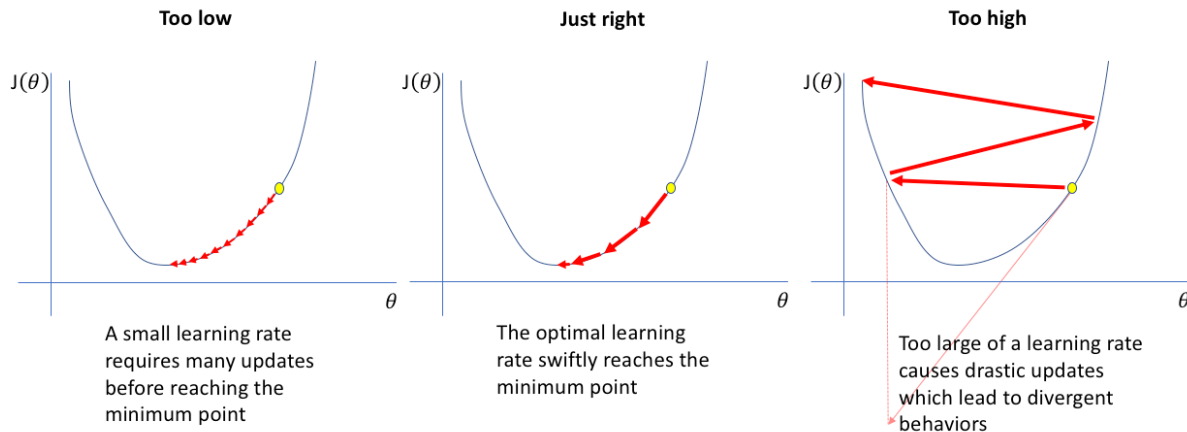


clideo.com

[demo credit: ICMS YouTube channel]

Gradient Descent: Setting the Step Size

- What is a good value for step size α ?
 - If α = too small, it may be too slow
 - If α = too large, it may oscillate



- It may take trial-and-errors to find the sweet spot.
- Another trick is to define a “schedule” for gradually reducing the learning rate starting from a large number.

GD vs Stochastic Gradient Descent (SGD)

- **Training data:** Given a set of inputs and output labels:

- Inputs: $X = (x_1, \dots, x_n)$
- Outputs: $Y = (y_1, \dots, y_n)$

- What I described is GD (**full batch**; *all* data is used for computing gradients):

$$\theta^{new} \leftarrow \theta^{old} - \alpha \nabla_{\theta} J(\theta^{old}), \quad J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f(x_i; \theta), y_i)$$

- In practice, one can use **stochastic samples** (mini-batches) for grad estimation.

- Let B be a randomly sampled batch of examples (pure SGD uses $|B| = 1$):

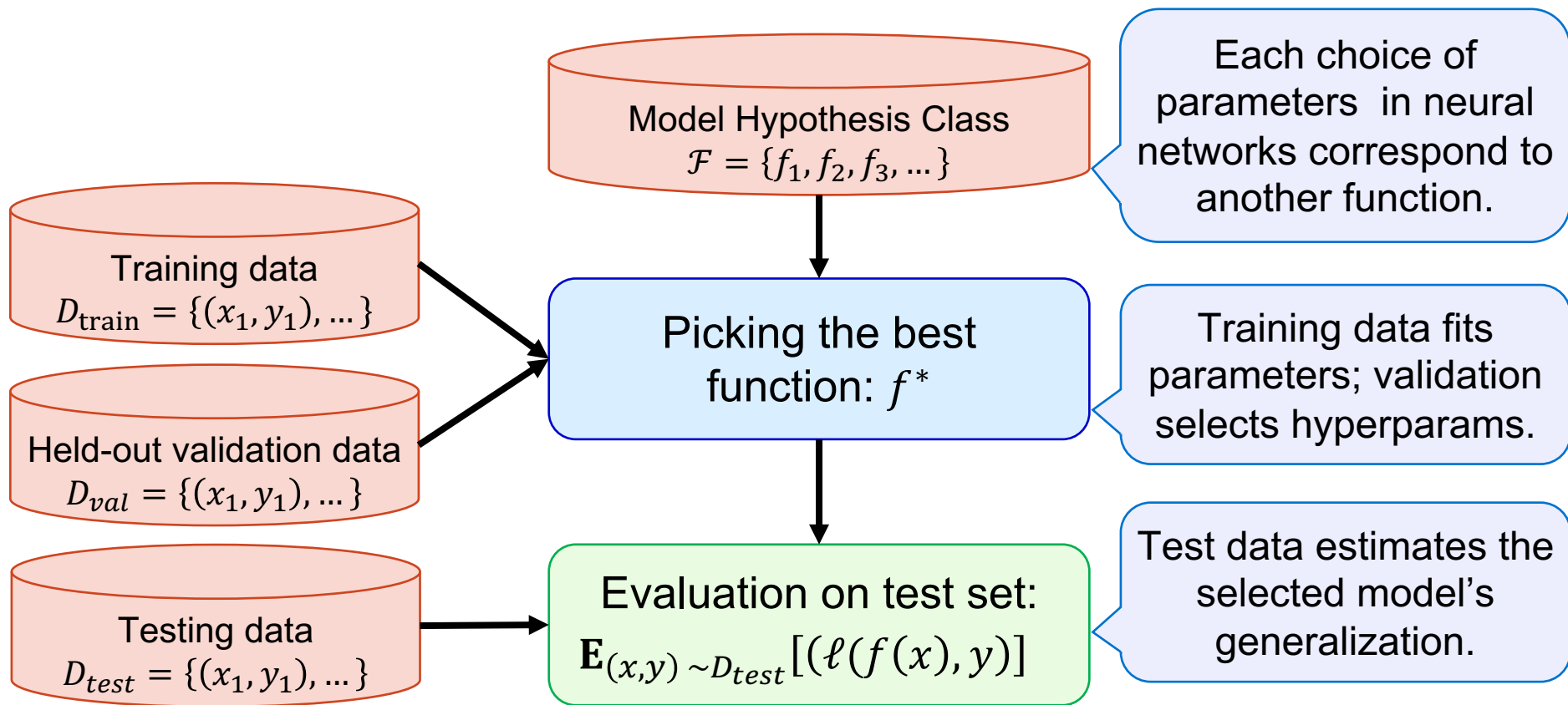
$$\theta^{new} \leftarrow \theta^{old} - \alpha \nabla_{\theta} \hat{J}(\theta^{old}), \quad \hat{J}(\theta) = \frac{1}{|B|} \sum_{(x_i, y_i) \in B} \ell(f(x_i; \theta), y_i)$$

- SGD is often faster and takes up less GPU memory (more on this later).

Gradient Descent: Summary Thus Far

- A statistical learning problem can be formulated as an **optimization** problem.
- The objective of this optimization consists of:
 - Learning data (input/outputs)
 - Predictive model architecture (encoding how an input gets mapped to an output)
 - Loss function (quantifying quality of predictions)
- Soon, we will see how to use Neural Nets as the predictive model.

A Typical Learning and Evaluation Protocol





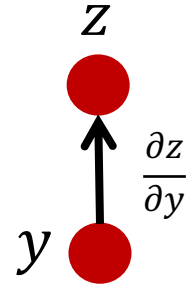
Algebra Refresher

(Needed for Backpropagation)



Derivatives

- First let's get the notation right:
- The **arrow** shows **functional dependence** of z on y , i.e. given y , we can calculate z .
 - For example: $z(y) = 2y^2$
- The derivative of z , with respect to y : $\frac{\partial z}{\partial y}$



Quiz time!

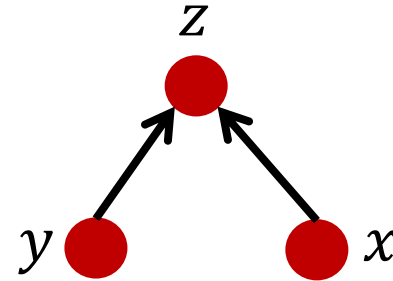
■ If $z(x, y) = y^4 x^5$ what is the following derivative $\frac{\partial z}{\partial y}$?

1. $\frac{\partial z}{\partial y} = 4y^3 x^5$

2. $\frac{\partial z}{\partial y} = 5y^4 x^4$

3. $\frac{\partial z}{\partial y} = 20y^3 x^4$

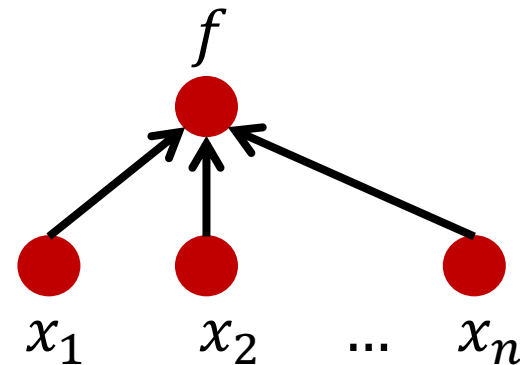
4. None of the above



Gradient

- Given a function with 1 output and n inputs

$$f(\mathbf{x}) = f(x_1, x_2, \dots, x_n) \in \mathbb{R}$$



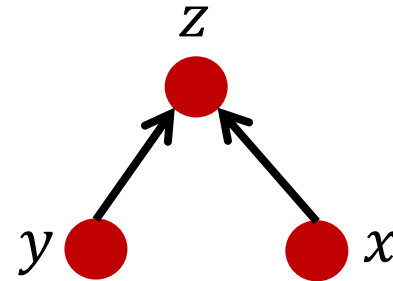
- Its gradient is a vector of partial derivatives with respect to each input

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \in \mathbb{R}^n$$

(always assume vectors are **column vectors**, i.e., they're in $\mathbb{R}^{n \times 1}$)

Quiz time!

- If $z(x, y) = y^4 x^5$ what is the following gradient ∇z ?
 1. $\nabla z(x, y) = 4y^3 x^5$
 2. $\nabla z(x, y) = (5y^4 x^4, 20y^3 x^4)$
 3. $\nabla z(x, y) = (5y^4 x^4, 4y^3 x^5)$
 4. None of the above

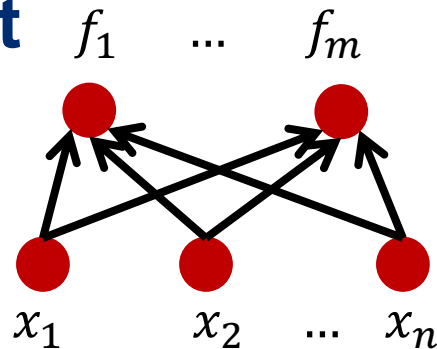


Jacobian Matrix: Generalization of the Gradient

- Given a function with **m outputs** and **n inputs**

$$\mathbf{f}(\mathbf{x}) = [f_1(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n)] \in \mathbb{R}^m$$

- Its Jacobian is an **$m \times n$ matrix** of partial derivatives: $(\mathbf{J}_f(\mathbf{x}))_{ij} = \frac{\partial f_i}{\partial x_j}$



$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n}$$

Quiz: Jacobian's special case (1)

- Remember Jacobians:

$$\mathbf{f}(\mathbf{x}) = [f_1(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n)] \in \mathbb{R}^m$$

$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad \text{or} \quad (\mathbf{J}_f(\mathbf{x}))_{ij} = \frac{\partial f_i}{\partial x_j}$$

- When $m=1$ (scalar-valued function), Jacobian reduces to ...?

$$\nabla^T \mathbf{f}(\mathbf{x}) \quad (\text{gradient transpose})$$

Quiz: Jacobian's special case (2)

- Remember Jacobians:

$$\mathbf{f}(\mathbf{x}) = [f_1(x_1, x_2, \dots, x_n), \dots, f_m(x_1, x_2, \dots, x_n)] \in \mathbb{R}^m$$

$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \dots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \dots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \quad \text{or} \quad (\mathbf{J}_f(\mathbf{x}))_{ij} = \frac{\partial f_i}{\partial x_j}$$

- When $m=n=1$ (single-variable function), Jacobian reduces to ...?

the derivative of $\mathbf{f}$

Jacobian for Matrix Inputs

- Given a function with m **outputs** and $n \times p$ **inputs**

$$\mathbf{f}(\mathbf{X}) = [f_1(\mathbf{X}), \dots, f_m(\mathbf{X})] \in \mathbb{R}^m, \text{ where } \mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1p} \\ \vdots & \ddots & \vdots \\ x_{n1} & \cdots & x_{np} \end{bmatrix} \in \mathbb{R}^{n \times p}$$

- Jacobian is a $m \times n \times p$ **tensor** (i.e., **matrix of matrices**) of partial derivatives:

$$(\mathbf{J}_f(\mathbf{X}))_{ijk} = \frac{\partial f_i}{\partial x_{jk}}$$

- The dimension of the Jacobian:= output dim x input dims.
- The Jacobian math holds if you keep adding **more dimensions** to the input or output.

Why Use Matrix/Tensor Form?

- In essence, matrix form (multi-variate calculus) is just an extension of single-variable calculus.

Two reasons:

- Compact derivations: with matrix form calculations we can compute a concise statements.
- Implementing algorithms in matrix form is much faster.
- GPUs are optimized for VERY FAST matrix/tensor operations.



Chain Rule

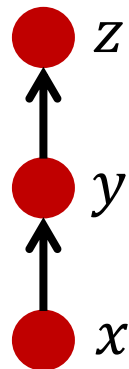
- Function composition:

$$z \circ y(x) = z(y(x)) = z(x)$$

If z is a function of y , and
 y is a function of x , then
 z is a function of x , as well.

Then:

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x}$$



Chain Rule for Multivariable Functions

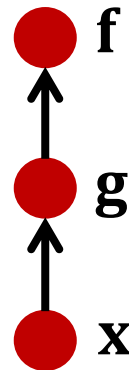
- Let $\mathbf{x} \in \mathbb{R}^d$, $\mathbf{g}: \mathbb{R}^d \rightarrow \mathbb{R}^n$, $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$
- Composing them: $\mathbf{f} \circ \mathbf{g}(\mathbf{x}) = \mathbf{f}(\mathbf{g}(\mathbf{x})): \mathbb{R}^d \rightarrow \mathbb{R}^m$

The result looks similar to the single-variable setup:

$$\mathbf{J}_{\mathbf{f} \circ \mathbf{g}}(\mathbf{x}) = \mathbf{J}_{\mathbf{f}}(\mathbf{g}(\mathbf{x})) \mathbf{J}_{\mathbf{g}}(\mathbf{x})$$

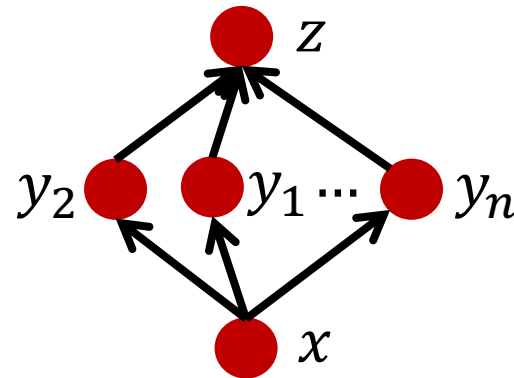
Note, the above statement is a **matrix** multiplication!

Function $\mathbf{f} \circ \mathbf{g}$ has m outputs and d inputs $\rightarrow$ Jacobian's dims: m by d



Quiz Time!

Let $x \in \mathbb{R}$, $\mathbf{y}: \mathbb{R} \rightarrow \mathbb{R}^n$, $\mathbf{z}: \mathbb{R}^n \rightarrow \mathbb{R}$



What is the Jacobian of $z \circ \mathbf{y}(x) = z(y_1(x), \dots, y_n(x))$?

1. $\mathbf{J}_{z \circ \mathbf{y}}(x) = \mathbf{J}_z(\mathbf{y}(x)) \mathbf{J}_y(x)$
2. $\mathbf{J}_{z \circ \mathbf{y}}(x) = \left[\frac{\partial z}{\partial y_1}, \dots, \frac{\partial z}{\partial y_n} \right] \left[\frac{\partial y_1}{\partial x}, \dots, \frac{\partial y_n}{\partial x} \right]^T$
3. $\mathbf{J}_{z \circ \mathbf{y}}(x) = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x}$
4. All the above!

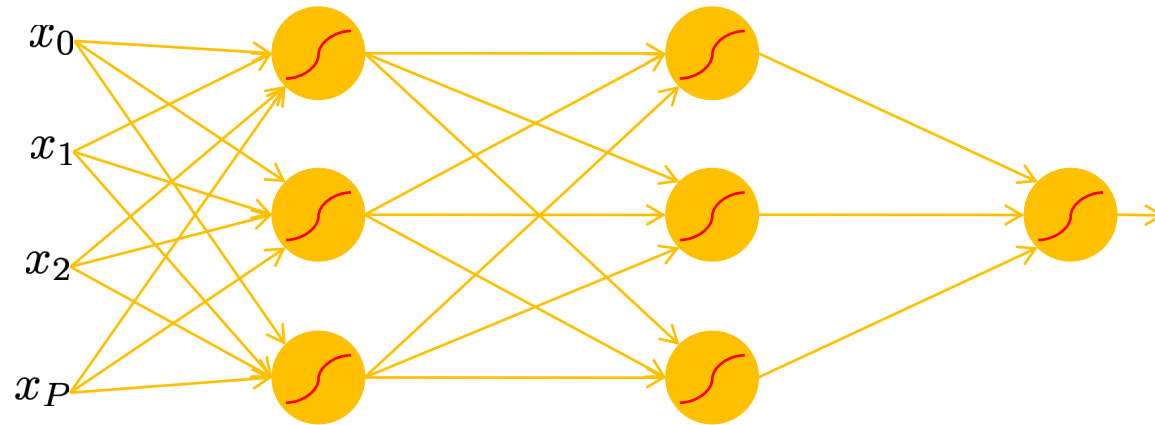
Summary

- We reviewed lots of background about neural networks!
 - Linear algebra foundation
 - Gradient descent
 - Extending gradients to tensor form: Jacobians

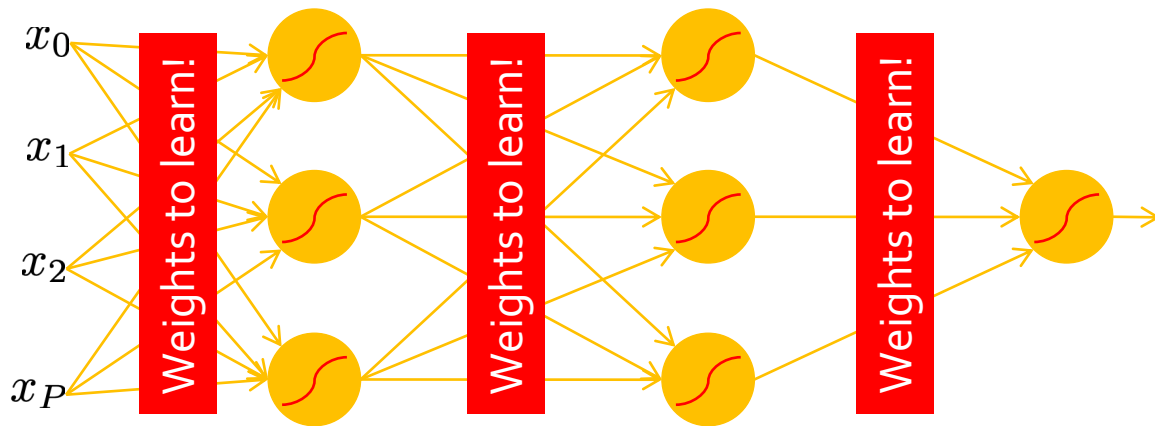
- **Next:** training a neural net!

Training Neural Networks: Analytical Backprop

Recap: Multi-Layer Perceptron

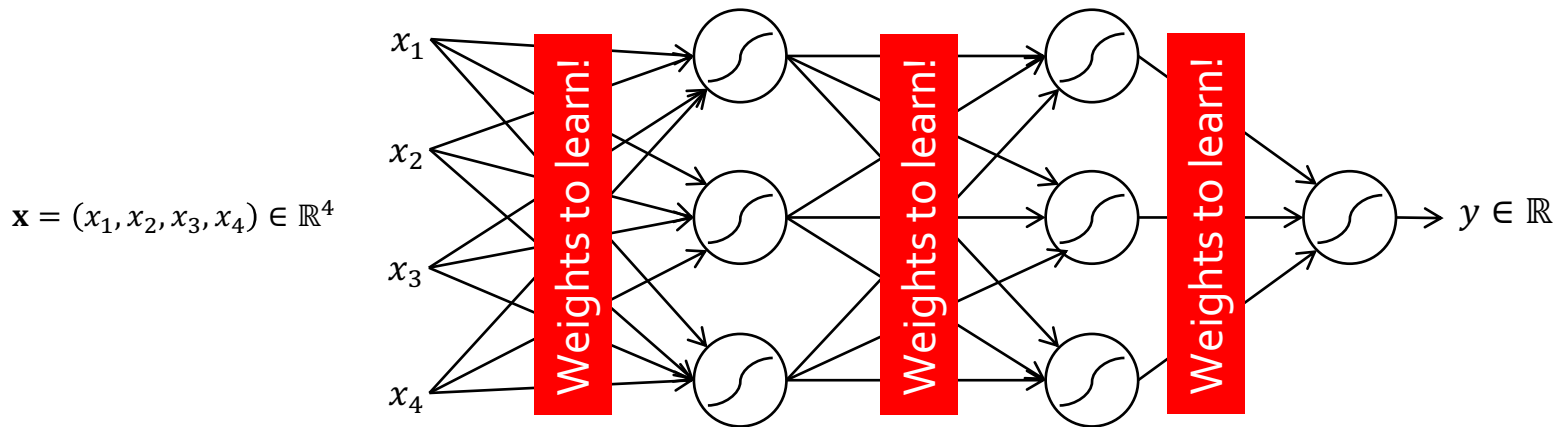


Recap: Multi-Layer Perceptron



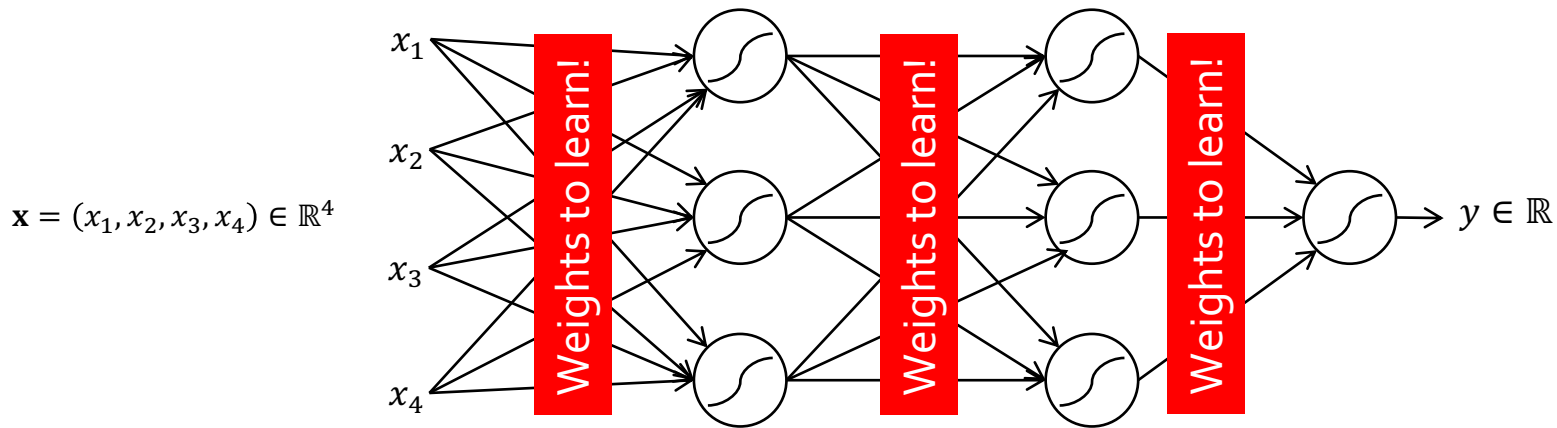
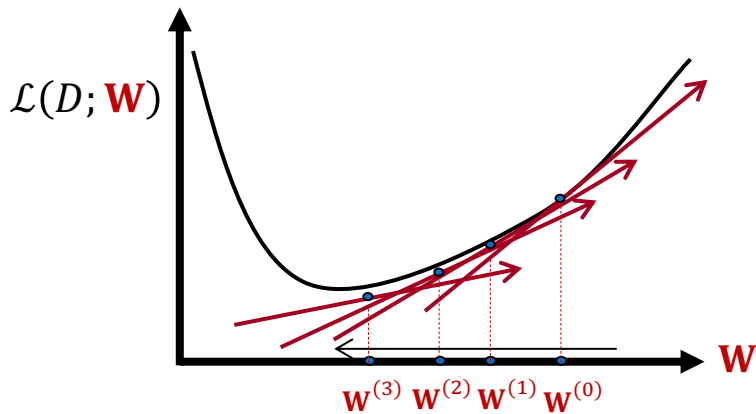
Training Neural Networks: Setup

- We are given a fixed architecture with trainable parameters $\mathbf{W}$.
- We are given a training data $D = \{(\mathbf{x}_i, y_i^*)\}$
- We are given a loss function $\ell: \mathbb{R} \times \mathbb{R} \rightarrow [0, +\infty)$
 - $\ell(y, y^*)$ quantifies distance between answer y^* and prediction $y = \text{NN}(\mathbf{x}; \mathbf{W})$; lower is better.
- Overall objective to optimize: $\mathcal{L}(D; \mathbf{W}) = \sum_{(\mathbf{x}_i, y_i^*) \in D} \ell(\text{NN}(\mathbf{x}_i; \mathbf{W}), y_i^*)$



Training Neural Networks $\approx$ Optimizing Parameters

- We can use **gradient descent** to minimize the loss.
- At each step, the **weight vector** is modified in the **direction that produces the steepest descent** along the error surface.

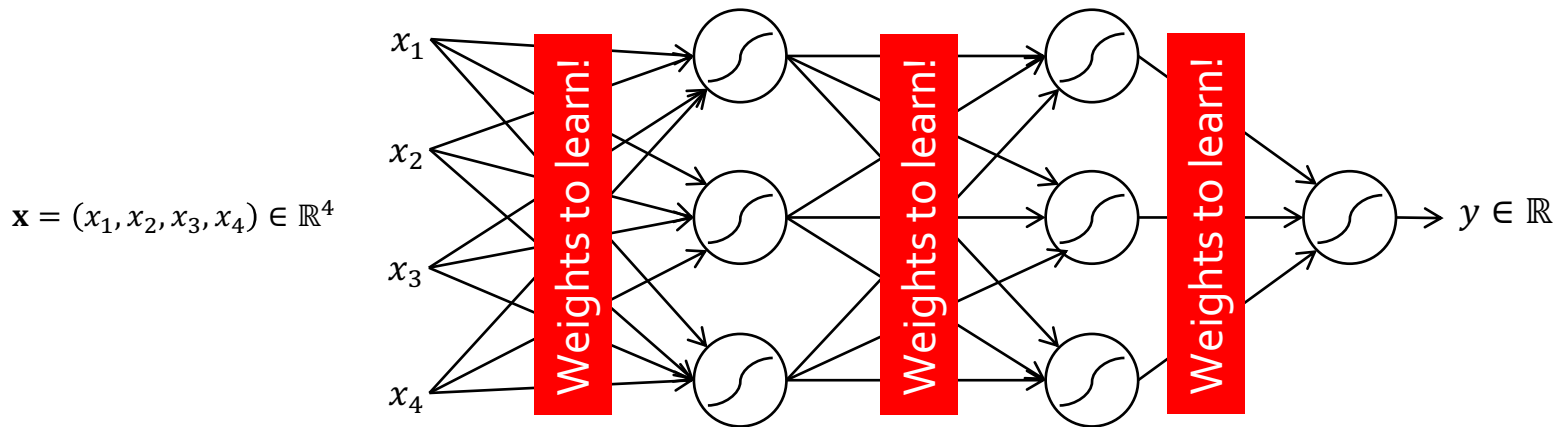
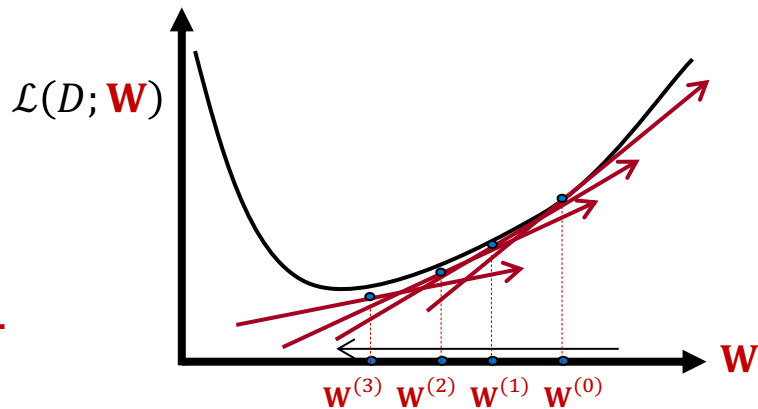


Training Neural Networks $\approx$ Optimizing Parameters

For each sub-parameter $W_i \in \mathbf{W}$:

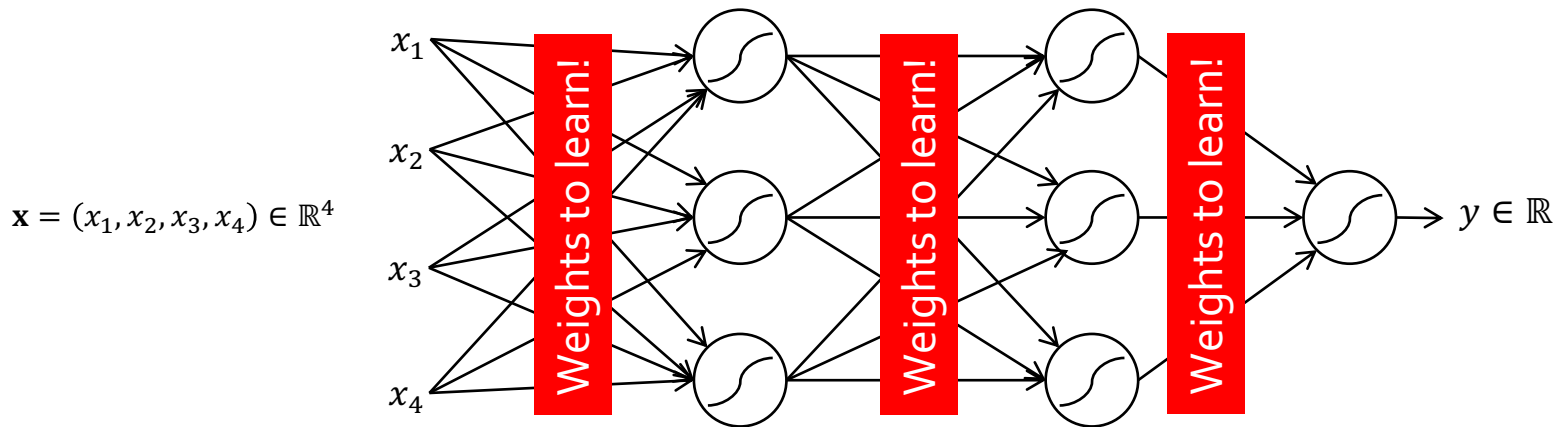
$$W_i^{(t+1)} = W_i^{(t)} - \alpha \frac{\partial \mathcal{L}}{\partial W_i}$$

It all comes down to **efficiently** computing $\frac{\partial \mathcal{L}}{\partial W_i}$.



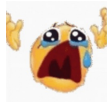
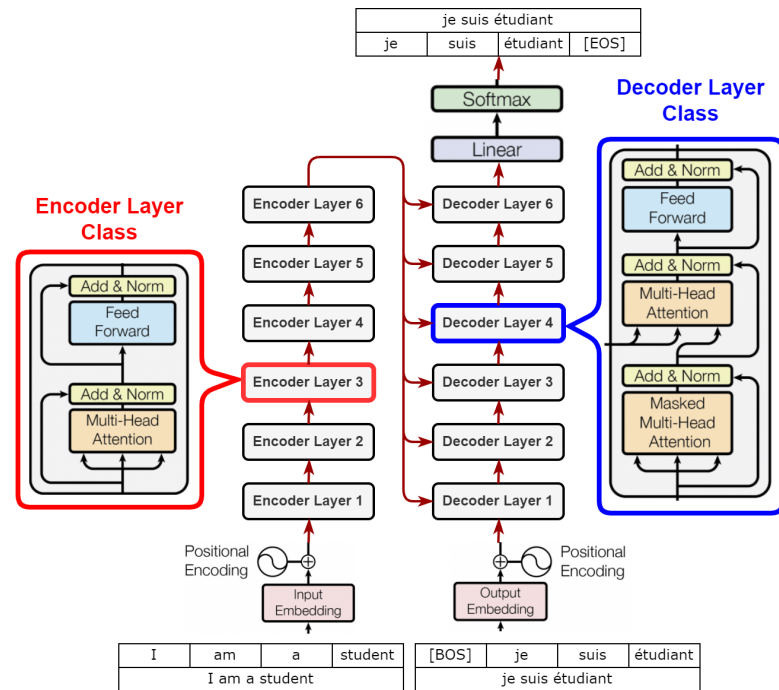
Training Neural Networks $\approx$ Computing the Gradients

- How do you **efficiently** compute $\frac{\partial \mathcal{L}}{\partial w_i}$ for all parameters?
- It's easy to learn the final layer – it's just a linear unit.
- How about the weights in the earlier layers (i.e., before the final layer)?



Necessity of a Principled Algorithm for Gradient Computation

- **Depth** gives more representational capacity to neural networks.
- However, computing gradients for deeper layers is **not trivial and tedious**.
- Even if we have analytical formula for gradient, if they're architecture-specific, they **must be repeated for each new architecture**.
- The solution is “Backpropagation” algorithm!



Transformers have many layers and billions of parameters
— we will study get to this model in a few weeks!

BP: Required Intuitions

1. Gradient Descent

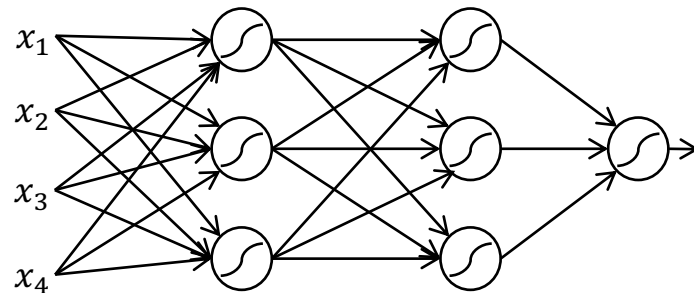
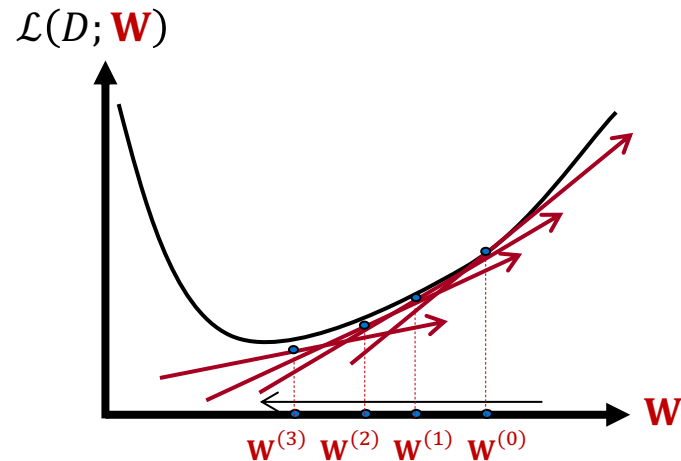
- Move parameters $\mathbf{W}$ in the **negative gradient** direction to reduce loss.

2. Chain Rule

- Use the chain rule to compute derivatives with respect to parameters in earlier layers.

3. Dynamic Programming

- Reuse intermediate loss gradients during the backward pass.



A Generic Multi-Layer Perceptron

- Given the following definition:

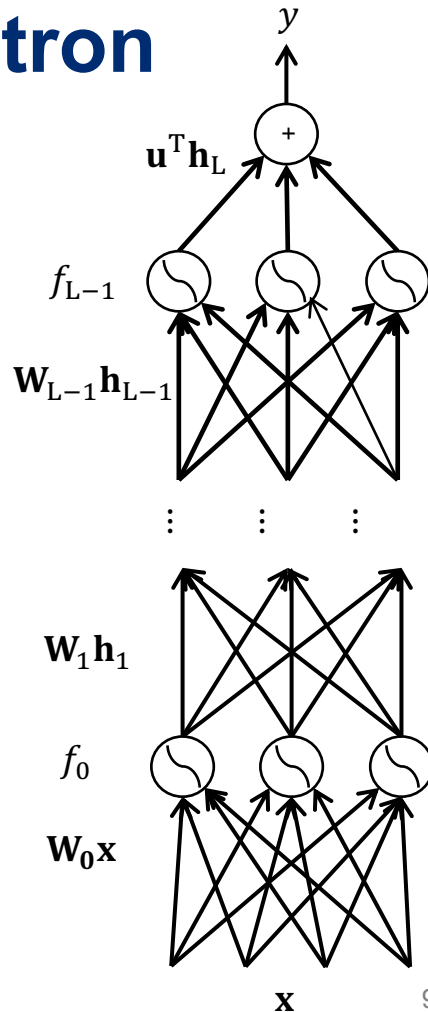
$\mathbf{x} = \mathbf{h}_0 \in \mathbb{R}^{d_0}$ (input), $\mathbf{W}_i \in \mathbb{R}^{d_{i+1} \times d_i}$ (params)

$\mathbf{h}_{i+1} = f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}}$ (hidden layer i , $0 \leq i \leq L - 1$)

$y = \mathbf{u}^T \mathbf{h}_L \in \mathbb{R}$ (output), $\mathbf{u} \in \mathbb{R}^{d_L}$ (linear proj)

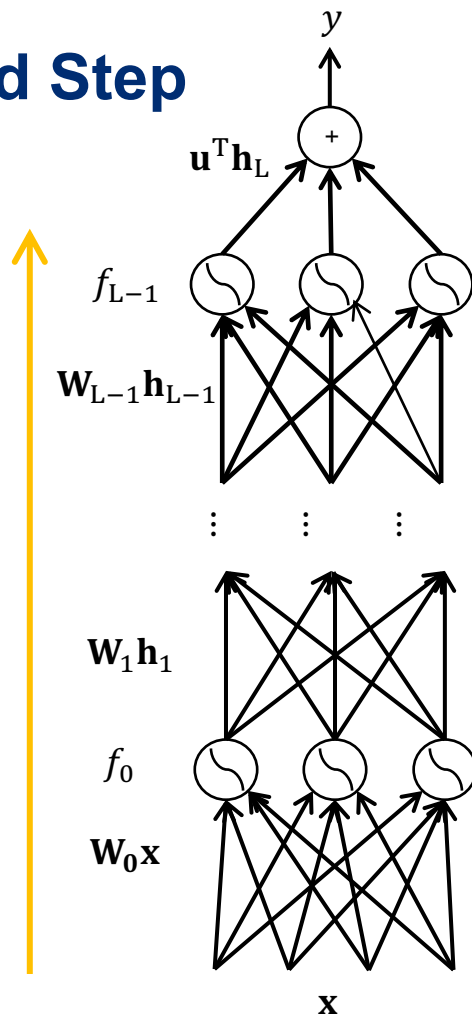
$\mathcal{L} = \ell(y, y^*) \in \mathbb{R}$ (loss)

- Trainable parameters: $\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\}$



A Generic Neural Network: Forward Step

- Given some [initial] values for the parameters, we can compute **the forward pass**, layer by layer.
- Forward pass is basically **L matrix multiplications**, each followed by an activation function.
- Matrix multiplication can be done efficiently with GPUs.
 - Therefore, **forward pass is somewhat fast**.
- Complexity of forward pass is **linear of depth $O(L)$** .



A Generic Neural Network: Direct Gradients

$\mathbf{x} = \mathbf{h}_0 \in \mathbb{R}^{d_0}$ (input)

$y = \mathbf{u}^T \mathbf{h}_L \in \mathbb{R}$ (output)

$\mathbf{h}_{i+1} = f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}}$

$\mathcal{L} = \ell(y, y^*) \in \mathbb{R}$ (loss)

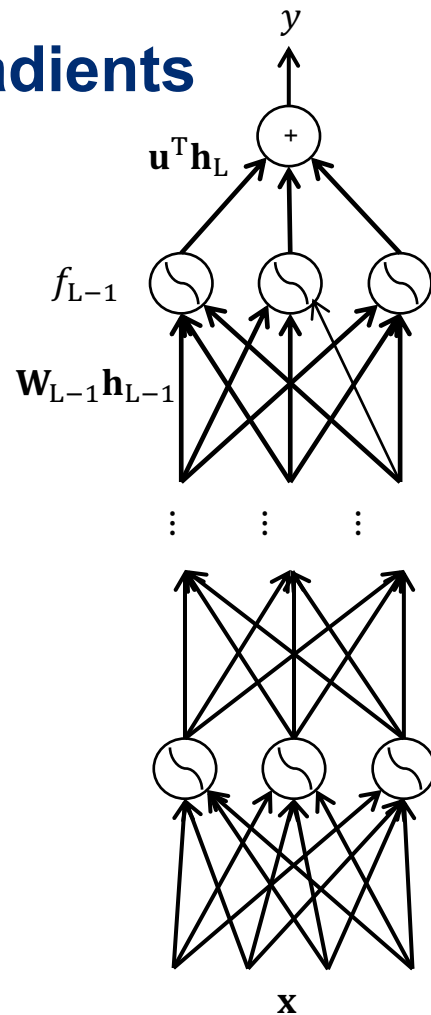
$(0 \leq i \leq L - 1)$

$\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\}$

We want the gradients of $\mathcal{L}$ with respect to model parameters.
Here is the *easy* gradient:

$$\nabla_{\mathbf{u}} \mathcal{L} = \frac{\partial \mathcal{L}}{\partial y} \left(\mathbf{J}_y(\mathbf{u}) \right)^T = \frac{\partial \mathcal{L}}{\partial y} \mathbf{h}_L$$

Now we focus on other model complex gradients.



A Generic Neural Network: Direct Gradients

$\mathbf{x} = \mathbf{h}_0 \in \mathbb{R}^{d_0}$ (input)

$y = \mathbf{u}^T \mathbf{h}_L \in \mathbb{R}$ (output)

$\mathbf{h}_{i+1} = f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}}$

$\mathcal{L} = \ell(y, y^*) \in \mathbb{R}$ (loss)

$(0 \leq i \leq L - 1)$

$\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\}$

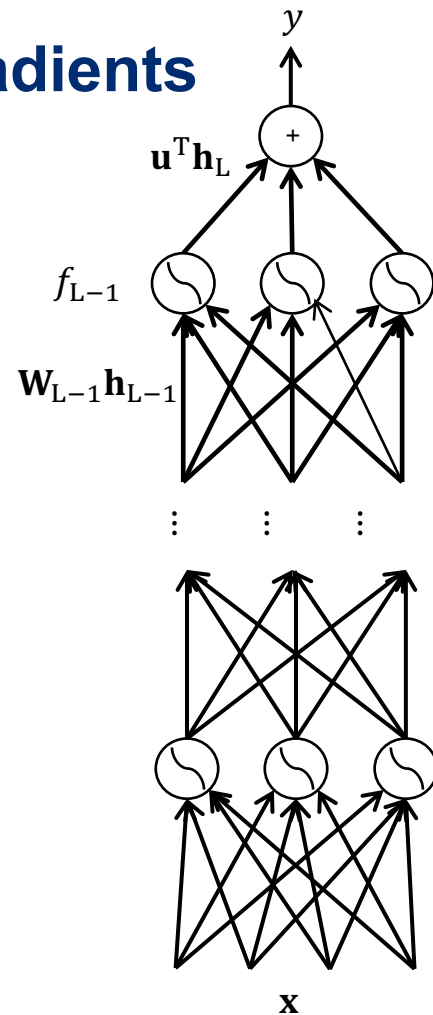
We want the gradients of $\mathcal{L}$ with respect to model parameters.

Use the chain rule to simplify the following term:

$$\nabla_{\mathbf{W}_{L-1}} \mathcal{L} = (\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-1}))^T =$$

$$\left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T$$

$$\boxed{\mathbf{W}_{L-1} \longrightarrow \mathbf{h}_L \longrightarrow y \longrightarrow \mathcal{L}}$$



A Generic Neural Network: Direct Gradients

$$\mathbf{x} = \mathbf{h}_0 \in \mathbb{R}^{d_0} \text{ (input)}$$

$$y = \mathbf{u}^T \mathbf{h}_L \in \mathbb{R} \text{ (output)}$$

$$\mathbf{h}_{i+1} = f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}}$$

$$\mathcal{L} = \ell(y, y^*) \in \mathbb{R} \text{ (loss)}$$

$$(0 \leq i \leq L - 1)$$

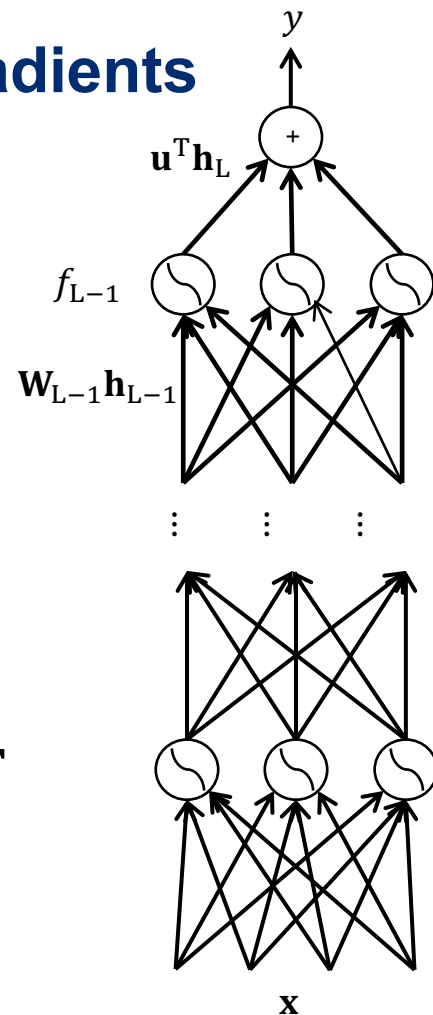
$$\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\}$$

We want the gradients of $\mathcal{L}$ with respect to model parameters.
Use the chain rule to simplify the following term:

$$\nabla_{\mathbf{W}_{L-2}} \mathcal{L} = (\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-2}))^T =$$

$$\left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{W}_{L-2}) \right)^T$$

$$\boxed{\mathbf{W}_{L-1} \longrightarrow \mathbf{h}_L \longrightarrow y \longrightarrow \mathcal{L}}$$



A Generic Neural Network: Direct Gradients

$\mathbf{x} = \mathbf{h}_0 \in \mathbb{R}^{d_0}$ (input)

$y = \mathbf{u}^T \mathbf{h}_L \in \mathbb{R}$ (output)

$\mathbf{h}_{i+1} = f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}}$

$\mathcal{L} = \ell(y, y^*) \in \mathbb{R}$ (loss)

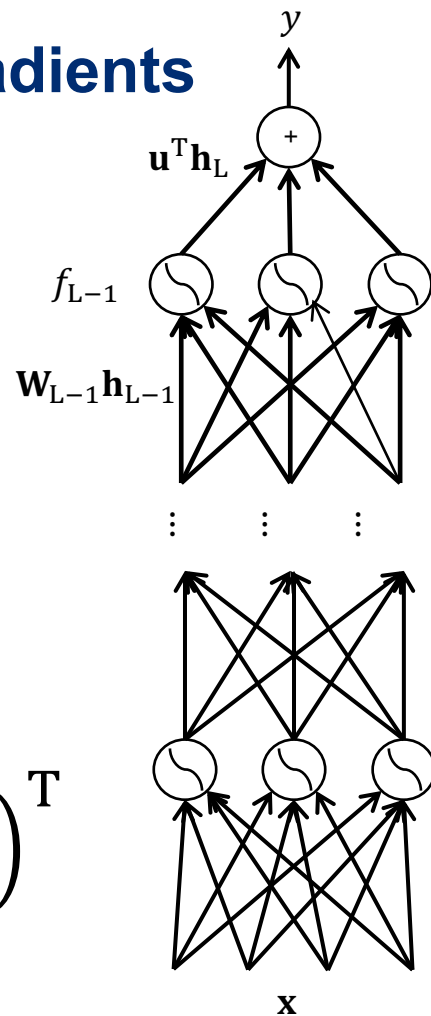
$(0 \leq i \leq L - 1)$

$\mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\}$

We want the gradients of $\mathcal{L}$ with respect to model parameters.
Use the chain rule to simplify the following term:

$$\nabla_{\mathbf{W}_{L-i}} \mathcal{L} = \left(\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-i}) \right)^T =$$

$$\left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \cdots \mathbf{J}_{\mathbf{h}_{L-i+1}}(\mathbf{W}_{L-i}) \right)^T$$

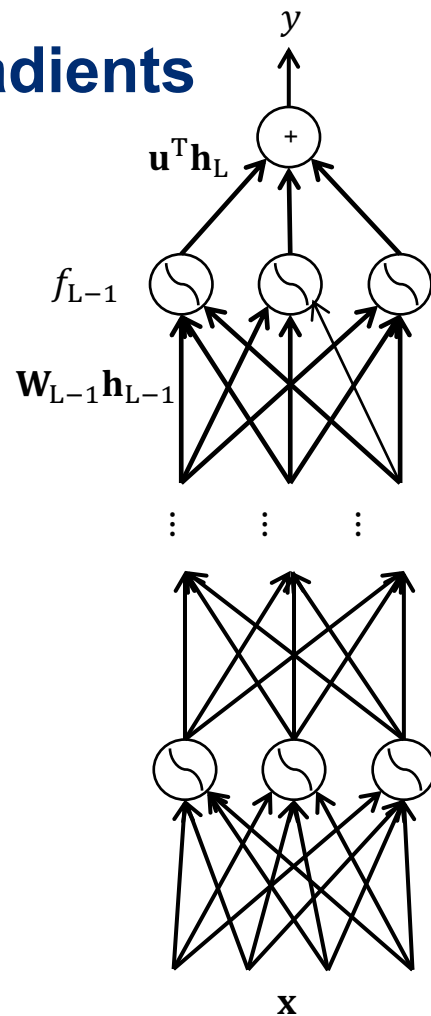


A Generic Neural Network: Direct Gradients

$$\nabla_{\mathbf{W}_{L-1}} \mathcal{L} = \left(\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-1}) \right)^T = \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T$$

Let's check the dimensions to ensure the math works:

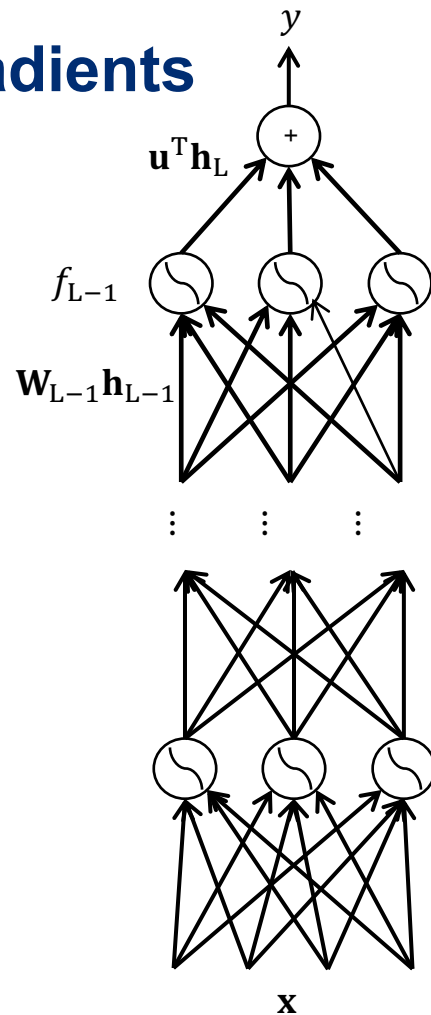
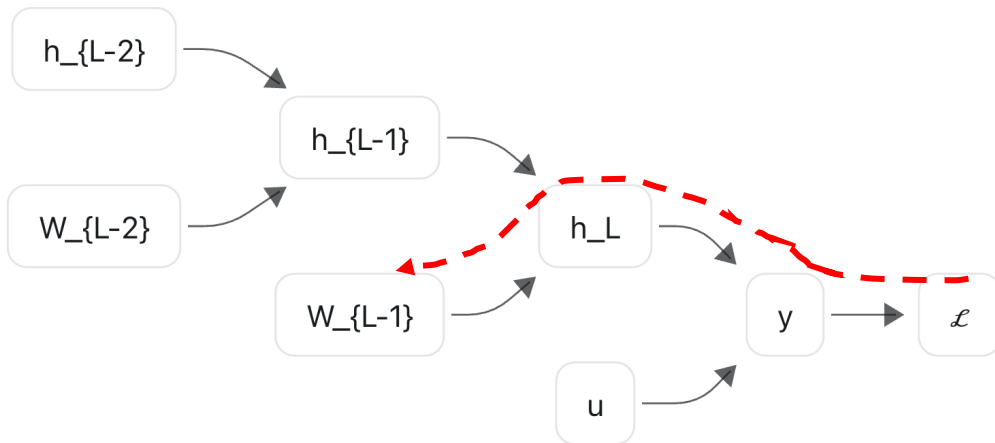
- What is the dimension of $\mathbf{W}_{L-1}$? $\mathbb{R}^{d_{L-1} \times d_L}$
- What should be the dim $\nabla_{\mathbf{W}_{L-1}} \mathcal{L}$? $\mathbb{R}^{d_{L-1} \times d_L}$
- Now check the dimensions of Jacobians:
 - $\mathbf{J}_{\mathcal{L}}(y) \in \mathbb{R}^{1 \times 1}$
 - $\mathbf{J}_y(\mathbf{h}_L) \in \mathbb{R}^{1 \times d_L}$
 - $\mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \in \mathbb{R}^{d_L \times d_L \times d_{L-1}}$
- So, the result of the chain rule lives in $\mathbb{R}^{1 \times d_L \times d_{L-1}}$
- A final transpose (and ignoring the dim=1) matches the expected dimension!



A Generic Neural Network: Direct Gradients

$$\nabla_{\mathbf{W}_{L-1}} \mathcal{L} = \left(\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-1}) \right)^T = \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T$$

- Why there are other params (besides $\mathbf{W}_{L-1}$) in this chain rule?
 - For example, why $\mathbf{u}$ doesn't appear here?
- The answer is, think about the graph of computation (below).
 - Other params don't appear on this path from $\mathbf{W}_{L-1}$ to $\mathcal{L}$.



A Generic Neural Network: Direct Gradients

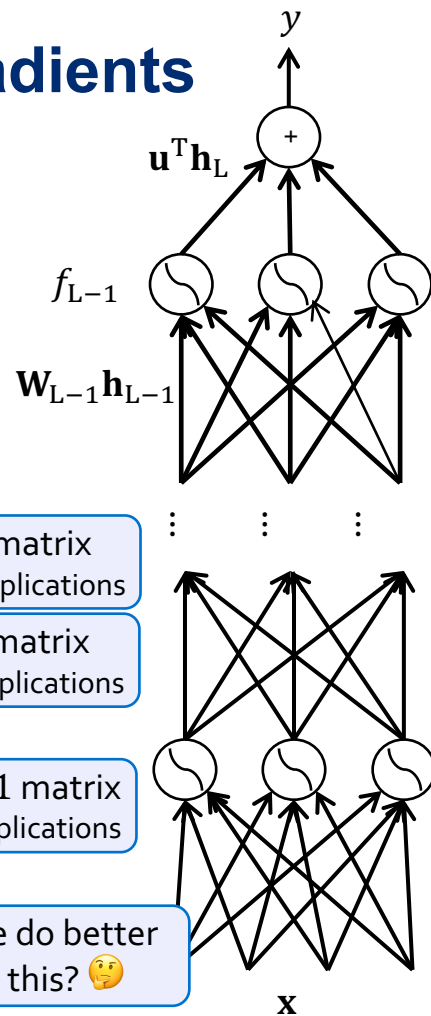
$$\begin{aligned}
 \mathbf{x} = \mathbf{h}_0 &\in \mathbb{R}^{d_0} \text{ (input)} & y = \mathbf{u}^T \mathbf{h}_L &\in \mathbb{R} \text{ (output)} \\
 \mathbf{h}_{i+1} &= f_i(\mathbf{W}_i \mathbf{h}_i) \in \mathbb{R}^{d_{i+1}} & \mathcal{L} = \ell(y, y^*) &\in \mathbb{R} \text{ (loss)} \\
 (0 \leq i \leq L-1) & & \mathbf{W} = \{\mathbf{W}_0, \mathbf{W}_1, \dots, \mathbf{W}_{L-1}, \mathbf{u}\} &
 \end{aligned}$$

We want the gradients of $\mathcal{L}$ with respect to model parameters.

- $\nabla_{\mathbf{W}_{L-1}} \mathcal{L} = (\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-1}))^T = (\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}))^T$
- $\nabla_{\mathbf{W}_{L-2}} \mathcal{L} = (\mathbf{J}_{\mathcal{L}}(\mathbf{W}_{L-2}))^T = (\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{W}_{L-2}))^T$
- ...
- $\nabla_{\mathbf{W}_0} \mathcal{L} = (\mathbf{J}_{\mathcal{L}}(\mathbf{W}_0))^T = (\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \dots \mathbf{J}_{\mathbf{h}_1}(\mathbf{W}_0))^T$

In total, how many matrix multiplications are done here?

- (A) $O(L)$ (B) $O(L^2)$ (C) $O(L^3)$ (C) $O(\exp(L))$



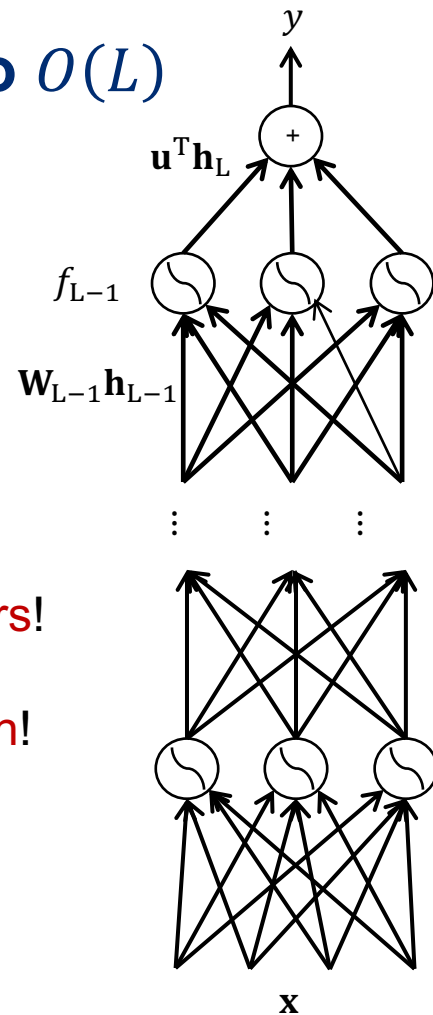
Caching Gradients to Turn $O(L^2)$ into $O(L)$

- Suppose we're computing:

$$\nabla_{\mathbf{W}_{L-1}} \mathcal{L} = \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T$$

$$\nabla_{\mathbf{W}_{L-2}} \mathcal{L} = \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{W}_{L-2}) \right)^T$$

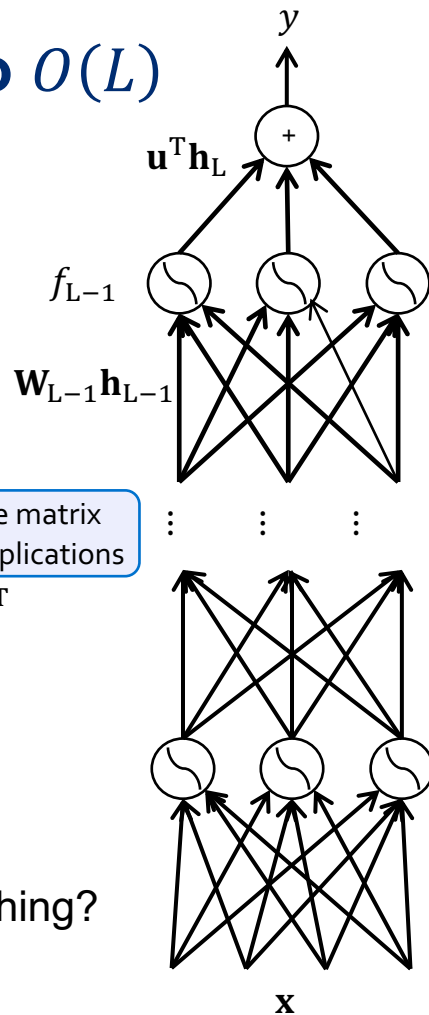
- Parameter gradients **depend on the gradients of the later layers!**
- When computing gradients, **we don't need to start from scratch!**
- I can **reuse gradients** computed for higher layers for lower layers (i.e., dynamic programming).



Caching Gradients to Turn $O(L^2)$ into $O(L)$

- First, precompute these cache values:
 - Initialize the output: $\delta_L = \mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L)$
 - Step back to earlier layers: $\delta_i = \delta_{i+1} \mathbf{J}_{\mathbf{h}_{i+1}}(\mathbf{h}_i), \forall i: 0 \leq i \leq L-1$
- Now, let's go back to the gradients:

$$\begin{aligned} \nabla_{\mathbf{W}_{L-1}} \mathcal{L} &= \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T = \left(\delta_L \mathbf{J}_{\mathbf{h}_L}(\mathbf{W}_{L-1}) \right)^T && \text{One matrix multiplication} \\ \nabla_{\mathbf{W}_{L-2}} \mathcal{L} &= \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{W}_{L-2}) \right)^T = \left(\delta_{L-1} \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{W}_{L-2}) \right)^T && \text{One matrix multiplication} \\ &\dots && \\ \nabla_{\mathbf{W}_0} \mathcal{L} &= \left(\mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \dots \mathbf{J}_{\mathbf{h}_1}(\mathbf{W}_0) \right)^T = \left(\delta_1 \mathbf{J}_{\mathbf{h}_1}(\mathbf{W}_0) \right)^T && \text{One matrix multiplication} \end{aligned}$$



In total, how many matrix multiplications are done here when using caching?

(A) $O(L)$ (B) $O(L^2)$ (C) $O(L^3)$ (D) $O(\exp(L))$

Gradient: Local Grad x Upstream Grad

- Gradients at each layer computed by

$$\nabla_{\mathbf{W}_i} \mathcal{L} = \left(\delta_{i+1} \mathbf{J}_{\mathbf{h}_{i+1}}(\mathbf{W}_i) \right)^T$$

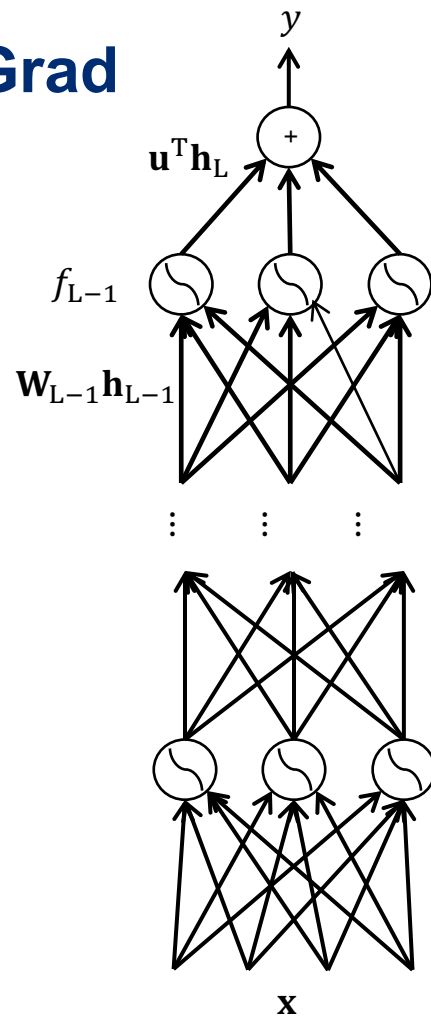
Upstream gradient ~ We lookup from the layer above.

Local Gradient

Let δ_i denote Jacobian at the output of layer i :

$$\delta_L = \mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L)$$

$$\delta_i = \delta_{i+1} \mathbf{J}_{\mathbf{h}_{i+1}}(\mathbf{h}_i)$$



A Generic Neural Network: Backward Step

- Compute loss gradients from the output toward the input, reusing the activations saved during the forward pass.

1. Initialize the loss gradient at the final hidden layer:

$$\delta_L = \mathbf{J}_{\mathcal{L}}(y) \mathbf{J}_y(\mathbf{h}_L)$$

2. For each layer $i = L - 1, \dots, 0$:

2.1 Compute the **param gradient**, using the local derivative with respect to weights:

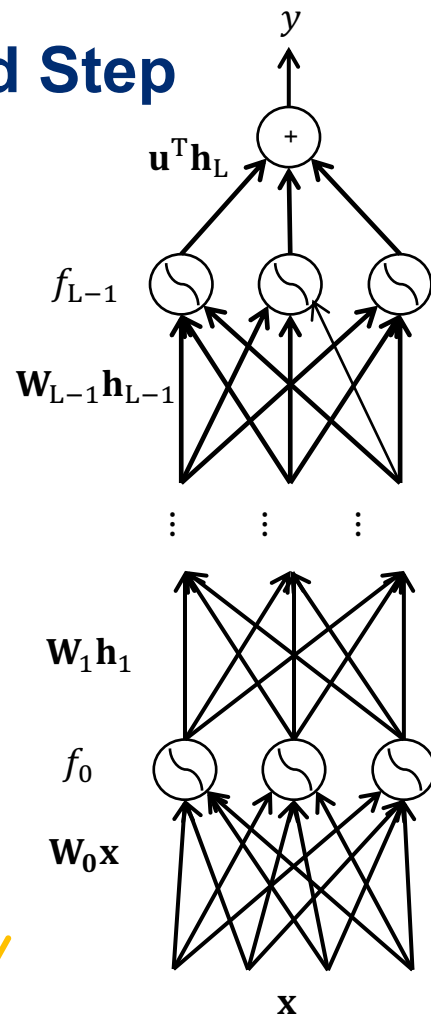
$$\nabla_{\mathbf{W}_i} \mathcal{L} = \left(\delta_{i+1} \mathbf{J}_{\mathbf{h}_{i+1}}(\mathbf{W}_i) \right)^T$$

2.2 Propagate the gradient to the layer's input, via the **local** grad of that input:

$$\delta_i = \delta_{i+1} \mathbf{J}_{\mathbf{h}_{i+1}}(\mathbf{h}_i)$$

- Computational cost:

- For fixed widths, the backward pass costs $O(L)$, the same asymptotic order as the forward pass.
- Independently recomputing the full chain for every layer costs $O(L^2)$.



A Generic Neural Network: Back Propagation

Initialize network parameters with random values

Loop until convergence

Loop over training instances

In practice, this step is done over **batches** of instances!

i. Forward step:

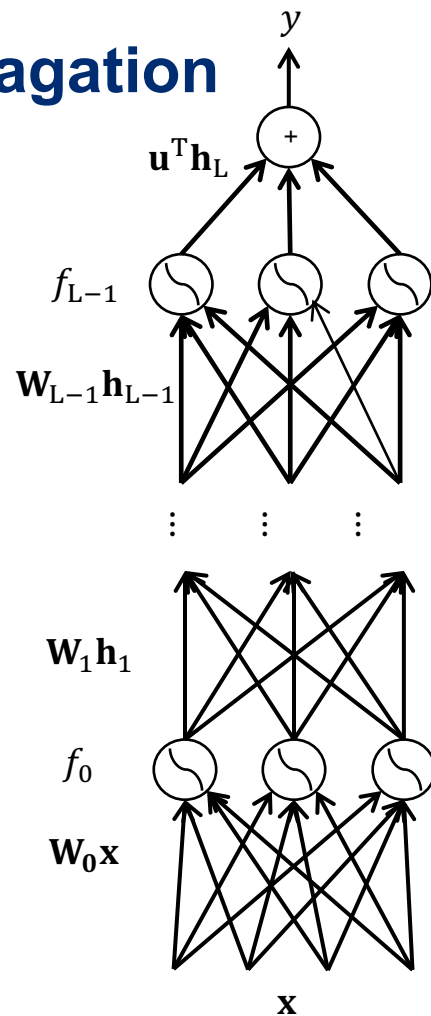
Starting from the input, compute the activations and compute loss at the end $\mathcal{L}$

ii. Backward step:

Compute gradients with respect to each param $\nabla_{\mathcal{L}}(\mathbf{W}_i)$

iii. Gradient update:

Update each parameter: $\mathbf{W}_i^{(t+1)} \leftarrow \mathbf{W}_i^{(t)} - \alpha \nabla_{\mathbf{W}_i} \mathcal{L}$



Summary

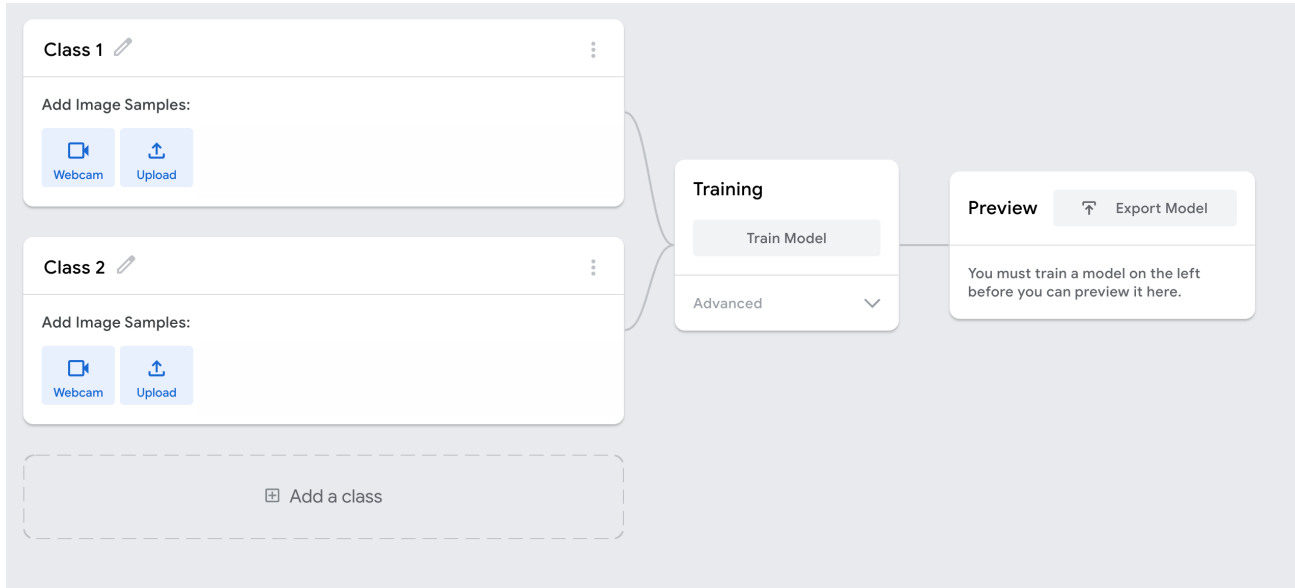
- Backpropagation: an algorithm for efficient computation of gradients in deep nets.
- Using Dynamic Programming for efficient computation of gradients.
- **Next:** Backprop in real practice.

Quick Pulse Check

- How is **Backprop** different from/related to **Gradient Descent**?
 - **Backpropagation** is an algorithm used to compute the gradient of the loss function with respect to each weight in the network.
 - **Gradient descent**, on the other hand, is an optimization algorithm that uses these computed gradients to update the network's weights.
 - *Basically, backpropagation provides the necessary ingredients for gradient descent to perform the weight updates.*

Demo Time!

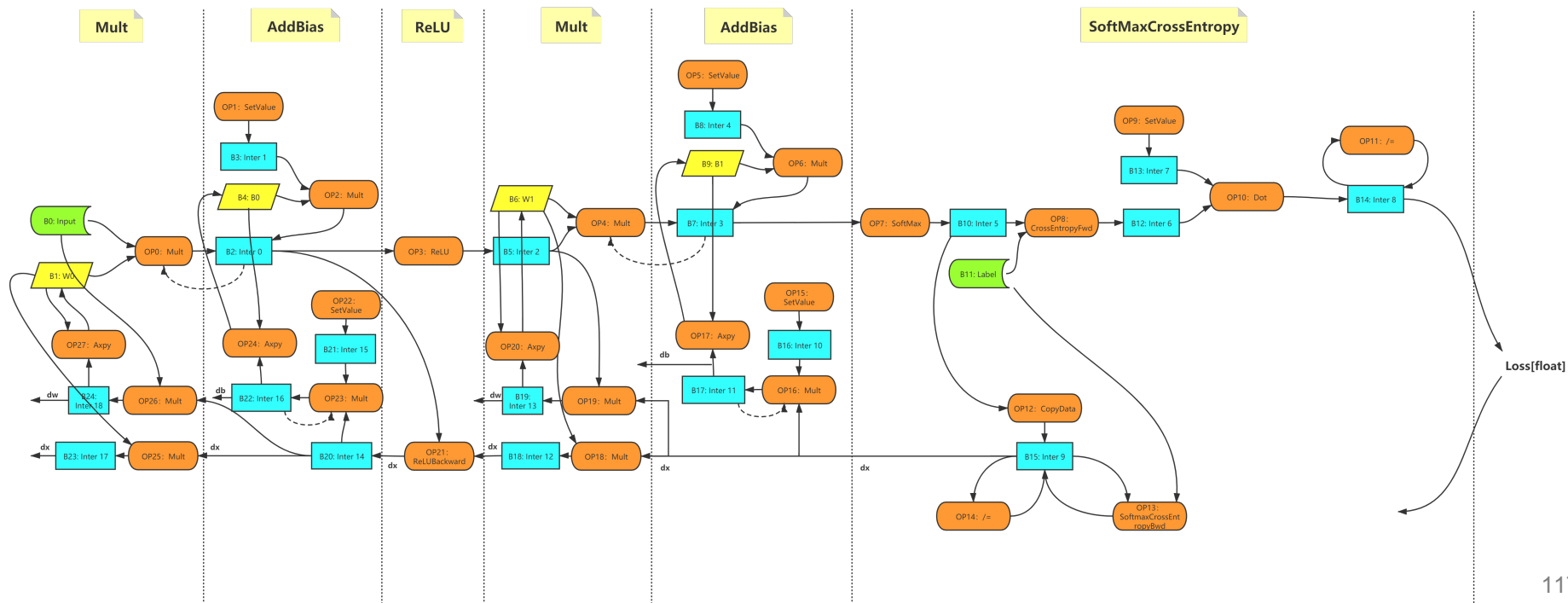
- <https://teachablemachine.withgoogle.com/>



Backprop in Real Practice: Computation Graph

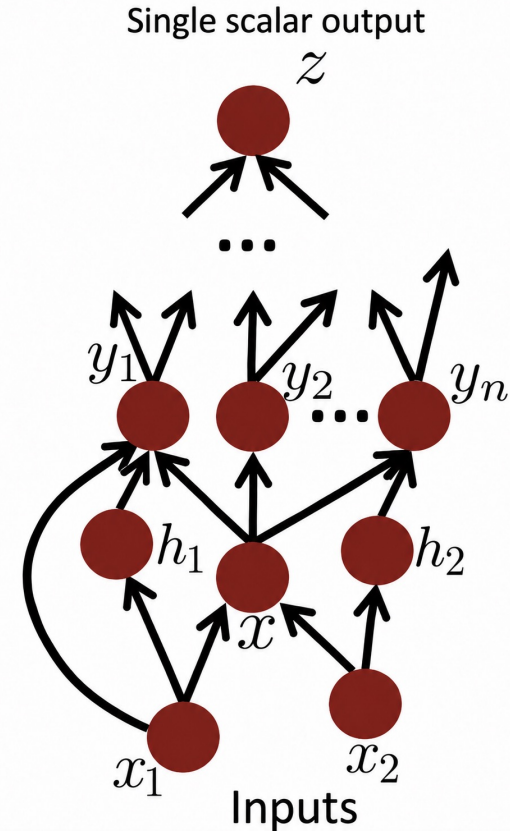
Computation Graph: Example

- In reality, neural networks are not as regular as the previous example ...



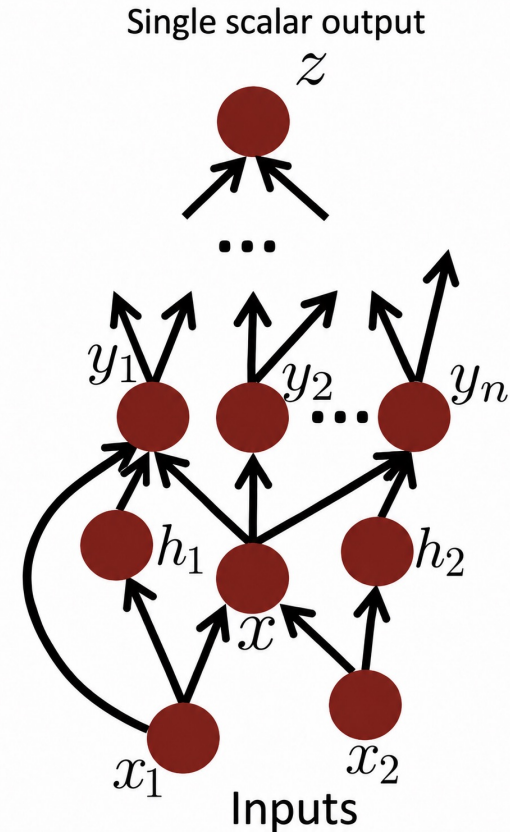
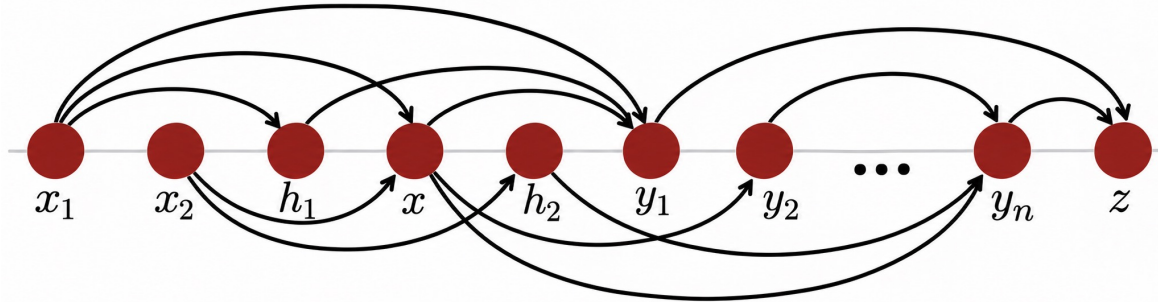
Backprop in General Computation Graph

- What if the network does not have a regular structure? Same idea!



Backprop in General Computation Graph

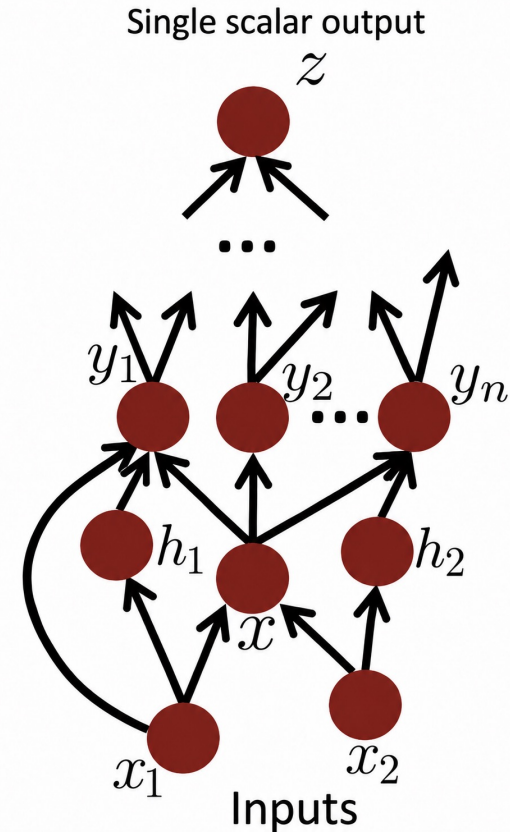
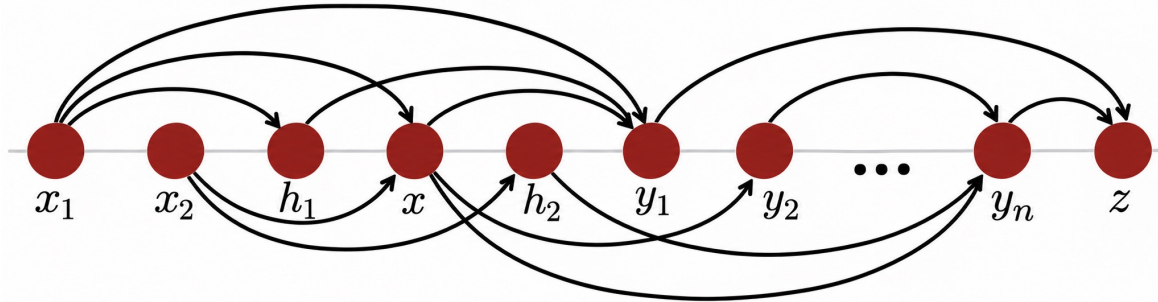
- What if the network does not have a regular structure? Same idea!
- Sort the nodes in **topological order** (what depends on what)
 - Common choice: Kahn's Algorithm or DFS-based.
 - Cost:** Linear in the number of nodes/edges.



Backprop in General Computation Graph

- What if the network does not have a regular structure? Same idea!

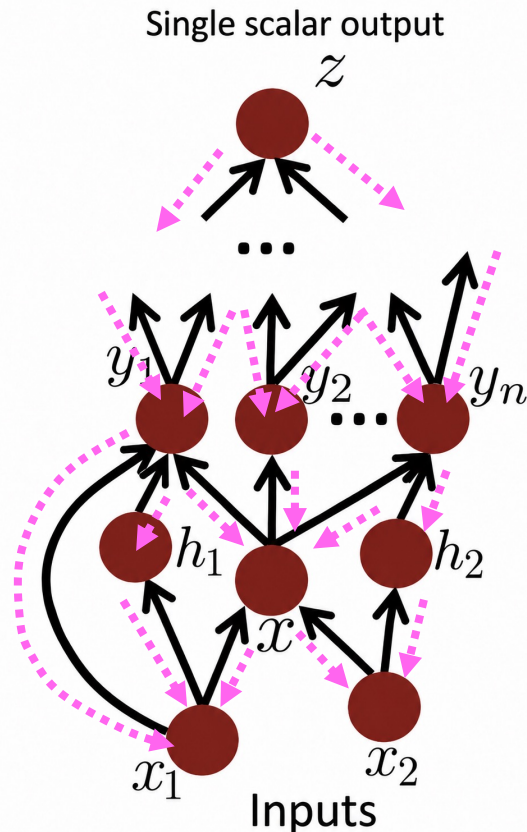
- Sort the nodes in **topological order** (what depends on what)
- Forward-Propagation:
 - Visit nodes in topological sort order and compute value of node given predecessors
 - Cost:** Graph traversal is linear in the number of node+edges



Backprop in General Computation Graph

- What if the network does not have a regular structure? Same idea!

- Sort the nodes in **topological order** (what depends on what)
- Forward-Propagation:
 - Visit nodes in topological sort order and compute value of node given predecessors
- Backward-Propagation:
 - Initialize the output's gradient to 1.
 - Visit nodes in reverse topological order.
 - At each node, compute its **global gradients** by **summing contributions from all successors**, each weighted by the corresponding **local gradients**.
 - Cost:** Linear in the number of nodes/edges.



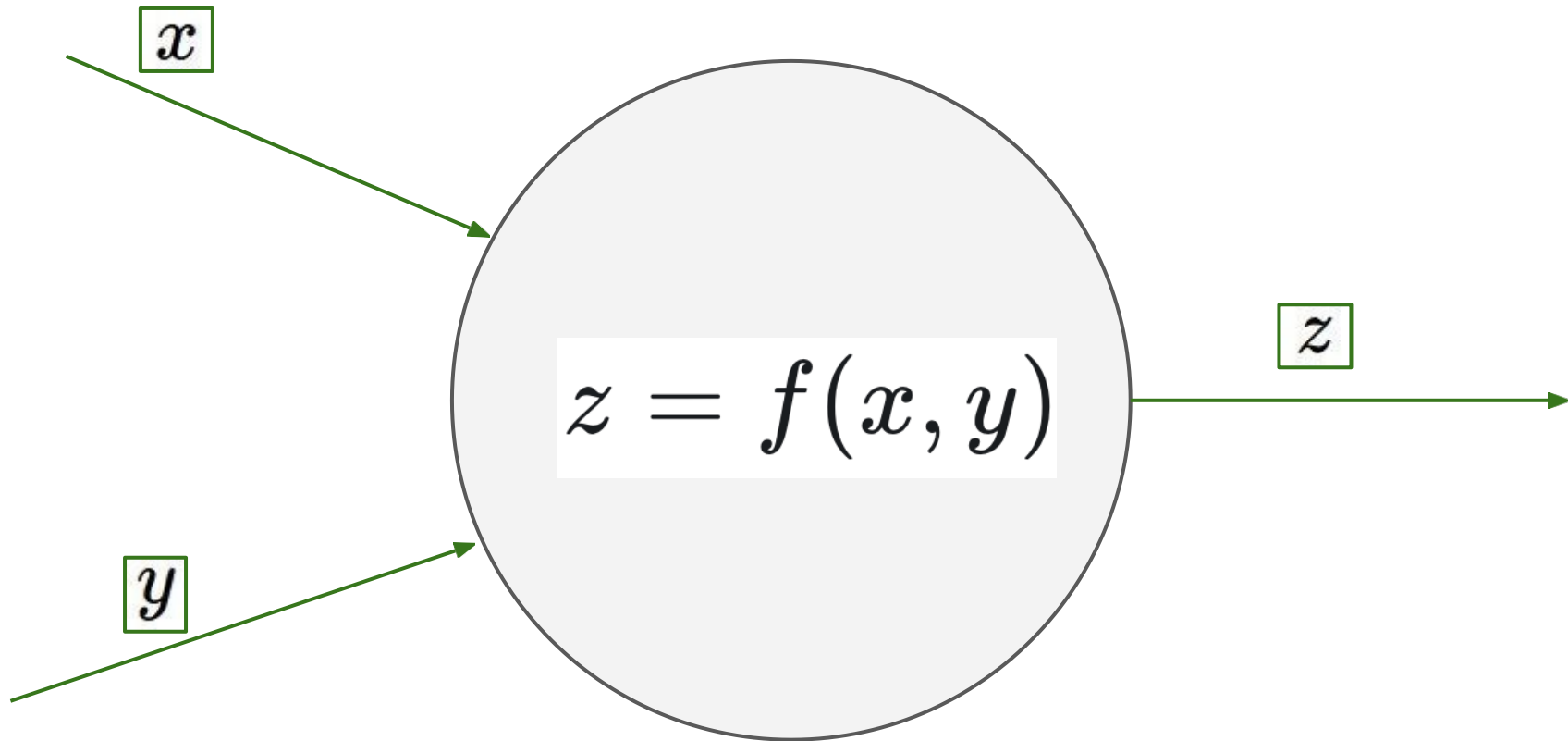
┌ Computation Graph:
View inside each node └

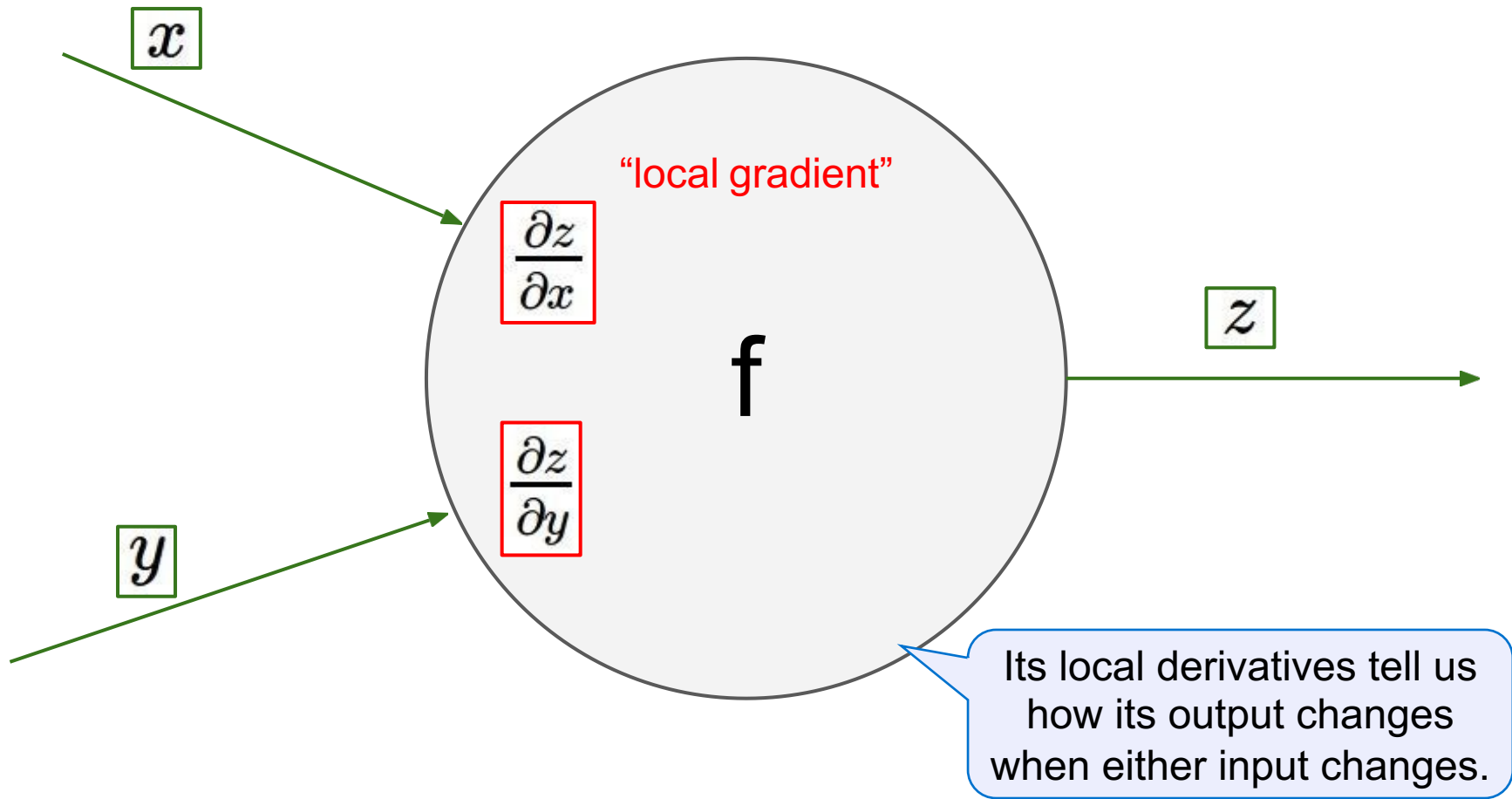
Computations inside a node

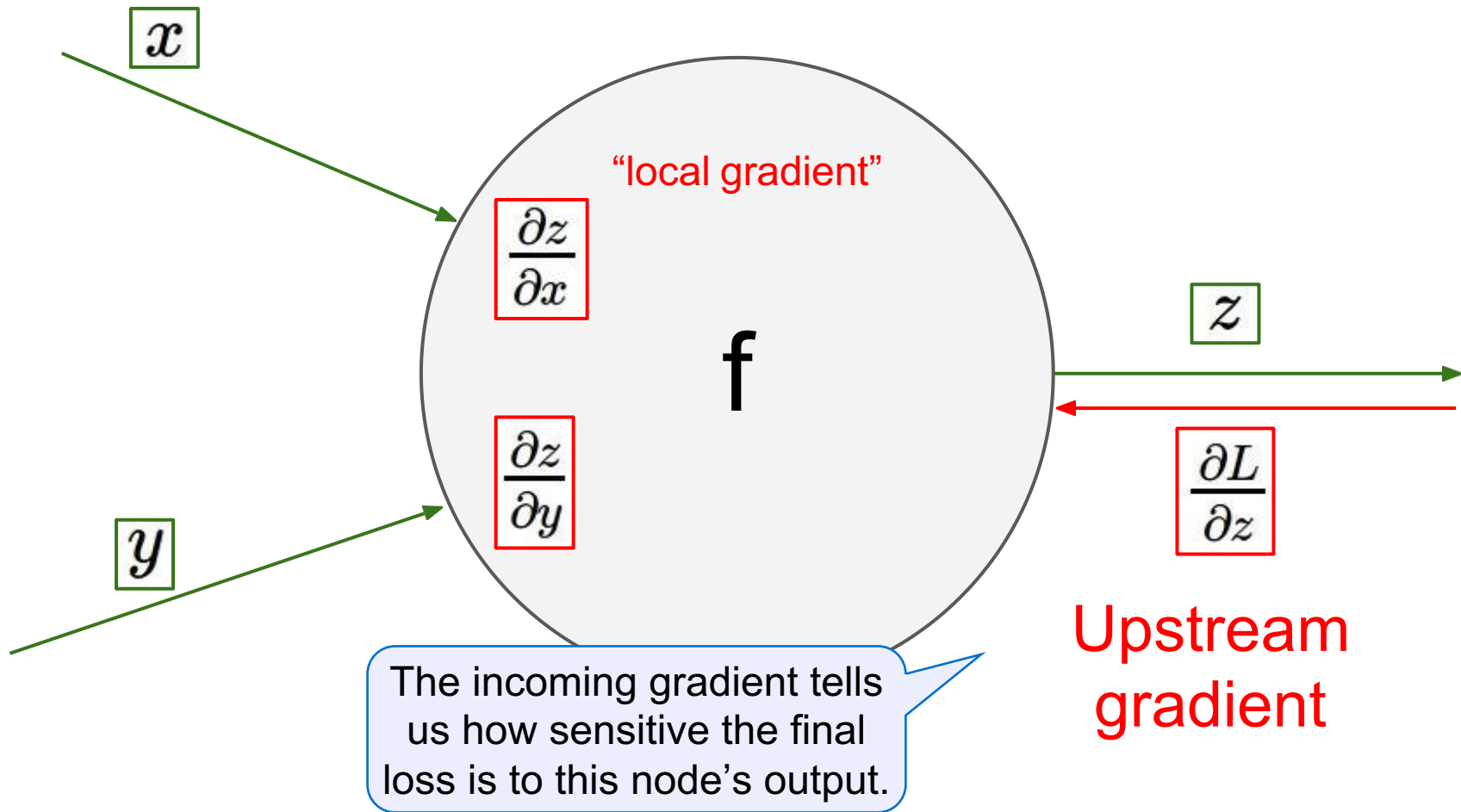
- Think of a single node z .

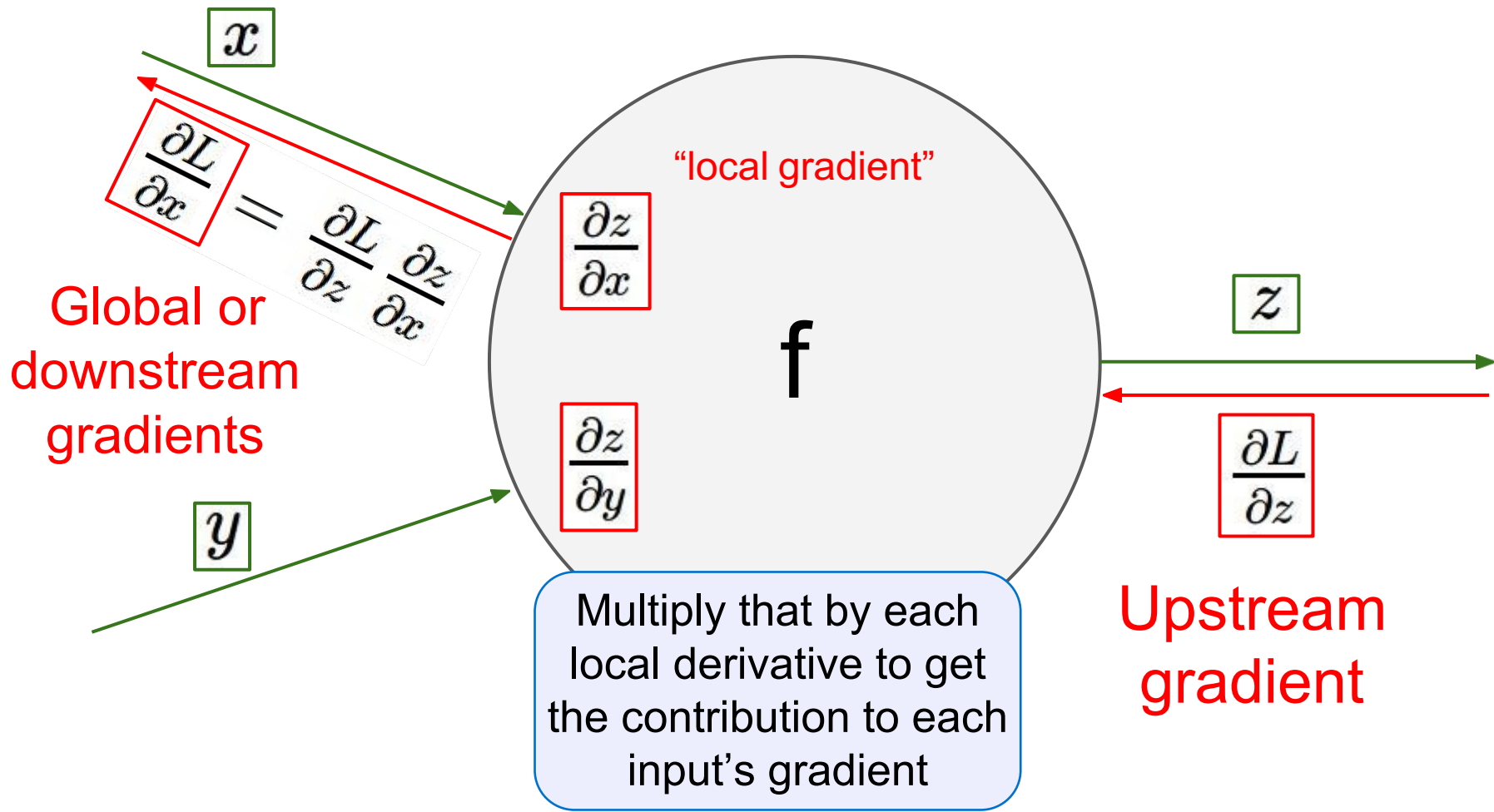
$$z = f(x, y)$$

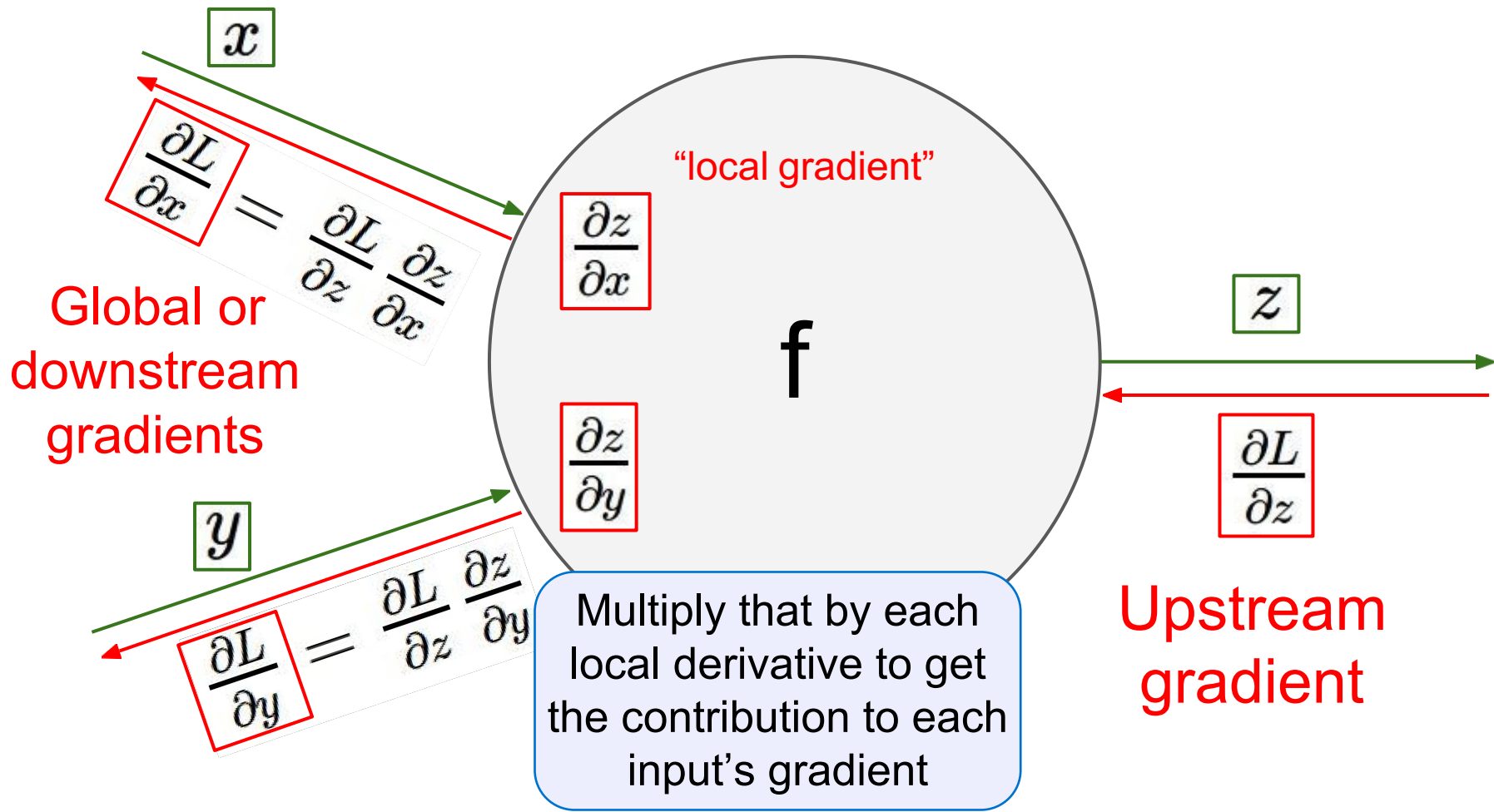
- The node only needs to understand its own operation to pass an upstream gradient (z) to the earlier layers (x, y).

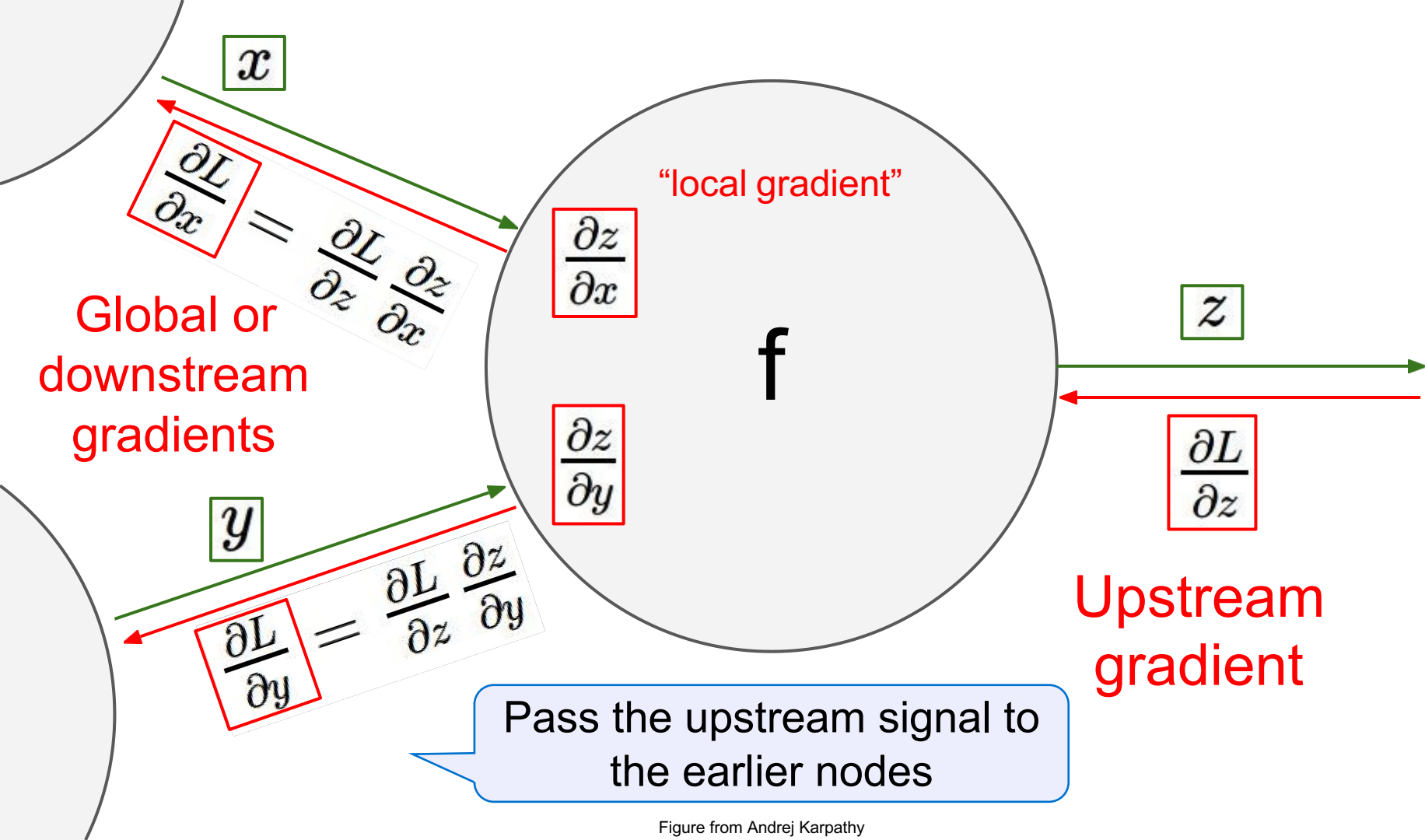












An important takeaway:

The implementation of backpropagation is **distributed** across the nodes of the computation graph: each node uses its local derivatives and the incoming gradient to accumulate gradient contributions for its inputs.

Global or
downstream
gradients

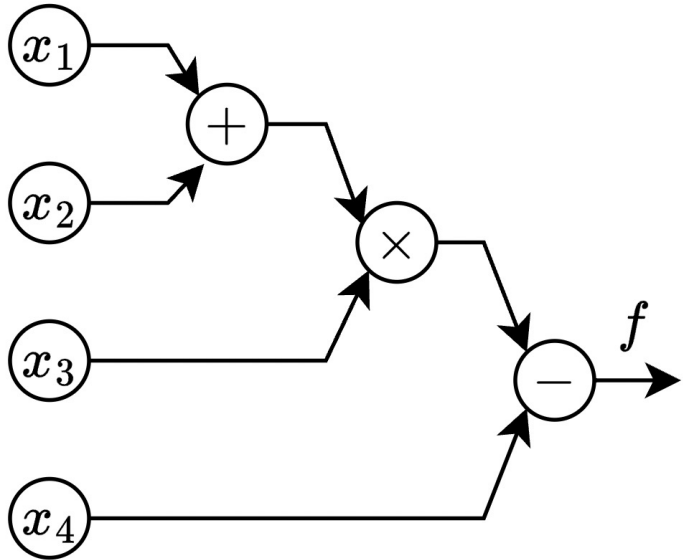
stream
gradient

Computation Graph: An Example

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$

Want: $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial x_4}$

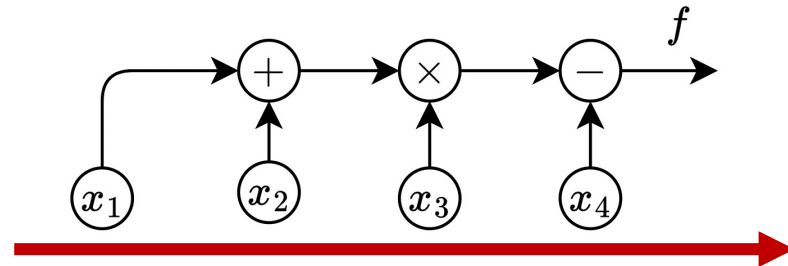
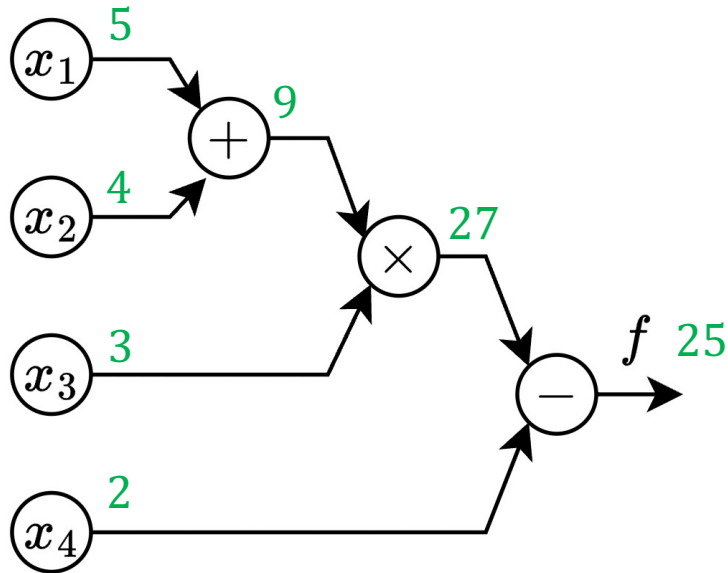


In what order should we process the **forward** step?

Computation Graph: An Example

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$

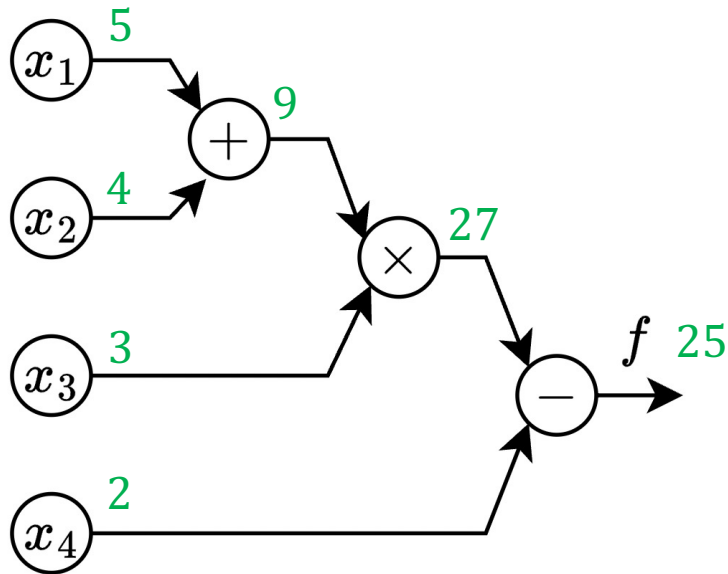


In what order should we process the **forward** step?

Computation Graph: An Example

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



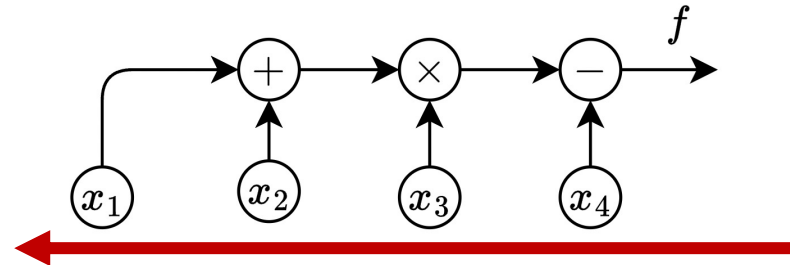
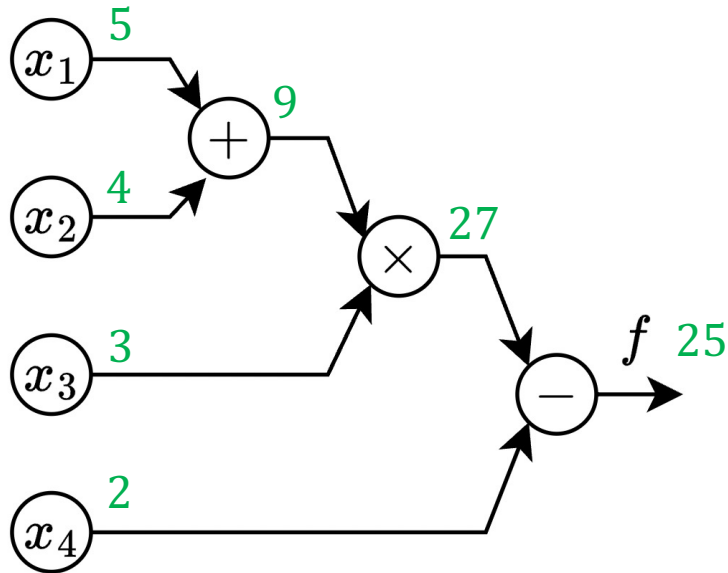
Want: $\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial x_4}$

In what order should we process the **backward** step?

Computation Graph: An Example

$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

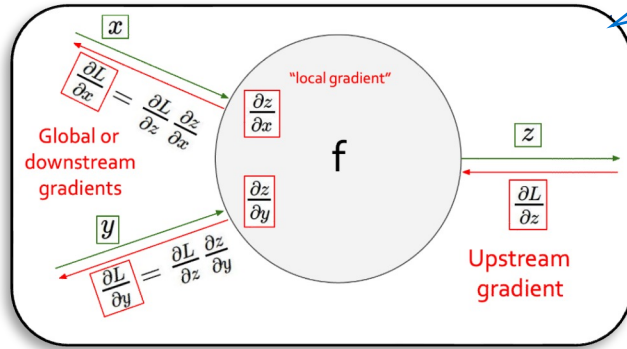
Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



In what order should we process the **backward** step?

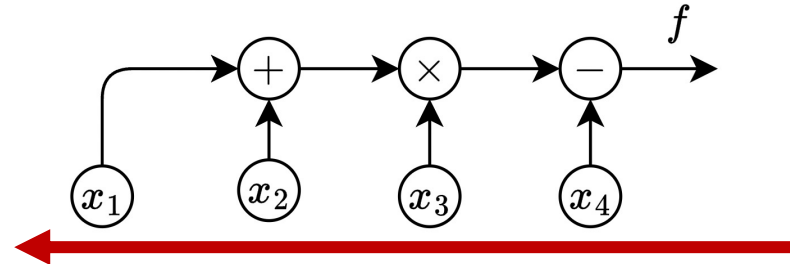
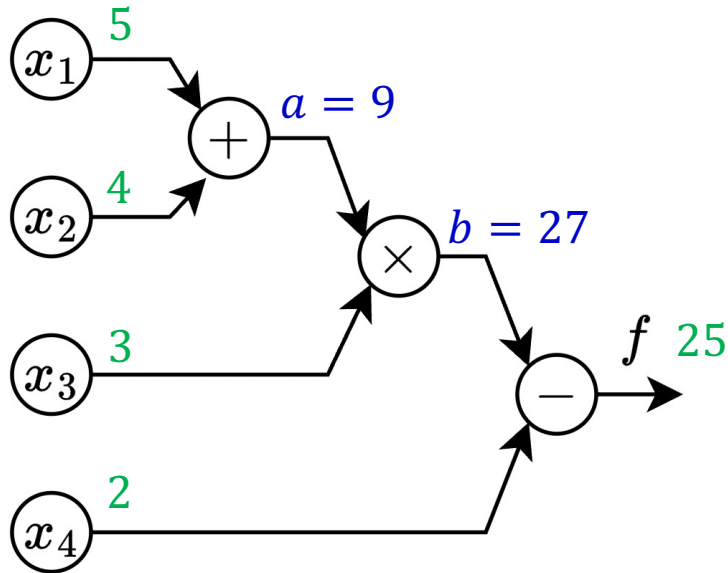
Remember this!

Computation Graph: An Example



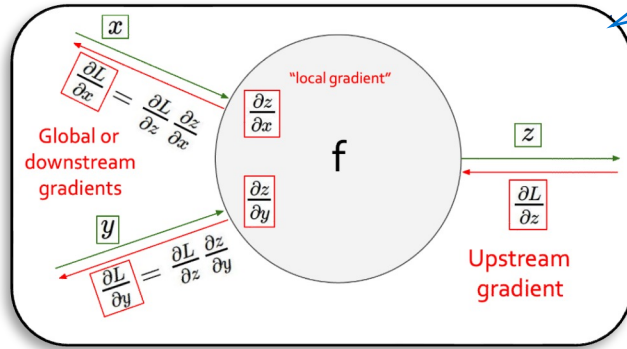
$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



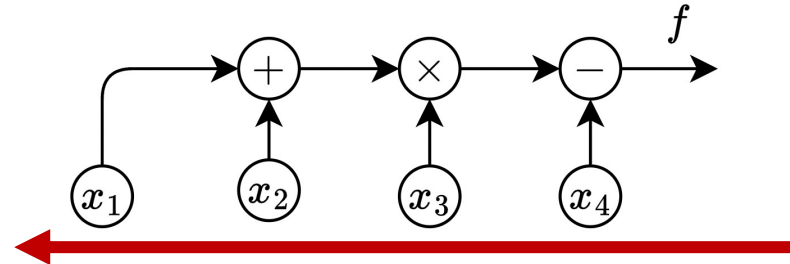
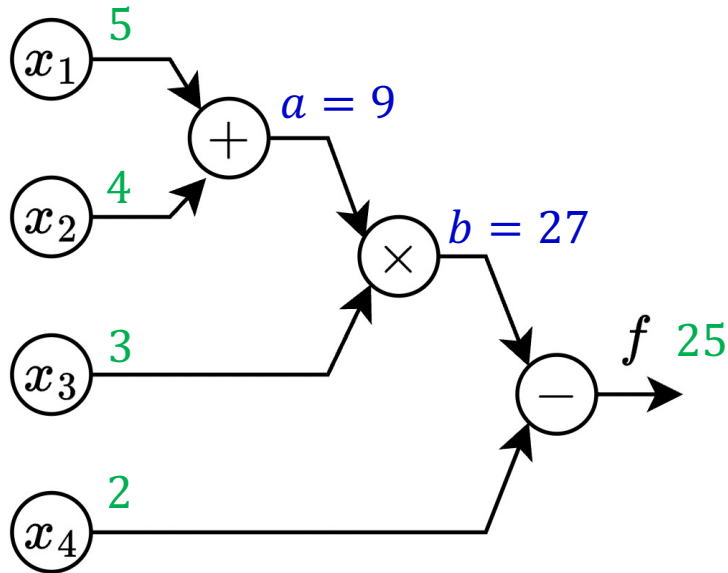
Remember this!

Computation Graph: An Example



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$

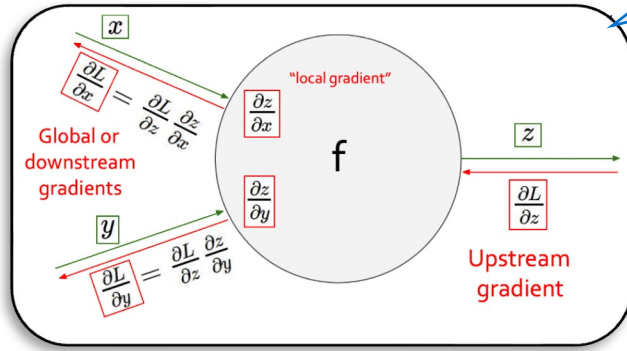


Name the intermediate variables. These intermediate values make the backward pass easy to follow.

Remember this!

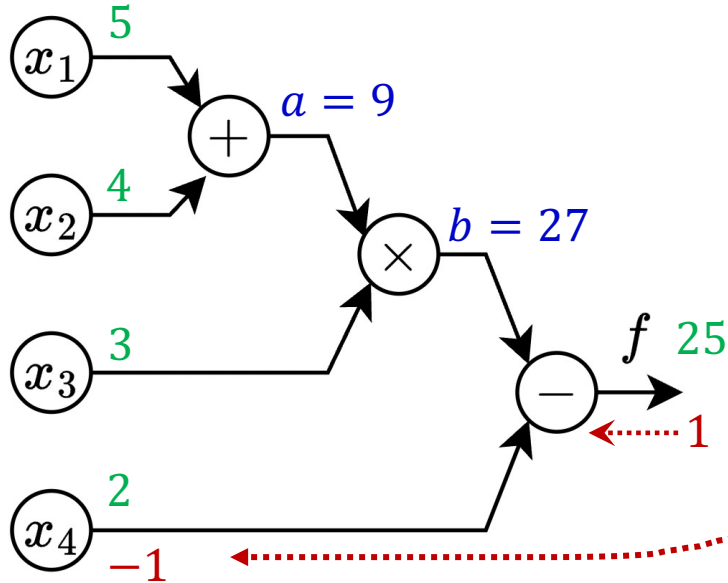
U: Upstream grad
L: Local grad

Computation Graph: An Example



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



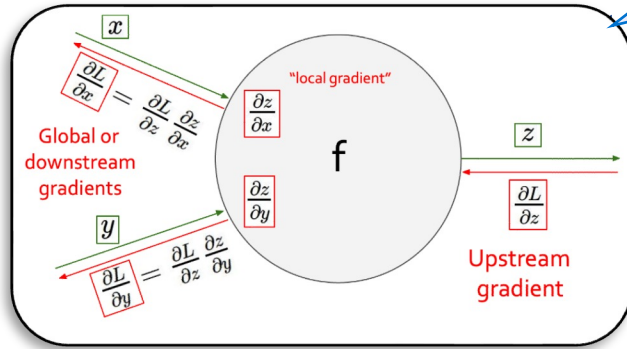
$$f = b - x_4$$

$$\frac{\partial f}{\partial x_4} = \overbrace{\frac{\partial f}{\partial x_4}}^L \times \overbrace{\frac{\partial f}{\partial f}}^U = (-1) \times 1 = -1$$

Remember this!

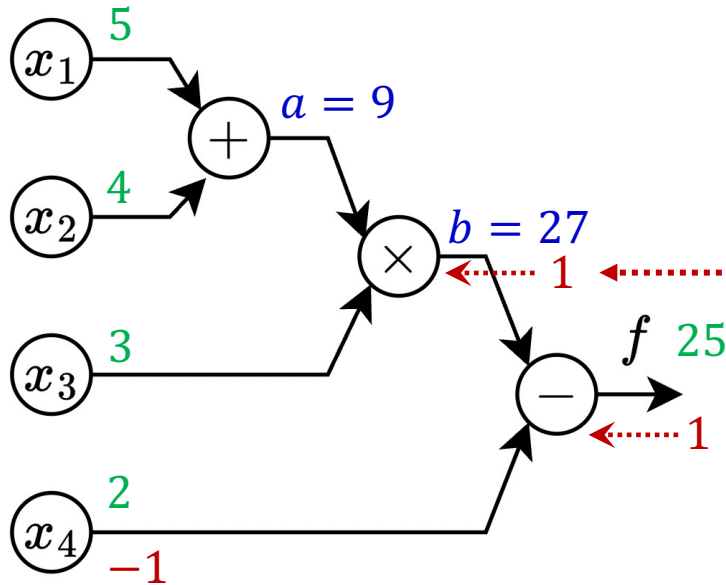
Computation Graph: An Example

U: Upstream grad
L: Local grad



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



$$f = b - x_4$$

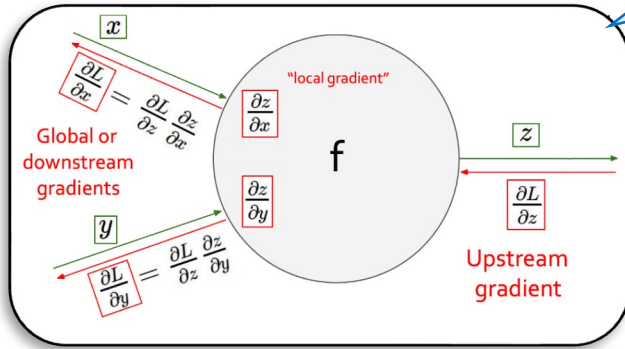
$$\frac{\partial f}{\partial x_4} = \overbrace{\frac{\partial f}{\partial x_4}}^L \times \overbrace{\frac{\partial f}{\partial f}}^U = (-1) \times 1 = -1$$

$$\frac{\partial f}{\partial b} = \overbrace{\frac{\partial f}{\partial b}}^L \times \overbrace{\frac{\partial f}{\partial f}}^U = 1 \times 1 = 1$$

Remember this!

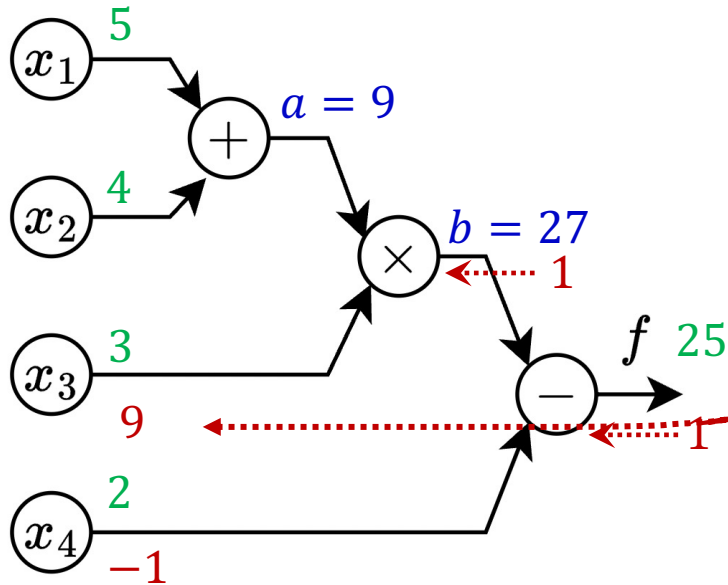
U: Upstream grad
L: Local grad

Computation Graph: An Example



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



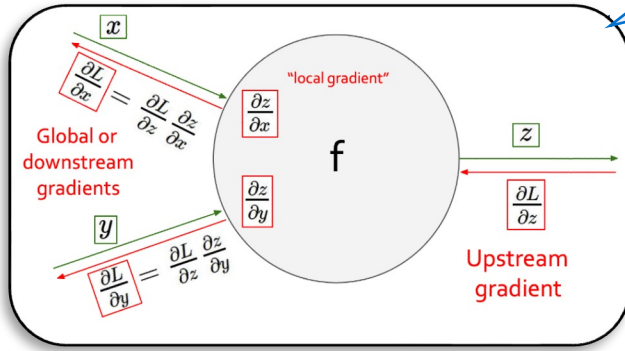
$$b = a \times x_3$$

$$\frac{\partial f}{\partial x_3} = \overbrace{\frac{\partial b}{\partial x_3}}^L \times \overbrace{\frac{\partial f}{\partial b}}^U = a \times 1 = 9$$

Remember this!

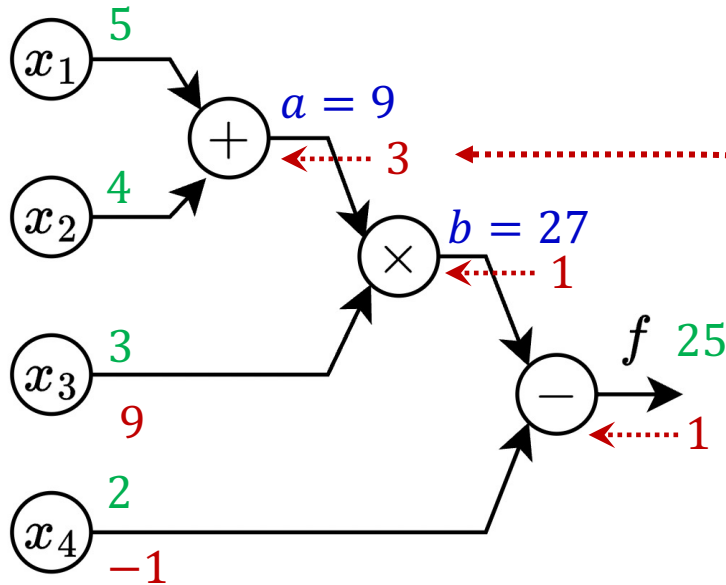
Computation Graph: An Example

U: Upstream grad
L: Local grad



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



$$b = a \times x_3$$

$$\frac{\partial f}{\partial x_3} = \frac{\partial b}{\partial x_3} \times \frac{\partial f}{\partial b} = a \times 1 = 9$$

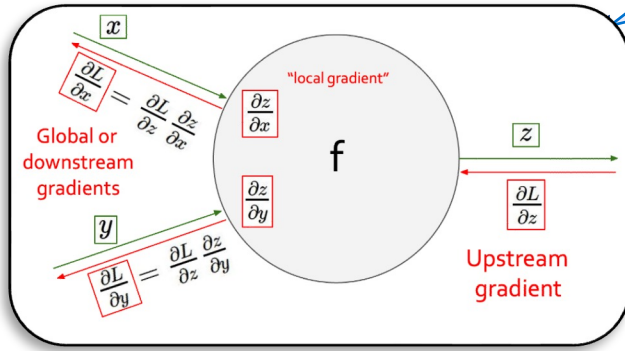
$$\frac{\partial f}{\partial a} = \frac{\partial b}{\partial a} \times \frac{\partial f}{\partial b} = x_3 \times 1 = 3$$

Remember this!

Computation Graph: An Example

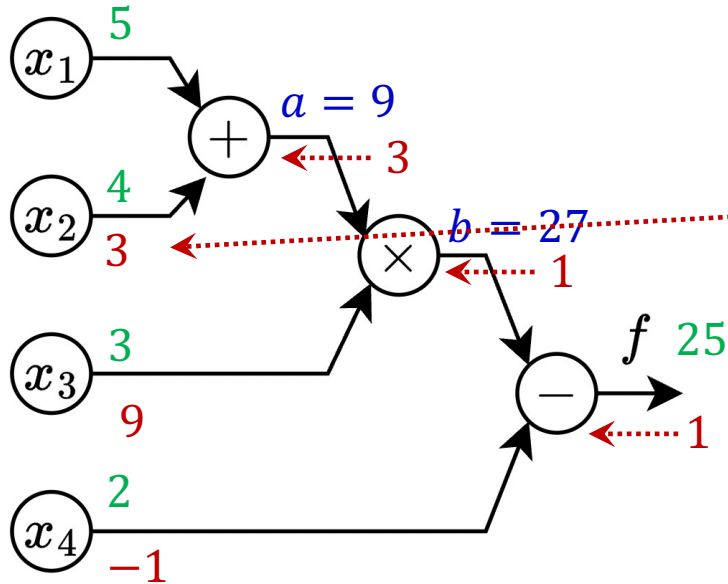
U: Upstream grad

L: Local grad



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



$$a = x_1 + x_2$$

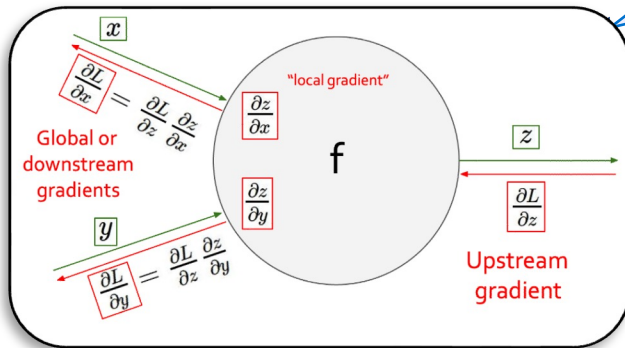
$$\frac{\partial f}{\partial x_2} = \frac{\partial f}{\partial a} \times \frac{\partial a}{\partial x_2} = 1 \times 3 = 3$$

Remember this!

Computation Graph: An Example

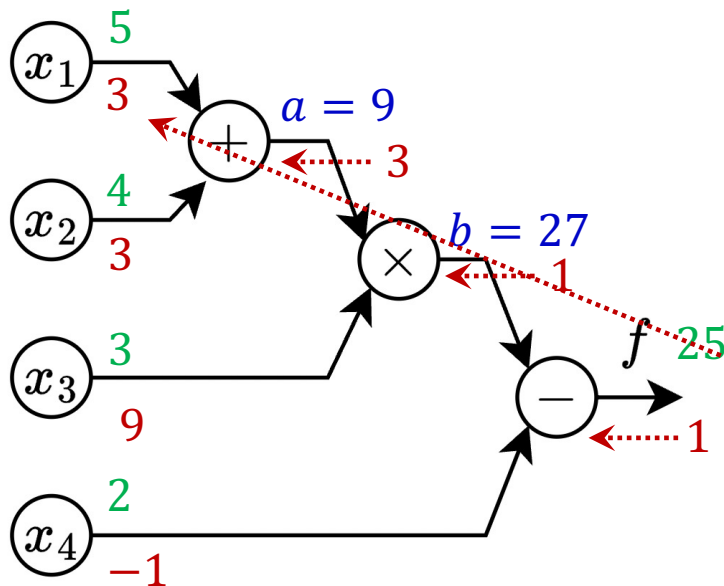
U: Upstream grad

L: Local grad



$$f(x_1, x_2, x_3, x_4) = (x_1 + x_2)x_3 - x_4$$

Evaluated at: $(x_1, x_2, x_3, x_4) = (5, 4, 3, 2)$



$$a = x_1 + x_2$$

$$\frac{\partial f}{\partial x_2} = \frac{\partial a}{\partial x_2} \times \frac{\partial f}{\partial a} = 1 \times 3 = 3$$

$$\frac{\partial f}{\partial x_1} = \frac{\partial a}{\partial x_1} \times \frac{\partial f}{\partial a} = 1 \times 3 = 3$$

Summary

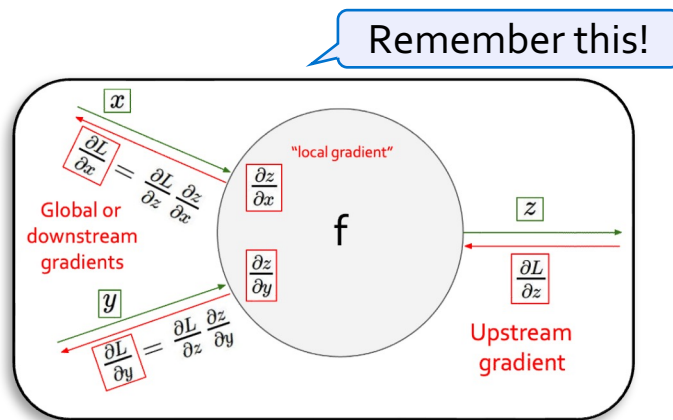
- **Computation graphs:** directed graph where the nodes correspond to mathematical operations.
 - A way of expressing mathematical operations.
- This allows general-purpose implementation of Backprop to **any** neural network.
 - This is why in practice you don't need to worry about implementing Backprop!!
- **Next:** Implementing Backprop yourself + industrial software libraries.



Backprop via Automatic Differentiation

How Should We Implement BackProp?

- The computation graph makes it easy to backpropagate all the way
- We need to implement:
 - The nodes are the values (tensors/values)
 - Connection of nodes (how tensors/values get propagated.)
- Let's think about how this is implemented!



What we actually want to run

```
# Creating tensors
x = Tensor(3.0, requires_grad=True)
y = Tensor(2.0, requires_grad=True)

# Forward pass
z = multiply(x, y) # z = x * y
w = add(z, y)      # w = z + y
o = relu(w)        # o = ReLU(w)

# Backpropagation
o.backward()

# Print gradients
print(f"x.grad: {x.grad}") # Should be y.value if ReLU is active
print(f"y.grad: {y.grad}") # Should be x.value + 1 (from addition)
```

What elements do we need to define here?

The Nodes

- The nodes are the values (tensors)
 - They store value, gradients and references to the functions that created them.

```
class Tensor:
    def __init__(self, value, requires_grad=False):
        self.value = value # Store the value of the tensor
        self.grad = 0 # Gradient initialized to zero
        self.requires_grad = requires_grad
        self._backward = lambda: None # Function to compute gradient
        self._prev = set() # Track previous nodes for backpropagation
```

The Nodes

```
class Tensor:
    def __init__(self, value, requires_grad=False):
        self.value = value  # Store the value of the tensor
        self.grad = 0  # Gradient initialized to zero
        self.requires_grad = requires_grad
        self._backward = lambda: None  # Function to compute gradient
        self._prev = set()  # Track previous nodes for backpropagation
```

Creating tensors

```
x = Tensor(3.0, requires_grad=True)
y = Tensor(2.0, requires_grad=True)
```

Now you can
define tensors!!

What we actually want to run

```
# Creating tensors
x = Tensor(3.0, requires_grad=True)
y = Tensor(2.0, requires_grad=True)

# Forward pass
z = multiply(x, y) # z = x * y
w = add(z, y)      # w = z + y
o = relu(w)        # o = ReLU(w)

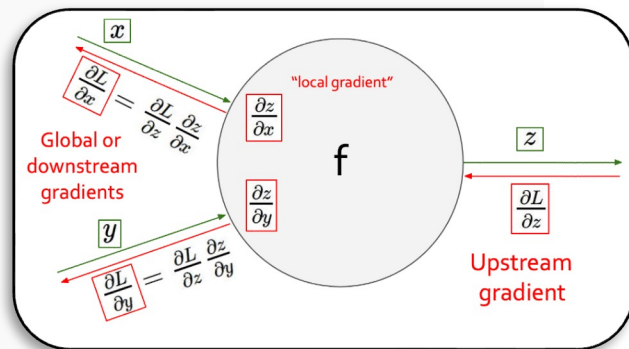
# Backpropagation
o.backward()

# Print gradients
print(f"x.grad: {x.grad}") # Should be y.value if ReLU is active
print(f"y.grad: {y.grad}") # Should be x.value + 1 (from addition)
```

What portion of this is defined so far?

Defining Operation Nodes

```
def multiply(a, b):  
    """Multiplication operation."""  
    out = Tensor(a.value * b.value, requires_grad=(a.requires_grad or b.requires_grad))  
  
    def _backward():  
        if a.requires_grad: Local grad  
            a.grad += b.value * out.grad  
        if b.requires_grad: Upstream  
            b.grad += a.value * out.grad  
        Important: +=, rather than =.  
    out._backward = _backward  
    out._prev = {a, b}  
    return out
```



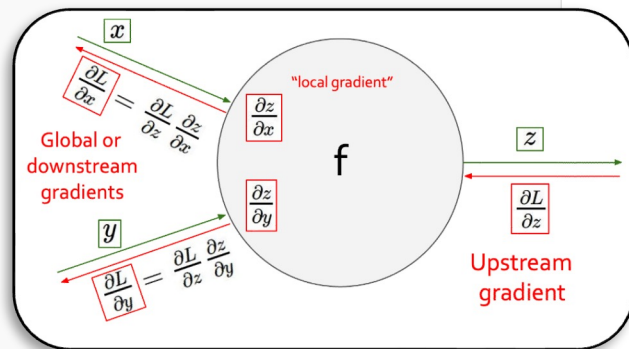
Defining Operation Nodes

```
def add(a, b):  
    """Addition operation."""  
    out = Tensor(a.value + b.value, requires_grad=(a.requires_grad or b.requires_grad))  
  
    def _backward():  
        if a.requires_grad:  
            a.grad += out.grad  
        if b.requires_grad:  
            b.grad += out.grad  
        out._backward = _backward  
        out._prev = {a, b}  
        return out
```

Important: +=, rather than =.

Upstream

Where is the local grad? 🤔



Defining Operation Nodes

- Operations (define how it propagates forward/backward) are also nodes!

```
def relu(x):  
    """ReLU activation function."""  
    out = Tensor(max(0, x.value), requires_grad=x.requires_grad)  
  
    def _backward():  
        if x.requires_grad:  
            x.grad += (out.value > 0) * out.grad # Gradient is 1 if x > 0, else 0  
  
    out._backward = _backward  
    out._prev = {x}  
    return out
```

Upstream

What we actually want to run

```
# Creating tensors
x = Tensor(3.0, requires_grad=True)
y = Tensor(2.0, requires_grad=True)
```

```
# Forward pass
z = multiply(x, y) # z = x * y
w = add(z, y)     # w = z + y
o = relu(w)       # o = ReLU(w)
```

```
# Backpropagation
o.backward()
```

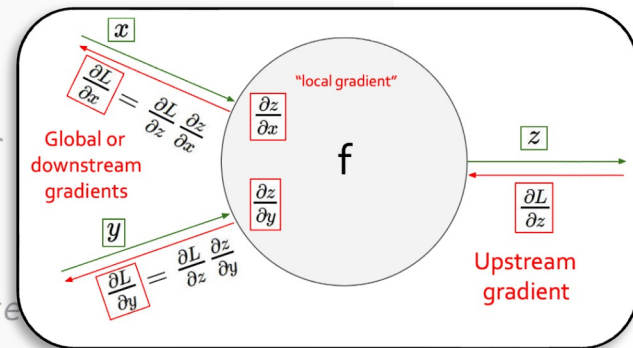
```
# Print gradients
print(f"x.grad: {x.grad}") # Should be y.value if ReLU is active
print(f"y.grad: {y.grad}") # Should be x.value + 1 (from addition)
```

What is missing?

```

class Tensor:
    def __init__(self, value, requires_grad=False):
        self.value = value  # Store the value of the tensor
        self.grad = 0  # Gradient initialized to zero
        self.requires_grad = requires_grad
        self._backward = lambda: None  # Function to compute
        self._prev = set()  # Track previous nodes for backpropagation

```



We also need to define the backward step!

```

def backward(self):
    """Computes gradients using reverse-mode automatic differentiation."""
    # Initialize gradient to 1 if this is the final output node
    if self.grad == 0:
        self.grad = 1

    # Topological ordering of nodes using depth-first search
    topo_order = []
    visited = set()

```

```
def backward(self):
    """Computes gradients using reverse-mode automatic differentiation."""
    # Initialize gradient to 1 if this is the final output node
    if self.grad == 0:
        self.grad = 1

    # Topological ordering of nodes using depth-first search
    topo_order = []
    visited = set()

    def build_topo(node):
        if node not in visited:
            visited.add(node)
            for parent in node._prev:
                build_topo(parent)
            topo_order.append(node)

    build_topo(self)

    # Reverse iterate for backpropagation
    for node in reversed(topo_order):
        node._backward()
```

DFS-based topological sort.
It starts from self, recursively visits every dependency, and adds each node to topo only **after** all its dependencies have been added.

Let's run Backprop!!

```
# Creating tensors
x = Tensor(3.0, requires_grad=True)
y = Tensor(2.0, requires_grad=True)

# Forward pass
z = multiply(x, y) # z = x * y
w = add(z, y)      # w = z + y
o = relu(w)        # o = ReLU(w)

# Backpropagation
o.backward()

# Print gradients
print(f"x.grad: {x.grad}") # Should be y.value if ReLU is active
print(f"y.grad: {y.grad}") # Should be x.value + 1 (from addition)
```

Question: draw the computation graph of this program!

Question: explain how "backward()" gets executed!

Question: At what point is the topological sort triggered?

Question: how can you further extend this program?

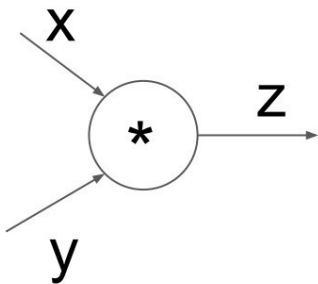


Auto-diff in PyTorch



PyTorch's Implementation: Forward/Backward API

- PyTorch uses the same separation between forward computation and backward derivative rules.
- An operation can save values during forward execution and reuse them when the backward pass receives an upstream gradient.
- Example: multiplication operator



```
class Multiply(torch.autograd.Function):  
    @staticmethod  
    def forward(ctx, x, y):  
        ctx.save_for_backward(x, y)  
        z = x * y  
        return z  
    @staticmethod  
    def backward(ctx, grad_z):  
        x, y = ctx.saved_tensors  
        grad_x = y * grad_z # dz/dx * dL/dz  
        grad_y = x * grad_z # dz/dy * dL/dz  
        return grad_x, grad_y
```

Need to cash some
values for use in
backward

Upstream
gradient

Multiply upstream
and local gradients

PyTorch Operators

- The framework supplies a large collection of operations, derivative rules, and optimized execution backends.
- We compose these operations in Python.
- While much of the expensive computation runs in optimized GPU code such as CUDA kernels and cuBLAS/cuDNN on NVIDIA GPUs.
- All these are implemented in C++ for performance but exposed in Python.

 mul_scalar_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 nchw_to_image_glsli	[vulkan] Enable 2D texture types (#86971)	2 months ago
 nchw_to_image2d_glsli	[vulkan] Enable 2D texture types (#86971)	2 months ago
 nchw_to_image_int32_glsli	[Vulkan] Enable copying QInt8 and QInt32 tensors from cpu to vulkan. (#...)	last month
 nchw_to_image_int8_glsli	[Vulkan] Enable copying QInt8 and QInt32 tensors from cpu to vulkan. (#...)	last month
 nchw_to_image_uint8_glsli	[Vulkan] Enable copying QInt8 and QInt32 tensors from cpu to vulkan. (#...)	last month
 permute_4d_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 quantize_per_tensor_qint32_glsli	[Vulkan] Enable QInt8 and QInt32 quantization (#89788)	last month
 quantize_per_tensor_qint8_glsli	[Vulkan] Enable QInt8 and QInt32 quantization (#89788)	last month
 quantize_per_tensor_quint8_glsli	[Vulkan] Enable QInt8 and QInt32 quantization (#89788)	last month
 quantized_add_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_conv2d_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_conv2d_dw_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_conv2d_pw_2x2_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_div_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_mul_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_sub_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 quantized_upsample_nearest2d_glsli	[Vulkan][TCC] Fix quantized shaders (#89456)	last month
 reflection_pad2d_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 replication_pad2d_glsli	[vulkan] replication_pad2d_glsli: use clamp() instead of min(max()) (#...)	7 months ago
 select_depth_glsli	[Vulkan] Implement select.int operator (#81771)	5 months ago
 sigmoid_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 sigmoid_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 slice_4d_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 softmax_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year
 stack_feature_glsli	[Vulkan] Implement Stack operator (#81064)	5 months ago
 sub_glsli	[Vulkan] Implement arithmetic ops where one of the arguments is a ten...	5 months ago
 sub_glsli	[Vulkan] Implement arithmetic ops where one of the arguments is a ten...	5 months ago
 tanh_glsli	[vulkan] Clamp tanh activation op input to preserve numerical stabili...	10 months ago
 tanh_glsli	[vulkan] Clamp tanh activation op input to preserve numerical stabili...	10 months ago
 threshold_glsli	[vulkan] fix some broken tests in vulkan_api_test (#80962)	6 months ago
 upsample_nearest2d_glsli	[vulkan] Add image format qualifier to glsl files (#69330)	last year

Example Activation Functions

- The framework supplies a large collection of operations, derivative rules, and optimized execution backends.
- We compose these operations in Python.
- While much of the expensive computation runs in optimized GPU code such as CUDA kernels and cuBLAS/cuDNN on NVIDIA GPUs.
- All these are implemented in C++ for performance but exposed in Python.

master ▾ [pytorch](#) / [aten](#) / [src](#) / [ATen](#) / [native](#) / [vulkan](#) / [glsl](#) / [tanh.glsl](#)

 SS-JIA [vulkan] Clamp tanh activation op input to preserve numerical stabili... 

🔍 2 contributors  

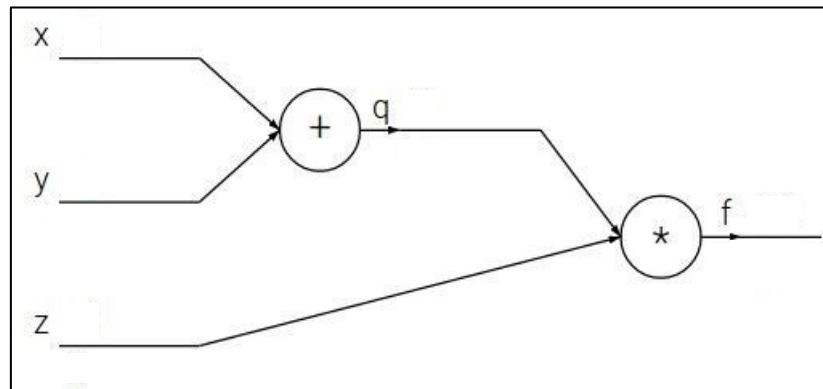
27 lines (21 sloc) | 777 Bytes

```
1  #version 450 core
2  #define PRECISION $precision
3  #define FORMAT    $format
4
5  layout(std430) buffer;
6
7  /* Qualifiers: layout - storage - precision - memory */
8
9  layout(set = 0, binding = 0, FORMAT) uniform PRECISION restrict writeonly image3D uOutput;
10 layout(set = 0, binding = 1)          uniform PRECISION                      sampler3D uInput;
11 layout(set = 0, binding = 2)          uniform PRECISION restrict              Block {
12     ivec4 size;
13 } uBlock;
14
15 layout(local_size_x_id = 0, local_size_y_id = 1, local_size_z_id = 2) in;
16
17 void main() {
18     const ivec3 pos = ivec3(gl_GlobalInvocationID);
19
20     if (all(lessThan(pos, uBlock.size.xyz))) {
21         const vec4 intex = texelFetch(uInput, pos, 0);
22         imageStore(
23             uOutput,
24             pos,
25             tanh(clamp(intex, -15.0, 15.0)));
26     }
27 }
```

Backprop in PyTorch

$$f(x, y, z) = (x + y)z$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



```
import torch

# Define tensors with requires_grad=True to track gradients
x = torch.tensor(-2.0, requires_grad=True)
y = torch.tensor(5.0, requires_grad=True)
z = torch.tensor(-4.0, requires_grad=True)

# Define the computation graph
f = (x + y) * z  # f = ( -2 + 5 ) * (-4)

# Perform backpropagation
f.backward()
```

Full model training in PyTorch

```
import torch
from torch import nn

# Model: 20 input features, 64 hidden units, 3 classes
model = nn.Sequential(
    nn.Linear(20, 64),
    nn.ReLU(),
    nn.Linear(64, 3),
)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

model.train()                                # Enable training behavior
for epoch in range(10):                      # One epoch visits the dataset
    for x, target in train_loader:
        optimizer.zero_grad()                # Clear previous gradients
        logits = model(x)                    # Forward pass
        loss = criterion(logits, target)
        loss.backward()                       # Compute parameter gradients
        optimizer.step()                     # Update parameters
```

Why Learn All These Internals?

- **Modern deep learning frameworks compute gradients for you!**
- “But why take a class on compilers when they are implemented for you?”
 - Understanding what is going on under the hood is useful!
- Backpropagation doesn’t always work perfectly out of the box.
 - Understanding why is crucial for debugging and improving models

Summary

- Modern deep learning libraries such as PyTorch implement a vast library of operations to allow automatic and efficient Backprop.
- We will make extensive use of PyTorch in this class (yay!)
- **Next:** We will discuss a few practical considerations regarding training NNs.

Practical considerations for training neural nets

torch.eval() vs. torch.no_grad() vs. torch.zero_grad()

	<code>model.eval()</code>	<code>torch.no_grad()</code>	<code>optimizer.zero_grad()</code>
Mechanism	Sets <code>self.training = False</code> recursively on the module and all submodules	Sets grad-mode off for the block; ops inside don't attach <code>grad_fn</code> , so no gradient graph is built	Sets <code>.grad</code> to <code>None</code> (or 0, if <code>set_to_none=False</code>) for every parameter the optimizer tracks
What change?	Dropout checks <code>self.training</code> internally and branches its forward pass on it	Autograd's engine checks <code>torch.is_grad_enabled()</code> before recording each op	[same as above]
What happens if you forget it?	Dropout stays active during inference → outputs get randomly zeroed, adding noise	Graph still gets built for unused backward passes → correct output, wasted memory/compute	New gradients get added to old ones on the next <code>backward()</code> → silent accumulation of gradients which can corrupt training

torch.eval() vs. torch.no_grad() vs. torch.zero_grad()

- Three operations that tend to appear together. But how are they different?

python

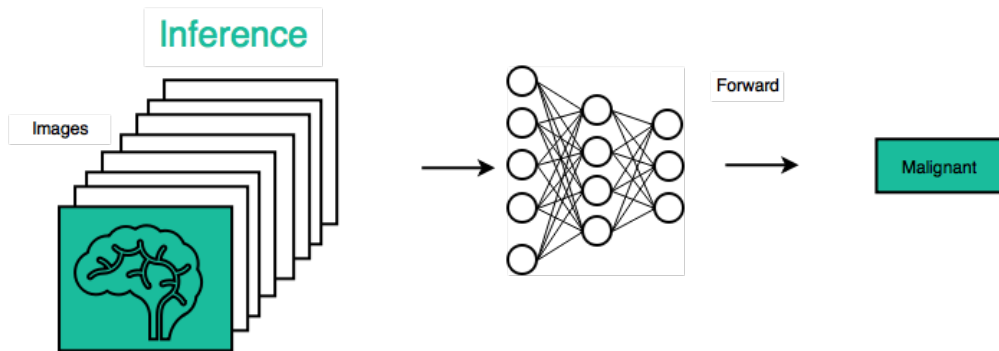
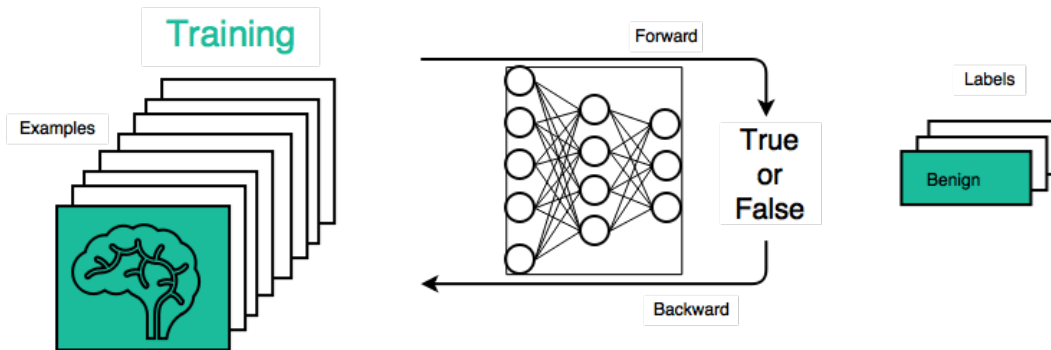
```
model.eval()                # switch Dropout/BatchNorm to inference mode
with torch.no_grad():        # stop building the autograd graph
    preds = model(x)
    loss = criterion(preds, y)
# no zero_grad() needed here - no backward() call, so no gradients were computed
```

python

```
model.train()                # switch back to training mode
for x, y in dataloader:
    optimizer.zero_grad()    # clear gradients from the previous step
    preds = model(x)
    loss = criterion(preds, y)
    loss.backward()          # computes gradients (graph was tracked, since no no_g
    optimizer.step()         # updates weights using those gradients
```

Batching

- GPUs are **fast with Tensor operations**
- This often improves hardware utilization and throughput on GPUs.



Batches of Data: Example

- The case of natural language:

- Each word (token) is mapped to a vector $\mathbb{R}^d$

$d = 1$

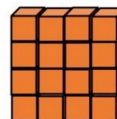


Vector

(1 axis)

- Then, each sentence of length ℓ is mapped to a matrix $\mathbb{R}^{\ell \times d}$

$d = 2$



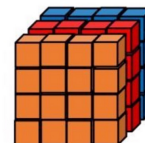
Matrix

(2 axes)

- A batch of sentences (size b) is mapped to a tensor $\mathbb{R}^{b \times \ell \times d}$

- (variable-length sequences require padding and masking, or packing – more on this later)

$d = 3$



Third-order tensor

(3 axes)

Data of any dimension in PyTorch

- PyTorch makes it easy to batch data.
 - Many PyTorch operations support batched inputs.
 - For example, you can create any tensor of **any** dimension.

TORCH.RAND

```
torch.rand(*size, *, generator=None, out=None, dtype=None, layout=torch.strided, device=None, requires_grad=False, pin_memory=False) → Tensor
```



Returns a tensor filled with random numbers from a uniform distribution on the interval $[0, 1)$

The shape of the tensor is defined by the variable argument `size`.

Parameters

size (*int...*) – a sequence of integers defining the shape of the output tensor. Can be a variable number of arguments or a collection like a list or tuple.

You can define tensors of any dimension

Batches of Data, In Practice

- Avoid loops, use tensors.

```
import torch

def matmul(A, B):
    C = A.new_zeros((A.size(0), B.size(1)))
    for i in range(A.size(0)):
        for j in range(B.size(1)):
            for k in range(A.size(1)):
                C[i, j] += A[i, k] * B[k, j]
    return C

# Example usage:
A = torch.randn(10, 10)
B = torch.randn(10, 10)
C = matmul(A, B)
```

```
import torch

# Example usage:
A = torch.randn(10, 10)
B = torch.randn(10, 10)
C = torch.matmul(A, B)
```

Batches of Data, in practice

- **Convention:** the batch dimension is typically the first (outermost) dimension of a tensor.
- **Example 1:** Batch of token IDs given to a model:

```
# Text batch: 32 sequences, each 512 tokens long
input_ids = torch.randint(0, 50000, (32, 512))

input_ids.shape
# torch.Size([32, 512])
```

- **Example 2:** Batch of token IDs given to a model:

```
# Transformer hidden states
# 32 sequences × 512 tokens × 768-dimensional representation
hidden = torch.randn(32, 512, 768)

hidden.shape
# torch.Size([32, 512, 768])
```

Element-wise broadcasting

- Broadcasting lets operations work on tensors of **different shapes** by **implicitly expanding the smaller tensor**.

- Take this code:

```
y = x + bias
```

- We know this would work if:

```
bias = torch.tensor([[b1, b2, b3],  
                    [b1, b2, b3],  
                    [b1, b2, b3],  
                    [b1, b2, b3]])  
x = torch.tensor([[x11, x12, x13],  
                 [x21, x22, x23],  
                 [x31, x32, x33],  
                 [x41, x42, x43]])
```

- The following also works. Broadcasting implicitly interprets the following like the above:

```
bias = [b1, b2, b3]    x = same definition ...
```

Element-wise broadcasting: rules

- PyTorch compares dimensions from right to left.
 - Two dimensions are compatible (for broadcasting) if **same size or one is 1**.
 - Missing leading dimensions are treated as size 1.

▪ **Q1:** Are these tensors compatible?

▪ **A1:** yes! Because 768 vs 768

```
[32, 512, 768]
[768]
```

▪ **Q2:** Are these tensors compatible?

▪ **A2:** yes! Because 768 vs 768 and 512 vs 1.

```
[32, 512, 768]
[1, 768]
```

▪ **Q3:** Are these tensors compatible?

▪ **A3:** no! Because 768 vs 512.

```
[32, 512, 768]
[1, 512]
```

- (it would have worked if it was [512, 1])

Element-wise broadcasting matters for batching

- Take this familiar operation:

$$y = x + \text{bias}$$

- This obviously works if you have one example:

```
x.shape # [768]  
bias.shape # [768]
```

- Now suppose you have **a batch** of examples in x:

```
x.shape # [32, 768]  
bias.shape # [768]
```

- Still works!**

- Broadcasting allows the same operation written for one example to automatically apply across an entire batch, as long as the tensor shapes are compatible.*

Broadcasting: cautionary example

- Suppose a batch has 3 examples.
 - You have one prediction and one target for each.

```
predictions = torch.tensor([[10.],  
                             [20.],  
                             [30.]]) # shape [3, 1]  
  
targets = torch.tensor([11., 22., 33.]) # shape [3]
```

- You want one error per example:
-1, -2, -3.

```
errors = predictions - targets
```

- **But you get this:**
- Why?

```
tensor([[ -1., -12., -23.],  
        [  9.,  -2., -13.],  
        [ 19.,   8.,  -3.]])
```

- Broadcasting aligns dims from the right. The missing leading dim of targets is treated as 1.

```
predictions: [3, 1]  
targets:      [ 3] → [1, 3]  
result:       [3, 3]
```

What operations accept element-wise broadcasting?

- All **elementwise** operations accept broadcasting.
 - **Arithmetic operators:**
 - `+`, `-`, `*`, `/` (element-wise multiply)
 - `**` (power), `%` (mod), `//` (floor division)
 - **Comparisons** (return a boolean tensor)
 - `>`, `<`, `>=`, `<=`, `==`, `!=`
 - **Logical / bitwise ops** (on boolean tensors)
 - `&` (and), `|` (or), `^` (xor)
 - **Elementwise functions**
 - `torch.maximum(x, y)`, `torch.minimum(x, y)`
 - `torch.where(condition, a, b)`
 - `torch.pow`, `torch.remainder`
- There are other forms of broadcasting too.

Broadcasting: the big picture

- What we just saw is the **element-wise** broadcasting.
- There are other forms of broadcasting as well.

Class	What gets broadcast	Rule applied to	Example
1. Elementwise	The two operand tensors	Entire shape	$x + \text{bias}$, $x * \text{scale}$, $x > 0$
2. Batch (matmul)	Only the leading batch dims	Shape minus last 2 dims	$a @ b$ with $a: [32, 10, 3, 4]$, $b: [10, 4, 5]$
3. Indexing	The index arrays	Shapes of the indices	$x[\text{rows}, \text{cols}]$ where rows, cols have different shapes

Beware of GPU memory limits

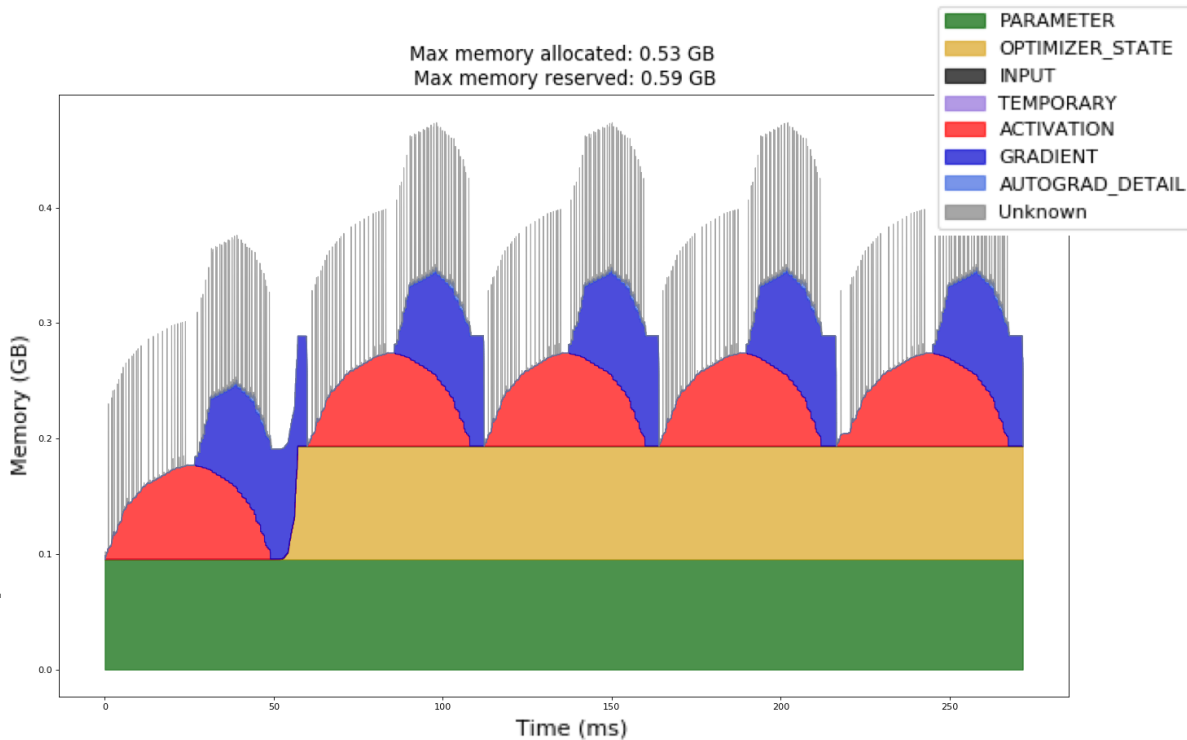
- Your GPU memory determines the limits of what you can process.
- During **inference** (indicated via `torch.no_grad()` and `torch.eval()`) these take up space:
 - Loading model **parameters**.
 - A fixed size, loaded as soon as you load the model.
 - **Activations** for a given input, computed during the forward pass.
 - **Larger batches** take up **more space** for activations.
 - Activations are released as soon as no computation needs them.
 - You get memory spikes every time that you start a forward pass, until its memory is released.
- This was during inference. At training time, you have more things.

Beware of GPU memory limits

- What else takes up space in GPUs during **training**?
 - **Activations** during the forward step.
 - While during inference intermediate activations can be released immediately, during training most activations need to be saved for backward computation.
 - After their corresponding backward operations finish, then activations get released.
 - This is why forward process during BP takes up a lot more memory during training.
 - **Gradients** are computed during backward (used by the optimizer to update parameters).
 - They're **not** released automatically.
 - Need to be cleared **explicitly** (zero_grad command in PyTorch).
 - Modern optimizers like Adam/AdamW/Muon (not plain SGD) maintain **optimizer states** (momentums) that needs to be kept in GPU memory.
 - Optimizer states are typically **initialized during the first optimizer step and retained across training iterations**.
 - So, the less optimizable params you have, the less memory your optim. will take up.

Beware of GPU memory limits

- The peak memory consumptions changes over time.
- If the peak exceeds your GPU limit, you'd get an **OOM** (out of memory) error.
- **Params** consistently take up memory.
- Same also for **optimizer states**, typically at the first optimizer step.
- Gradients are computed during backward, used by the optimizer to update parameters.

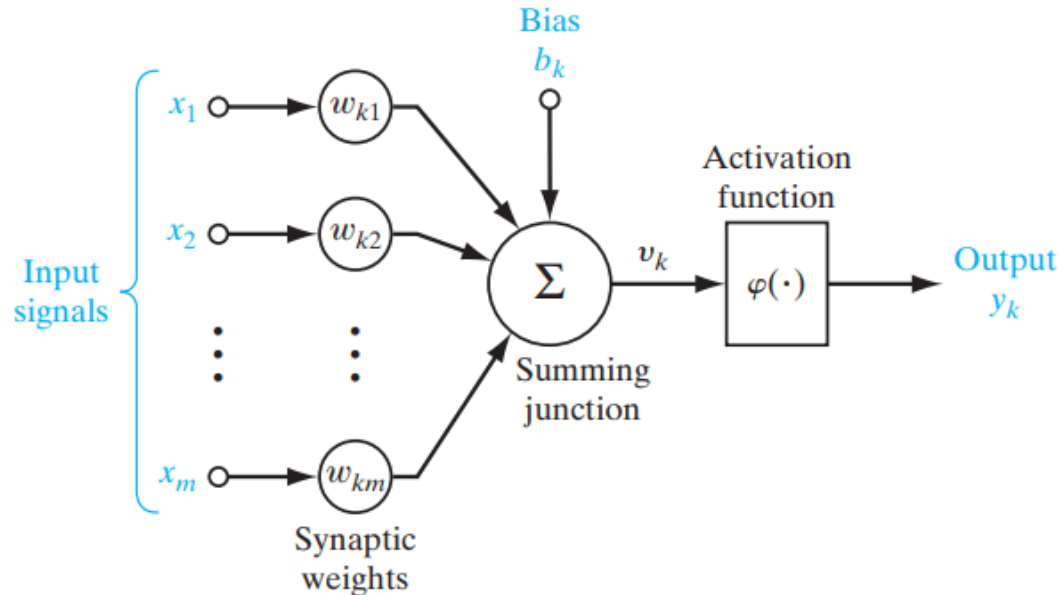


Possible responses to OOM

- **Quick fixes (no code change to the model):**
 - **Reduce batch size** -> directly shrinks activations, the biggest variable cost.
 - **Reduce input (sequence) length** -> activation memory scales with sequence length too
 - **Use `torch.no_grad()` / `model.eval()` wherever you're not training** -> skips storing activations needed for backward.
- **Trade compute for memory:**
 - **Gradient accumulation** -> simulate a large batch by running several small forward/backward passes and only calling `optimizer.step()` + `torch.zero_grad()` every N steps; same effective batch size, much lower peak memory.
 - **Gradient checkpointing (activation checkpointing)** -> don't store activations from the forward pass; recompute them during backward instead. Classic memory-for-compute trade.
- **Other options that we will discuss later:**
 - Mixed-precision, distributed training, CPU offloading of the optimizer states.

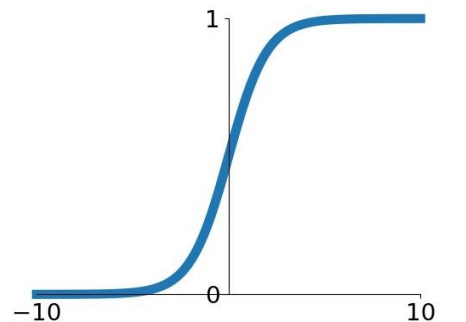
Which “activation function” should we use?

- In general, it is problem-specific and might require trial-and-error.
- Here are some tips about popular activation functions.

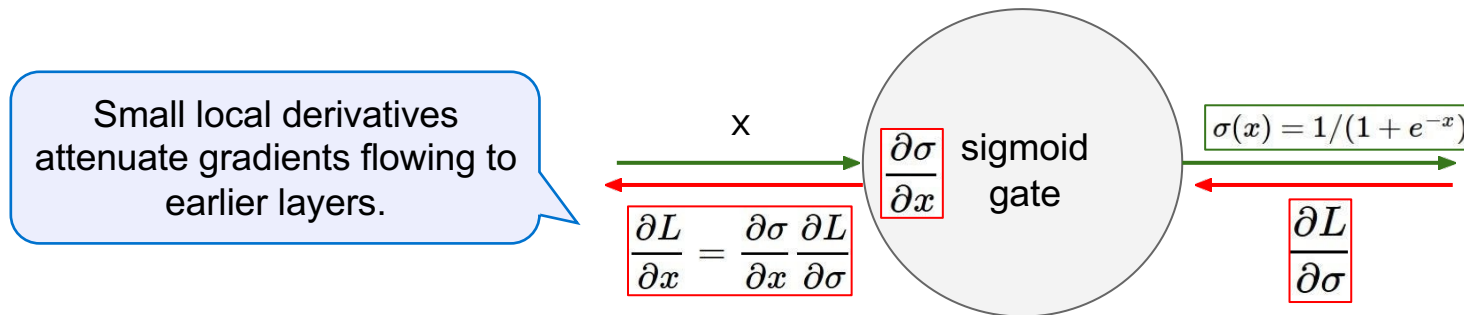


Activation Functions: Sigmoid

- Squashes numbers to range (0,1)
- Historically popular, interpretation as “firing rate” of a neuron
- **Limitation 1:** Saturated neurons “kill” the gradients
 - At $x=5$ the derivative is ~ 0.00665 .
 - Whenever $|x| > 5$, the gradients are basically zero.
- **Limitation 2:** Not centered around zero.

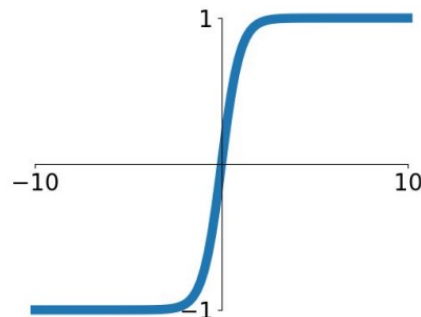


$$\sigma(x) = 1/(1 + e^{-x})$$



Activation Functions: Tanh

- Output range around $(-1, 1)$
- Effectively saturates $|x| > 3$ and “kill” the gradients
- Compared with sigmoid, tanh’s outputs can be closer to zero-centered, which can make optimization faster.
 - Tanh produces both positive and negative outputs, whereas sigmoid’s outputs are always positive.

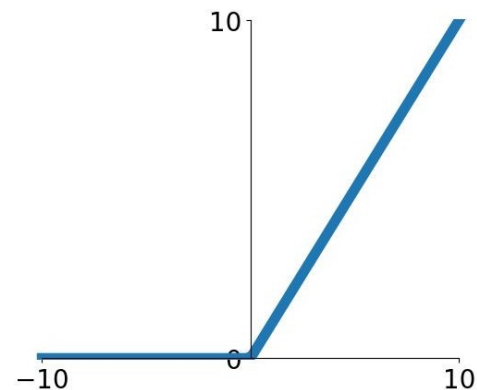


$\tanh(x)$

[LeCun et al., 1991]

Activation Functions: ReLU

- Computationally efficient
- In practice, converges faster than sigmoid/tanh
- Does not saturate (in +region) — will die less!

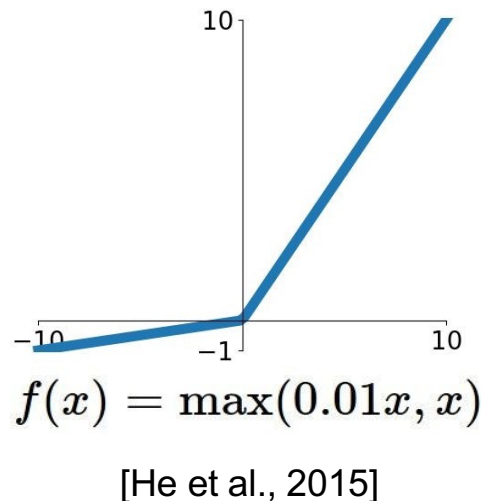


ReLU
(Rectified Linear Unit)

[Krizhevsky et al., 2012]

Activation Functions: Leaky ReLU

- Nonzero negative slope avoids ReLU's zero derivative on the negative branch (gradients do not die!)
- Computationally efficient
- In practice it converges faster than sigmoid/tanh.
- Other variants:
 - Parametric Rectifier (PReLU):
$$f(x) = \max(\alpha x, x)$$
 - More flexibility at the cost of more parameters.

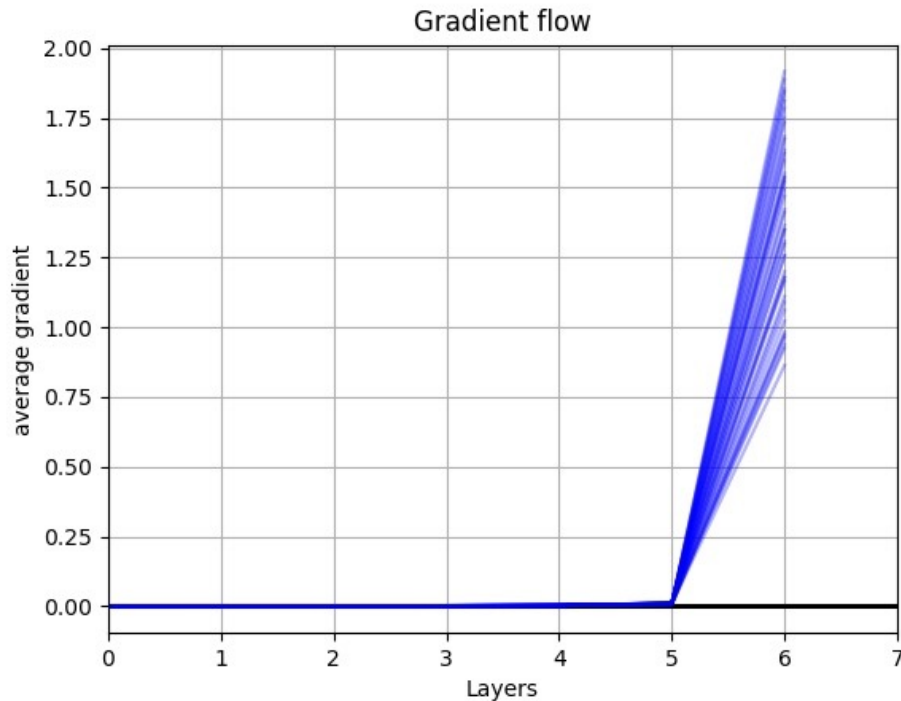


Choose Activations: In Practice

- Other activations that we didn't cover: GELU, Swish, SwiGLU, Softplus,
- In general, it is problem-specific and might require trial-and-error.
- A useful recipe:
 1. Generally, ReLU is a good activation to start with.
 2. Time/compute permitting, you can try other activations to squeeze out more performance.

Exploding/Vanishing Gradients

- If many numbers $|x| > 1$ get multiplied, the result will become **too big**.
- NaN gradients --> no learning!
- If many numbers $|x| < 1$ get multiplied, the result will become **too small**.
- Zero gradients -> no learning!

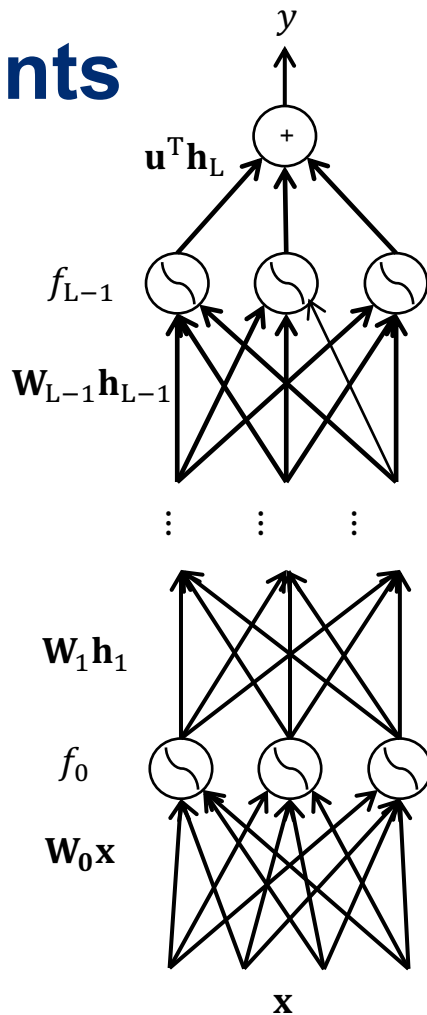


Exploding/Vanishing Gradients

- Remember gradient computation at layer $L - k$:

$$\nabla_{\mathcal{L}}(\mathbf{W}_{L-k}) = \underbrace{\left(\mathbf{J}_{\ell}(y) \mathbf{J}_y(\mathbf{h}_L) \mathbf{J}_{\mathbf{h}_L}(\mathbf{h}_{L-1}) \mathbf{J}_{\mathbf{h}_{L-1}}(\mathbf{h}_{L-2}) \dots \mathbf{J}_{\mathbf{h}_{L-k+1}}(\mathbf{W}_{L-k}) \right)^T}_{O(k)\text{-many matrix multiplication}}$$

- Repeated Jacobian products** can **amplify** or **attenuate** gradients.

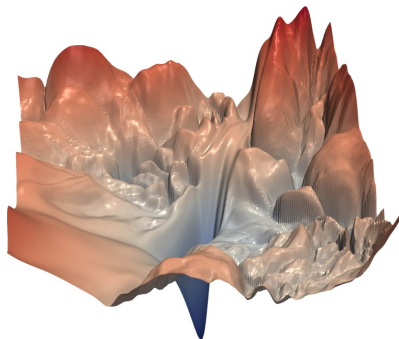
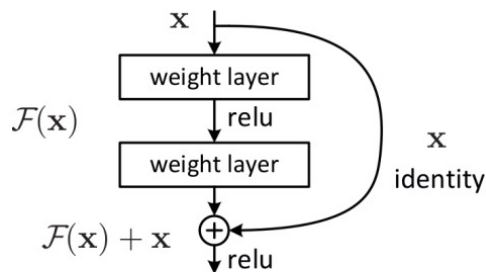


Exploding/Vanishing Gradients

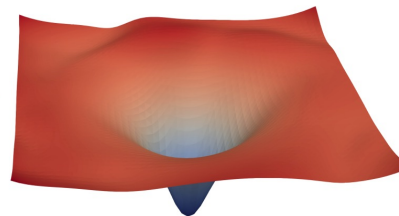
- We will see three complementary mechanisms for addressing this problem:
 1. Residual connections
 2. Weight initialization
 3. Normalization layers
- These are complementary elements.
 - 2 and 3 control the magnitude of activations. 1 controls the flow of information between layers.

Residual Connections/Blocks

- Without residual connection: $\mathbf{y} = F(\mathbf{x})$
- With residual connection: $\mathbf{y} = F(\mathbf{x}) + \mathbf{x}$
- Interpretation:**
 - Identity (direct connection) is the default.
 - A layer must *learn* to pass information through.
 - If the model benefits from depth, it will adaptively learn to use it.
 - Gradients get an undamped path; it's Jacobian now is $\mathbf{J}_F(\mathbf{x}) + \mathbf{I}$ (*identity*)
 - One can also interpret this as a smoother loss landscape



(a) without skip connections

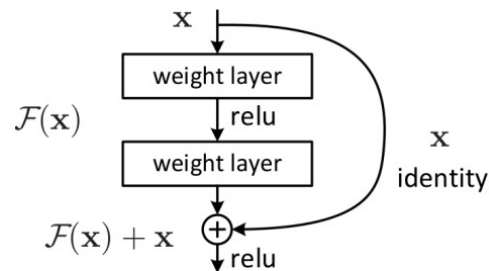


(b) with skip connections

Residual Connections: Example

Define a simple residual block with a skip connection

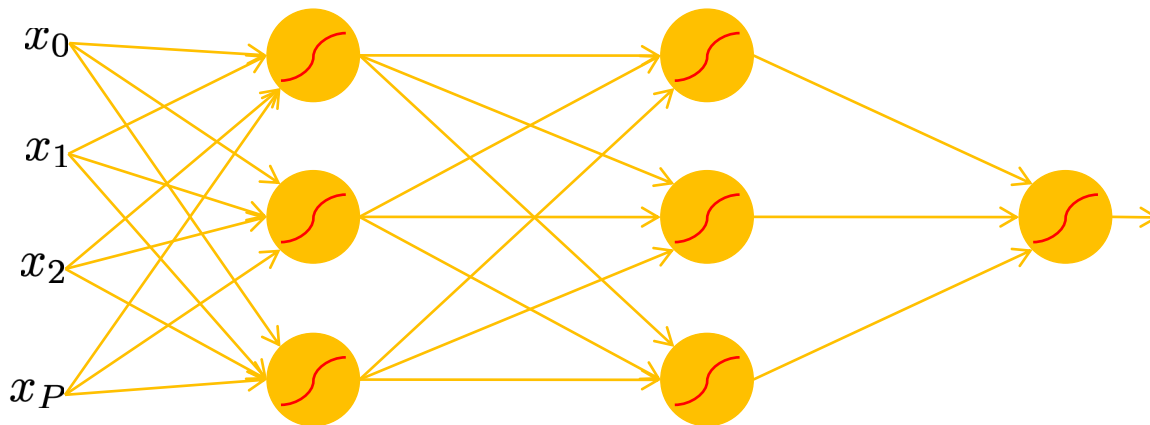
```
class SimpleResidualBlock(nn.Module):  
    def __init__(self, input_dim):  
        super().__init__()  
        self.linear1 = nn.Linear(input_dim, input_dim)  
        self.relu = nn.ReLU()  
        self.linear2 = nn.Linear(input_dim, input_dim)  
  
    def forward(self, x):  
        identity = x # Skip connection (input is added back)  
        out = self.linear1(x)  
        out = self.relu(out)  
        out = self.linear2(out)  
        out += identity # Residual connection  
        out = self.relu(out)  
        return out
```



Question: identify residual connections here

Importance of Weight Initialization

- Initializing all weights in a layer with a **fixed constant** (e.g., 0's) is a **bad idea!** (why?)



- If the neurons start with the same weights, then all the neurons will follow the same gradient, and will always end up doing the same thing as one another.
- Effective initialization is one that breaks such “symmetries” in the weight space.

Fix #2: Weight Initialization

- Better idea: initialize weights with random Gaussian noise.

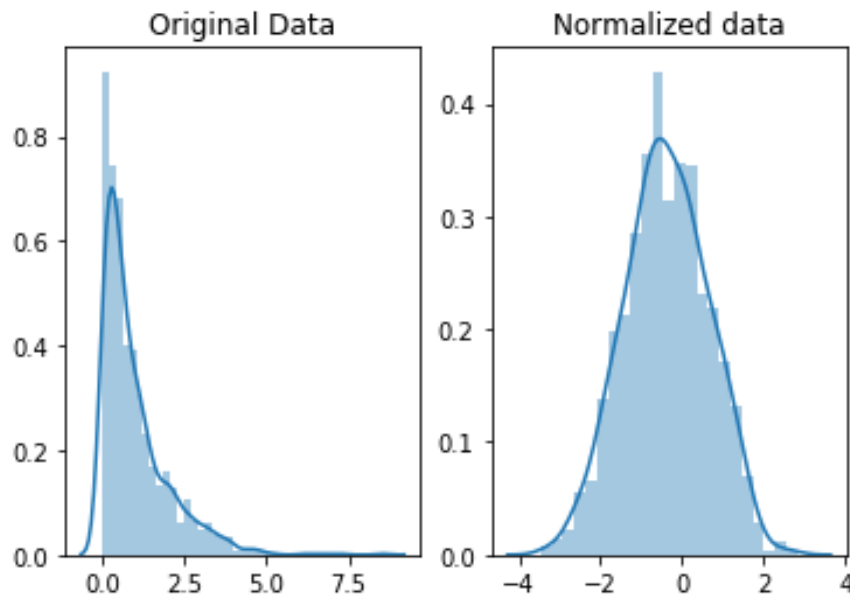
```
w = torch.empty(3, 5)  
nn.init.normal_(w)
```

- You can specify the std of the Gaussian (a hyper-parameter).
- Other alternatives:
 - **Xavier/Glorot initialization:**
 - Scales weights based on the number of input and output units so that the variance of activations stays roughly constant across layers.
 - **Kaiming initialization:**
 - Intended for ReLU-like activations.
 - It scales the initial weights so that, even after ReLU throws away half the values, the signal's variance stays the same layer after layer.

Fix #3: Normalize Your Data!

- We do not like very large numbers.
 - Large numbers lead to numerical problems (e.g., overflow) and lead to NaNs 😭
- We prefer if our data is distributed around zero.
- Estimate preprocessing statistics (mean and std) on the training set; reuse them on validation/test data.

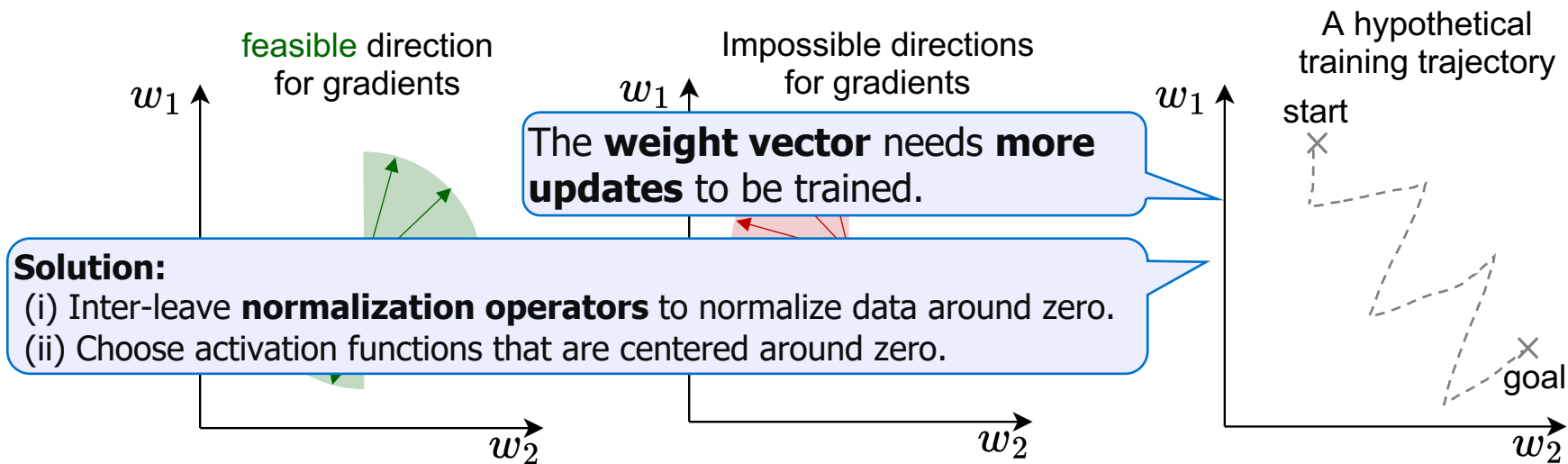
$$z = (x - \mu) / \sigma.$$



Non-zero-centered data case study: A single neuron training one training example

$$f = \mathbf{w}^\top \mathbf{x} + b \quad \Rightarrow \quad \frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial f} \frac{\partial f}{\partial w_i} = \text{upstream} \times x_i$$

- If data is always positive (i.e., $\forall i: x_i > 0$), all the dimensions of $\nabla_{\mathbf{w}} \mathcal{L}$ would have the same sign (all positive or all negative, same sign as **upstream**).

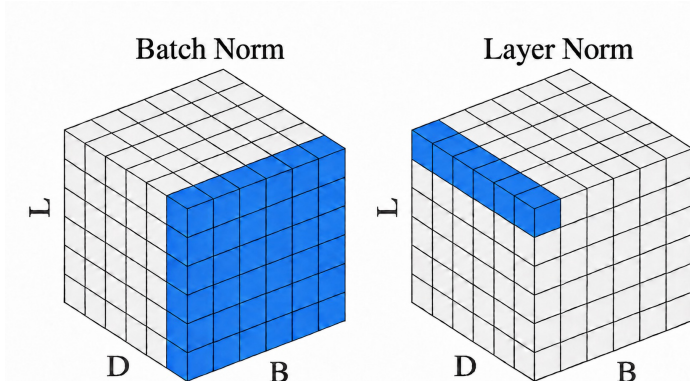


Normalization

γ and β are learned params;
 ϵ is for numerical stability.

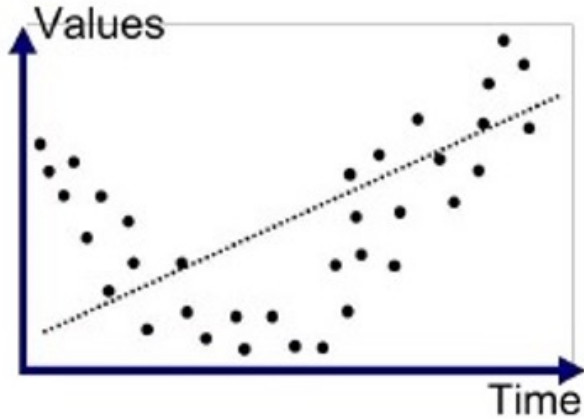
$$y = \frac{x - \mathbf{E}[x]}{\sqrt{\mathbf{Var}[x] + \epsilon}} * \gamma + \beta$$

- Normalization controls the scale of hidden activations
- **Benefits:**
 - Stabilizes optimization by preventing value disparities
 - Stable optimization => faster training
- **Naming:** Depending on how you apply it, it may have a different name.
 - **LayerNorm:** normalize the features within each token position and each item in batch.
 - **BatchNorm:** normalize for each feature over examples in the current minibatch.

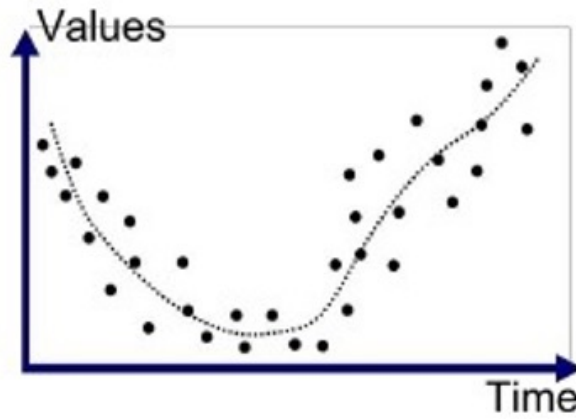


Overfitting

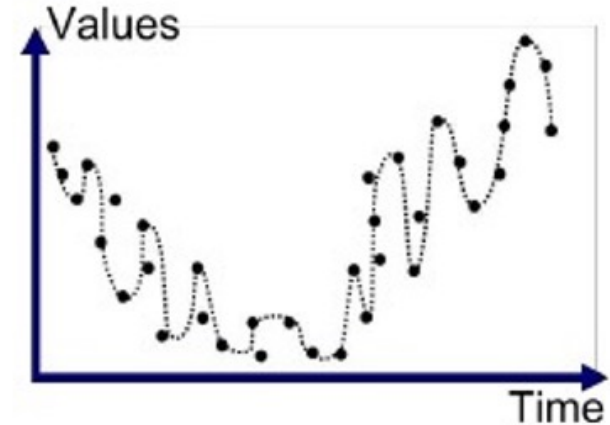
- Running too many epochs, a NN may learn the training data **too well**.
 - (in a way that would prevent generalization to held-out validation/test)



Underfitted



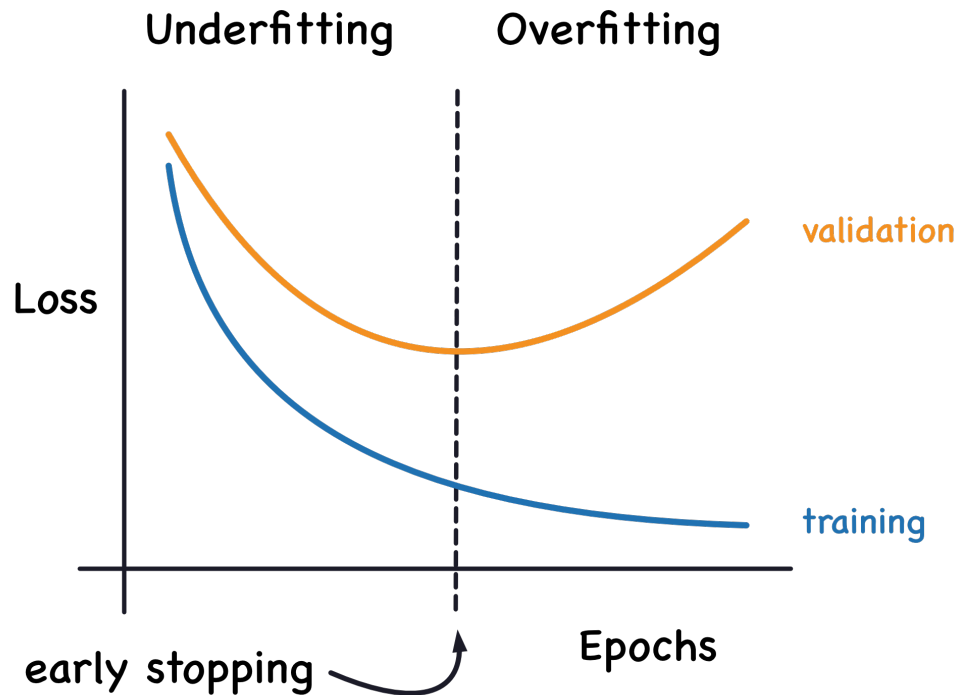
Good Fit/Robust



Overfitted

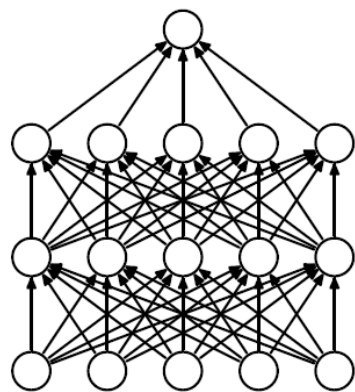
Preventing overfitting

- Keep a **held-out validation** set and evaluate validation loss after every epoch
- **Early stopping**: maintain weights for best performing network on the validation set and return it when performance decreases significantly beyond that.

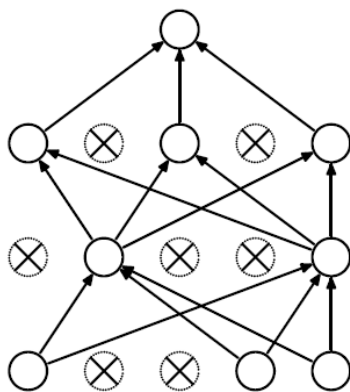


Dropout Training

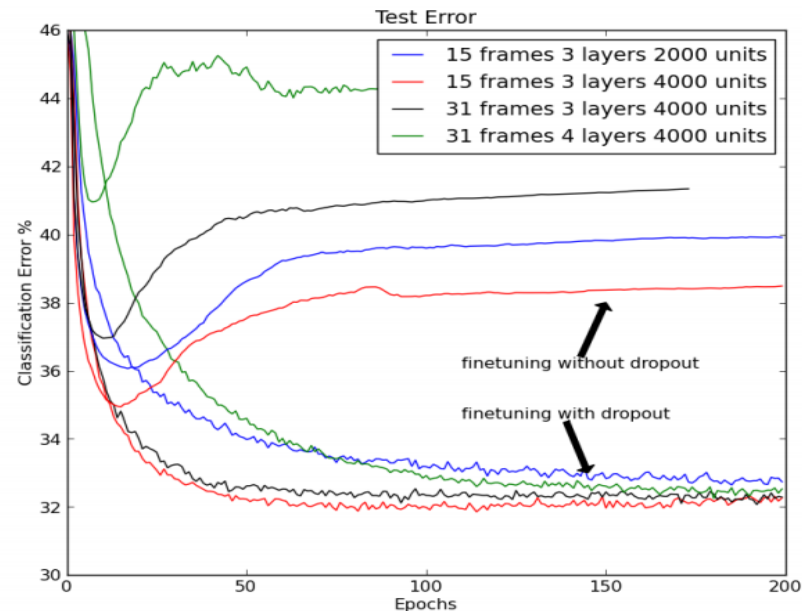
- In each forward pass, **randomly zero activation elements during training.**
- Probability of dropping is a **hyperparameter**; 0.5 is common.



(a) Standard Neural Net

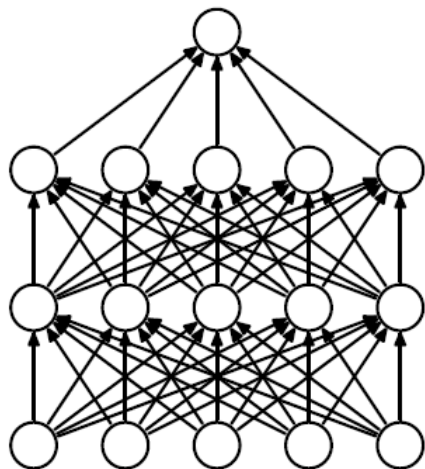


(b) After applying dropout.

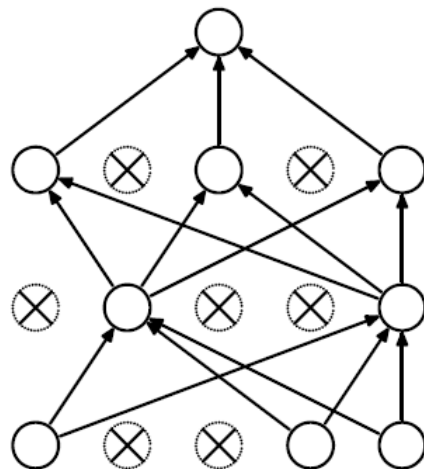


Dropout Training; intuition as an ensemble

- Dropout is **implicitly an ensemble** (average) of many models that share parameters.
 - Each binary mask is one model
 - For example, a layer with 4096 units has $2^{4096} \sim 10^{1233}$ possible masks!
 - Only $\sim 10^{82}$ atoms in the universe ...



(a) Standard Neural Net



(b) After applying dropout.

Dropout During Test Time

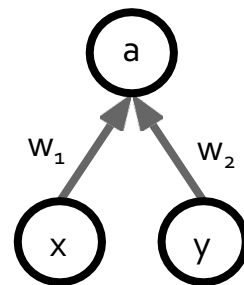
- What do we do at the **test** time?
 - Dropout **adds randomization**. ☹️
 - Each dropout mask would lead to a slightly different outcome.
- Without dropout, the input values to each neuron would be higher than what was seen during the training (**mismatch between train/test**).
- In ideal world, we would like to “average out” the outcome across all the possible random masks.
 - Not feasible since there are many possible networks.

Dropout During Test Time: Example

- Consider the following network: $w_1x_1 + w_2x_2$
- Suppose we apply dropout ($p=0.5$) to the following model.
- How should we use this model at the inference time?
- Think about training activations:

$$\begin{aligned} E[a] &= \frac{1}{4}(w_1x_1 + w_2x_2) + \frac{1}{4}(0 + 0) + \frac{1}{4}(0 + w_2x_2) + \frac{1}{4}(w_1x_1 + 0) \\ &= \frac{1}{2}(w_1x_1 + w_2x_2) \end{aligned}$$

- If we do not apply dropout: $E[a] = w_1x_1 + w_2x_2$
- **Solution:** Scale up the values proportional to dropout probability.
 - A very common interview question! 😊



Dropout in Practice

Just call the PyTorch function!

```
dropout = nn.Dropout(p=0.2)
x = torch.randn(20, 16)
y = dropout(x)
```

It automatically activates dropout for **training**.

```
# training step
...
model.train()
...
```

Deactivates it during **evaluations** and scales the values based on dropout param.

```
# evaluate model:
...
model.eval()
...
```

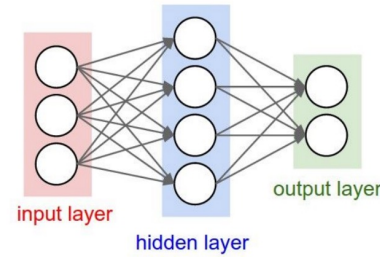
The Only Time You Want to Overfit: The First Tryout

- **Why?** Failure to fit a tiny dataset is a useful warning sign.
 - So. overfitting a small sample is a useful sanity check
 - This is really the first test you should do, before any hyperparameter tuning.
- **Procedure:** Try to train to 100% **training** accuracy on a few (<30) examples.
- **Monitor the signs of life:**
 - Does the loss go down? If not, something must be wrong.
 - Try checking the **learning rate** or modifying the initialization.
 - If those don't help, check the gradients.
 - If they're **NaN** or **Inf**, might indicate **exploding gradients**.
 - If they're **zeros**, might indicate **vanishing gradients**.

Final Comments on Training

- **No guarantee of convergence**; neural networks form non-convex functions with multiple local minima
 - To **avoid local minima**: several trials with different random initial weights with majority or voting techniques
 - Model scale also helps with optimization issues (loss closer to convex).
- May be tricky to set hyper-parameters (e.g., learning rate, number of layers). You need to set these with numerous trials and evaluation on held-out validation data.

Summary



- Feed-forward network architecture
 - But many of the concepts here hold for any architecture.
- Backprop := a general-purpose algorithm for efficient gradient calculations.
 - The most important algorithm in neural networks! 🎉
- Lots of empirical tricks for training neural networks:
 - Things to be careful about: over-fitting, activations, exploding/vanishing gradients, ...